

VECTOR

Finnish Forests VI plus ...

- | | |
|------------------------------------|-----|
| • Norman goes Jotting with Style | 41 |
| • Veli-Matti expounds Dfns | 50 |
| • Adrian takes a PDF Apart | 77 |
| • Michel Dumontier plays with 666 | 88 |
| • Stephen Taylor on Coding Clarity | 113 |
| • Cannon on Packages in Dyalog | 126 |



*The Journal of the
British APL Association*

ISSN 0955-1433

www.vector.org.uk

Vol.18 No.4 April 2002

A Specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2001-2002

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: apl385@compuserve.com.

Contents

		Page
Editorial	Stefano Lanzavecchia	3
APL NEWS		
Quick Reference Diary		7
Accepted Papers at APL2002		8
British APL Association News		
News from Sustaining Members		12
APL Product Guide	Gill Smith	14
The Education Vector		
Zark Newsletter Extracts	edited by Jon Sandles	29
Crossword		38
J-ottings 32: the Stylish Part of Vector	Norman Thomson	41
RECENT MEETINGS		
Report: Finnish Forests VI	Adrian Smith	48
Dynamic Functions in Practice	Veli-Matti Jantunen	50
Using OLE with Dyalog APL and Excel	Conrad Hoesle-Kienzlen	64
A PDF Primer for APLers	Adrian Smith	77
Report: APL Germany in Munich	Adrian Smith	85
GENERAL ARTICLES		
Playing with the Beast Number with J	Michel Dumontier	88
The Abstraction Mechanisms of Vector Pascal	Paul Cockshott	100
Three Principles of Coding Clarity	Stephen Taylor	113
TECHNICAL SECTION		
Hacker's Corner: Packages in Dyalog APL	Ray Cannon	126
Technical Correspondence	Norman Thomson	129
At Play with J: Second-order Josephus	Gene McDonnell	132
Hypergeometric Functions and CDFs in J	Ewart Shaw	139
Index to Advertisers		143

dyalog APL

for **.net**

Thanks to Dyadic's ground-floor involvement in Microsoft's .NET project, Dyalog.Net is fully integrated with .NET and gives you access to all its features, just like other .NET languages. Better still, **its APL!**

This is your chance to beat VB programmers at their own game!

- Fully integrated with .NET class libraries
- Write new .NET class libraries in APL
- Publish APL web pages, integrated with IIS
- Implement XML Web Services in APL
- Call XML Web Services from APL
- Includes APLScript scripting language
- Save workspaces as DLLs and .EXEs
- and lots more ...

Dyalog.Net

<http://www.dyadic.com>

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
Hants. RG24 7AL, United Kingdom.

Tel: +44 1256 811125 Fax: +44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

Editorial

by Stefano Lanzavecchia

"With every gust of wind,
the butterfly changes its place
on the willow."

Basho (1644 - 1694)

On the official APL and J *fora*, with a regularity that is almost frightening, somebody starts complaining about the lack of the traditional APL symbols in the J programming language. With the same regularity people also start wondering about the possibility of spreading the knowledge of APL using schools and students as the main vessel. And sooner or later the discussion crashes into the problems carried by the unusual symbol set. As frequent as the two just mentioned is the recurring quest for a good way to exchange APL snippets via e-mail, web pages or other forms of simple electronic communication. On one side stand the conservative promoters of the idea that 7 bits were fine all this time and it would be a sin to give into anything different like, for instance, HTML encoding, therefore ruling out the possibility of direct inclusion of the symbols. On the other side are those who claim that with all the bandwidth available nowadays one could even use high resolution bitmaps if it was easy.

At the base of all the arguing there is the undeniable fact that APL seems to be intrinsically linked to its, most would define, beautiful symbol set and that the Anglo-centric bias of computing has always limited the most common widespread interfaces to the 26 letters of the extended Latin alphabet (doubled by the distinction lowercase/uppercase) coupled with a handful of punctuation signs (dot, comma, colon, semicolon and a few more) and the 10 Arabic digits. In this clean and simple world, not only the fancy *domino* or the architectural *delta-underbar* are a bit of a nuisance, but even the simpler *rho* and *iota* end up being a serious challenge.

When J was designed from the ground up, the basic concept of using simple symbolic sequences instead of tedious keywords was maintained but instead of relying on exotic glyphs, the creators decided to use standard Latin signs and combinations of these based on simple rules. I guess that the rationale behind this choice is that the power of APL is to be found not only in its array manipulation capabilities, but also in its conciseness, due to the use of powerful symbolic primitives. Since the number of digrams and trigrams allowed by J's lexing rules

is much larger than the APL character set, J has a set of primitives larger in the same measure. By the way, this richness of primitives, in my opinion, is the main technical obstacle to the creation of a J front-end based on one-character glyphs. Nevertheless I will show later that a large symbol set is not an obstacle by itself.

Arthur Whitney's K went in a similar but somewhat different direction: K is also based on an ASCII representation, but the number of symbols required is not larger than APL's. In fact it's smaller, since the entire set fits almost completely in single character sequences, and I believe that it's only for simplicity and regularity that the symbols for *each-left* and *each-right* are digrams. This makes statements in K amongst the most concise ever, good or bad this may be. I am often confused by the way symbols in K are overloaded with meanings. In fact I am not one of the lucky elect ones who can claim to be able to read any J or K sentence and make sense of it at first glance, but I like to think that it's only lack of practice on my part and not intrinsic obscurity of the languages themselves.

Morgan Stanley's A+ is a strange site in the modern landscape: not only can one use the traditional APL symbols, but a slightly more cryptic digram ASCII-based mode is available as well as a fully keyword-based dictionary of primitives. The latter possibility shouldn't surprise since that's the way I learnt APL, in the mid 80's, on the glorious STSC interpreter, the first one available on IBM compatible Personal Computers. The lack of a monitor with graphical capabilities and of the ROM with the special APL character set didn't stop me. I thought that keywords were good enough and that the power of an array processing language could easily overcome the bother of having to type space-hash-i-s-space (reads " #is ") for an assignment, instead of the nicer *left-arrow*. To the eyes of a young teen-ager, the inefficient but breath-taking expression to produce the first n prime numbers based on the school definition (numbers with two divisors only, themselves and the unit) didn't look much less impressive simply because there were a few hash-prefixed keywords in between the variables' names. My impression is that the people behind SmartArrays (<http://www.smartarrays.com/>) are of the same opinion: the verbosity of the syntax is only marginally detrimental when it comes to harness the power of an array-oriented language. And, in fact, many modern languages are rediscovering array processing, disguised with different names and different syntactic tricks. Take Sather (older) and Ruby (newer), for instance, two relatively famous Object Oriented languages: don't their iterators remind you a lot of scans, reductions, and APL2's *each's*? But I am digressing. I will probably discuss the influence of array oriented problem solving in modern computing languages some other time.

What I wanted to concentrate on was the good points and the bad points of a language whose primitives are expressed as symbols. Let's open a book of an

advanced course in Calculus, or Quantum Mechanics, or Linear Algebra, or Relativity, or Fluid Dynamics or even Theory of Compilers and let's take a look at its pages. A non-prepared reader will probably be overwhelmed by the amount of symbols used. I remember that as a kid I used to stare in contemplation at some of the equations in a book of Gravimetric Theory I'd found at home. In my admiration there was the hope that one day I would know enough to be able to read them and understand them (as equations, not necessarily in the context of the theory), but I was also truly fascinated by their beauty. Those of us who had fun with Knuth's TeX know how painful and rewarding it is to typeset an equation. APL was designed to be more regular. Nevertheless the simple lack of the symbols on all the standard computer keyboards, combined with the 7-bit centric traditional view of electronic mail, makes APL scripts a difficult challenge when it comes to writing them, exchanging them, and reading them. Let's blame the Americans for once: their idea that 26 characters plus a few signs was enough to communicate is not only a barrier to the exchange of APL snippets, but also of messages composed in most of the other written human languages of the world. One of the first things one learns when writing Italian (or French) on the Internet is to avoid accented characters because on most (primitive and misconfigured) mail clients they are translated into rubbish, that prompts an immediate flame war. Too bad that in Italian and French accents are necessary. At the same time nobody must have told the creators of most computer languages that the tilde symbol is NOT on any Italian keyboard and that on laptops it's really hard to locate. Believe me when I say that it's easier to type APL than K on the keyboard of the laptop now under my hands.

But all this is laughable if we just raise our heads above the wall of our cities and let our gaze reach countries like China or Japan, without forgetting Egypt, India or Korea. I've just returned from a 3 weeks holiday in Japan, 3 weeks on my own to explore in complete freedom Tokyo and the surroundings. I left for Japan with the ability to read the two phonetic alphabets (52 symbols each) only to get the confirmation that, hadn't there been signs for foreign visitors kindly written in Latin alphabet, those 104 symbols aren't even enough to use the public transportation, I don't say to buy a ticket, but not even to find the right stop. The use of kanji pictograms is an essential part of the life of a Japanese person. A kid is supposed to learn some three or four hundred in the first couple of years at school, but a well-educated grown-up is likely to learn ten or twenty thousand in his life. My guess is that to read an average newspaper a couple of thousand kanjis are required. This many symbols are enough to encode all the primitives of all the computer languages existing, with space left to spare. And yet, it's as common as it is in Europe to see young people using their mobiles to exchange electronic messages, with a keyboard consisting of the ten standard digit buttons

plus a couple of function keys and 4 arrow keys. The difference between us and them is that they are composing messages building from a set of symbols in the thousand units. This certainly proves that where there is necessity, problems which seem to be technically challenging can be overcome with relative ease. But there is more. Japanese people seem to be so used to symbols that we Europeans find even their advertisements on the trains so packed with information to be almost disturbing. Untrained people often complain that APL is opaque because of the few extra symbols. My Japanese experience would lead me to conclude that this is just laziness due to the upbringing. Humans as such don't have any problem with handling symbols used for structured communications in quantities that I have trouble conceiving.

I am running out of space (and stamina) but I would like to add that I don't have solutions for the problems I have been listing in these paragraphs. Some think that Unicode-aware software will solve most of the issues connected with the encoding of exotic scripts, be it Japanese or APL. I have good reasons to doubt this claim, but I will discuss the facts some other time. For the time being I would recommend keeping in mind that even if English is more and more becoming a *de facto* standard communication language, there are more than a billion Chinese people who are beginning to use the Internet, that Chinese doesn't have a separate phonetic alphabet like Japanese, and that like most of us Westerners, they don't like to learn foreign languages but they are potential customers of an amazingly vast market.

Quick Reference Diary

Date	Venue	Event
22-25 July, 2002	Madrid, Spain	APL2002 Conference
November TBA	Naples, Florida	APL2000 User's Meeting

Dates for Future Issues of VECTOR

	Vol.19 No.1	Vol.19 No.2	Vol.19 No.3
Copy date	31st May	6th Sept	6th Dec
Ad booking	7th June	13th Sept	13th Dec
Ad Copy	14th June	20th Sept	20th Dec
Distribution	July 2002 (Madrid)	October 2002	January 2003

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

**Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.**

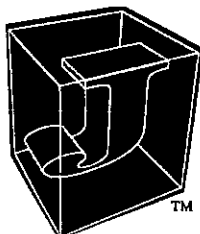
Please note that Vol.1 No.2 is now out of stock.

Jsoftware Inc.
19235 Covington Ct.
Shorewood, MN 55331
info@jsoftware.com
Tel (952) 470-7345
Fax (952) 470-9202

Announcing Jsoftware Version 5.01™

**Watch our web site for the most
advanced Jsoftware release
yet— version 5.01!**

- ♦ **Session Manager, window driver,
IDE, and GUI support in Unix and
Linux**
- ♦ **GUI event support now extended to
closely match VB and Java**
- ♦ **Improved online documentation**
- ♦ **Performance Improvements**
- ♦ **Online documentation available in
hardcopy books**
- ♦ **New debug facilities for Pro users**



www.jsoftware.com

APL2002 – MADRID

Accepted Papers

1. Alfonseca, Enrique: A Wordnet Interface to APL2
2. Barghoorn, Martin: Intelligent Quota Management on Windows
3. Bellini, Francesco; Anniballi, Antonio: Bingo Pricing: a Game Evaluation Using the Derivatives Approach
4. Brigham, Georges: Exhaustive Representation of Multiple Instances for Programming and Verification, and the use of Base-value Discriminants
5. Brigham, Georges: Evolution of an APL Electronic Document Interchange System
6. Brown, James A.: SmartArrays for the APL Programmer
7. Burchfield, Joseph; Lipovaca, Samir: Using APL Approach of Solving Programming Problems in Java
8. Chachaty, Claude: APL, a Powerful Research Tool in Magnetic Resonance Spectroscopy
9. Chastney, Phil: Formal Semantics of APL
10. Cockshott, Paul: Vector Pascal: an Array Language for Multimedia Code
11. Eriksson-Bique, Stephen: A New Calculus for Multidimensional Arrays
12. Halasz, Steven: Whom does the Grail Serve? Using APL for Semantic Filtering of Relational Data Access
13. Henriques, J.M.: A Table of the First 800 Million Prime Numbers with APL
14. Lucas, Jim: Out-of-control Structures (Observations and Thoughts on Control Structures in APL)
15. Lucas, Jim: Looking Back on Looking Ahead
16. McCormick, Devon: J and Dyalog APL: Can't We All Just Get Along? A Memory-mapped File Interface
17. Mansour, Stephen M.: User Defined Functions in a Database Application

-
18. Mitchell, David: APL: Delivering Power with Minimal Restraint
 19. Ortega, Alfonso; Sanchez-Alonso, Rafael; Alfonseca, Manuel: Automatic Composition of Music by Means of Grammatical Evolution
 20. Otto, Tilman: SAPL -- APL as Client & Server for SAP R2/R3
 21. Rey, Demetrio; Stubblefield, Joss; Canning, James: Porting the Parallel Array Programming Language ZPL to an Embedded Multicomputing System
 22. Saigusa, Kyosuke: Dialog Template Description Tool
 23. Skomorokhov, Alexander: Radial Basis Function Networks in A+
 24. Smith, Adrian: A+ for Windows
 25. Smith, Stuart: APL 'Wish Lists': The Nial Programming Language Revisited
 26. Surkan, Alvin J.; Khuskivadze, Amiran: Evolution of Circuit Hardware Configurations with APL Genetic Algorithm Simulations of Circuit Models of Quantum Algorithms
 27. Sykes, A.M.; Mayer, A.D.: Dynamic Graphic Regression
-

Registration Details

	Early Registration (until 31/May/02)	Late Registration (after 31/May/02)
Normal Fee	300 \$ 330 euro	325 \$ 360 euro
Members of ACM, SIGAPL or BAA	275 \$ 300 euro	300 \$ 330 euro
Students	115 \$ 125 euro	135 \$ 150 euro

The registration fee does not include the Conference Dinner nor the tourist activities.

Hotels

HOTEL NH NACIONAL ****

PASEO DEL PRADO, 48

Special Price for the Conference (a double room for double or individual use, tax included):

EURO 101 (plus EURO 17 for breakfast per person)

<http://www.viajes-ejecutivo.es/madrid2002/castellano/4estrellas/hotelnhnacional.html>

213 rooms with direct telephone, colour TV, mini-bar, air-conditioning, free toilet kit, hair-dryer, buffet breakfast, and 24 hour room service. Restaurant, bar-cafetería, laundry service and parking.

HOTEL NH ZURBANO ***

ZURBANO, 79

Special Price for the Conference (a double room for double or individual use, tax included):

EURO 91 (plus EURO 14 for breakfast per person)

<http://www.viajes-ejecutivo.es/madrid2002/castellano/3estrellas/hotelnhzurbano.html>

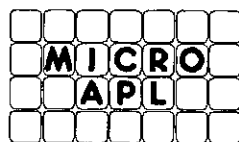
255 rooms with mini-bar, air-conditioning, colour TV, parabolic antenna, direct telephone, and laundry service. Restaurant, bar-cafetería, buffet breakfast and parking.

Additional offers are under consideration.

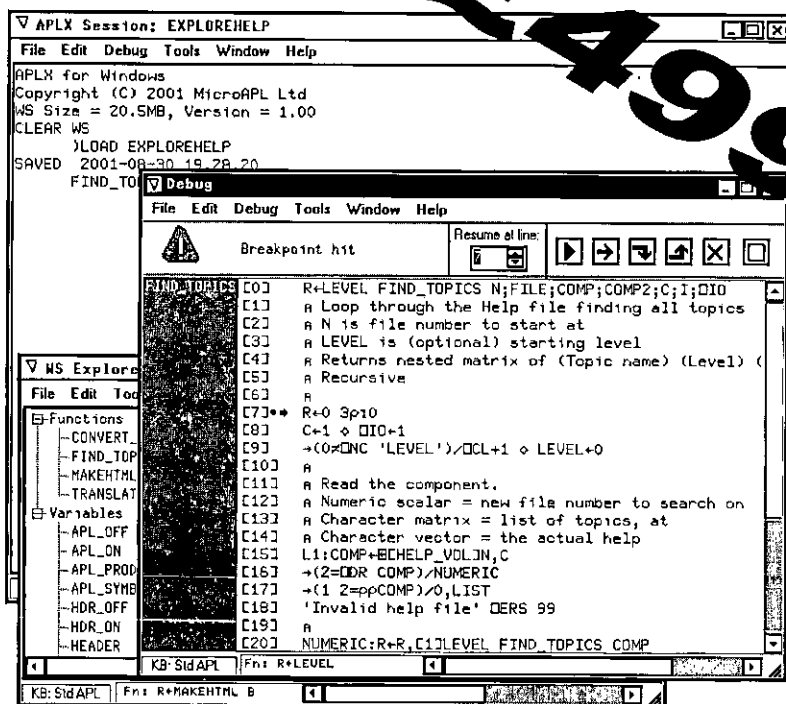
Reservations should be done via our travel agency (to assure the Conference Price).

More information at the Conference web page: <http://www.apl2002.com>

APLX



for Mac OS 9 Windows 95/98/ME
Mac OS X Windows NT/2000/XP



The new, cross-platform, full-featured, user-friendly, GUI-enabled, cost-effective APL.
Check it out at <http://www.microapl.co.uk/apl>

APLX and MicroAPL are trademarks of MicroAPL Ltd. Mac is a trademark of Apple Computer Inc. Windows, Windows NT ME & XP are trademarks of Microsoft Inc. Price shown excludes VAT.

BAA: Sustaining Members' News

MicroAPL Ltd

MicroAPL is currently working on version 1.1 of *APLX*, our cross-platform APL development system for Windows, MacOS, Linux and AIX. This upgrade is scheduled for release in late June. Although we are not yet ready to publish the full list of enhancements (details will be given in Vector 19.1), amongst the new features which will definitely be included are:

- Support for the 'structured control' programming keywords such as *:If*, *:For* and *:While*.
- Support for using the double-quote rather than the traditional single-quote character for delimiting APL character vectors, making it easier to enter strings containing quotes.
- Enhanced support for graphics and images in the *□WI* sub-system.

Version 1.1 will be available as a free upgrade for existing users of *APLX*.

In the meantime, we have placed the two main *APLX* Reference Manuals as PDF documents on our website <http://www.microapl.co.uk/apl>. (These include the *APLX* font as an embedded TrueType font – you need Adobe Acrobat 4.0 or later to view them.)

Causeway Graphical Systems Ltd

Coo, we won an Award for the website! Causeway will be proudly displaying the "Cool Site Award" from <http://dmoz.org> for the next few weeks. I suppose it's because we actually included some interesting and useful content – I even get the occasional fanmail from non-APL readers of the old Vector articles in the /pubs section. I think I will start to develop a lot more SVG content soon (you can try the Climate charts in SVG now) as it looks as if Adobe have succeeded in smuggling out the viewer (Acrobat Reader 5 just installs it for you without the option) to the point where it will soon be a better bet than VML.

The interesting challenge for GraPL (and by implication RainPro) is supporting some of the nice new stuff 'native' in our own viewer, and ideally in PDF and PNG format too. We are unlikely to do animations, but look out for decent gradient fill support and transparency over the summer. If I can keep on top of the buglist, that is – the more we sell the better it gets tested!

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.

Please help us keep it up to date!

All contributions and updates to the Vector Product Guide should be sent to: Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERSystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERSystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERSystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL+Win v3.6 Full version	1895	A 32-bit Windows-hosted Interpreter that runs under Windows 95/98/ME/NT/XP
	Upgrade to v3.6 from v3.x	474	
	to v3.6 from APL+Win v2.x	1034	
	to v3.6 from APL+Plus III v1.x	1249	
	Migration to v3.6 from APL+Plus II or APL+Plus PC	1484	
	APL+Unix		
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.

	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Aitos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL.
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APIs; TIME facility
	APL2 for Windows version 1.0	\$1500	Product No. 5639-d46, part number 4229558.
	APL2 Runtime environment	\$250	Part No. 39L8419 - Packager and runtime CDs. One additional runtime install \$200, 5 additional \$900, 10 \$1,500.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyadic Systems Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only
	Books and accessories		
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	

Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
Oasis	Dyalog APL	poa	Dyadic Systems
	APL*PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and JSoftware products	poa	
	USA		
	Dyadic and JSoftware products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.

APL Software/Services			
	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software, Books; APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick.holt@juno.com.
APL Systems IDC SL	APL+Linkpro 3.0 New System	646	Database Access SQL Client for OBDC
	Upgrade to v3.0		
	from APL+Linkpro-32	280	
	from APL-Link or APL+Linkpro	474	
	GraphX (Includes ChartX)	599	
	GraphX Lite (No distribution)	340	
	SPREAD (An APL grid control)	86	
	QPLOT (x,y plotting in APL)	47	
	QWIN	215	(Runs APL+DOS functions in APL+Win)
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0. At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/98/NT. Future plans include an APL+WIN version and later a LINUX version.

CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled GUI. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. Because of its unique open construction within the language itself, this environment is clearly competitive in a large domain with the APLs from the other vendors, for just a token cost. K.CoSy notepad nature, interactivity, and open K code vocabulary make learning Arthur Whitney's K far less daunting and far more productive than its raw console, or any external scripting method. K.CoSy being young and developing open code, lends itself to offer by subscription. Charter subscriptions are just \$30 (half that for students). (Requires separate download of K from Kx.com.)
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLD1, IC/E and IC/1 users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.
	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark
	<i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>		
JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version; \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.

	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	QWIN	poa	Legacy DOS Windowing support for APL+Win
	ODBC/127	poa	IBM AP127-like ODBC Interface for APL+Win and Dyalog APL/W
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +i1 v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Ajay Askookum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.
Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL+Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1989.
APL Systems IDC SL	Consultancy	poa	Consultancy and maintenance available by retainer or on call.
AUSCAN Software	Consultancy	poa	APL software development, training
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modeling. A great deal can be done in a day.
David Crossley	Consultancy	poa	Experienced in large APL system developments since 1969 for PC or mainframe.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration

Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa, competitive	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C++, VBA, SQL servers.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from: SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APLW. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
George MacLeod	Consultancy	poa	Design and programming of new APL applications. Enhancing and maintaining existing APL applications. Porting existing APL applications from one APL system to another. Supporting users of APL applications. Experienced on both mainframe, UNIX and PC APL interpreters.
Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MasterWork Software	Consultancy	poa	Consulting and J programming for econometrics and statistics in public policy, health and food industries.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.

Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to APL-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Weighahead Systems	Consultancy	poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adlee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
ComLog	Comic-Logger	\$25.95+p&p	APL *PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL	FF350 (private) FF2800 (Company)	
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL 100FIM(private), 30(student), 1000 (Co)	
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join
NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Allanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qily SI-Info	SF60 (SI) + SF20 (SAUG)	
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Stellinger Weg 19, 20255 Hamburg, Germany. Tel: +49 40 40170951 Fax: +49 40 40170952. Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
Adaytum Limited	Douglas Rowley	Castlegate, Tower Hill, BRISTOL BS12 0JA. Tel: 0117 921 5555 Fax: 0117 922 7749. Email:sales@adaytum.co.uk
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL2000 (Europe)	Fred Honea	see APL Systems IDC SL
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461

APL Software Services	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: aplsi@starpower.net
APL Systems IDC SL	Fred Honea	Alfredo Marquerie, 12 - 2 F, 28034 Madrid, Spain. Tel: +34 91 730 7008 (Office) +34 60 680 5949 (Mobile); Fax: +1 775 743 6131. Email: uksales@apl2000.net
Ajay Askoolum	Ajay Askoolum	42 Hanworth Road, Redhill, Surrey RH1 5HT Tel: 01737-771643 Email: ajay@askoolum.freemove.co.uk
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@interlog.com
BACUS	Joseph De Kerf	Roclinberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888 Email: BeautifulSystems@golfs.net
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@tesco.net
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
Ian Clark	Ian Clark	1, Heifer Mill Cottages, Mosterton, Beaminster, Dorset DT8 3HG, England. Tel: +44 (0)7931 370304. Email: earthenpot@aol.com
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652 Email: codework@codework- it.com
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 280-1435 Email: jeff@softnred.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinsoft Oy	Pertti Kalliojärvi	Lönnroinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinsoft@dinsoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, UK. Tel: 01256-811125 Fax: 01256-811130
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Evastic AB	Olle Evero	Bertellusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakawata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakawata@compuserve.com

General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749328 Fax: 01622 749365 E-mail: martin@gssoft.freemove.co.uk
Godin London Incorporated	Gaétan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email:HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.Ltd.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: 100612.1057@compuserve.com
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept REN/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOSTROY	Alexey Miroshnikov	3 S. Tulen Lane, St. Petersburg 191186 Russia. Tel: +7 812 325-9797 Fax: +7 812 311-2184 Email: aim@infostroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
JAD Software	David Crossley	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net
Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buld.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141- 0022. Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KYY00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	Hochsteingasse 13/26, 8010 Graz, Austria. Tel: 0043 (0)316 91 42 51 Mobile: 0043 (0)699 1 91 42 51 2. Email: joachim.hoffmann@gmx.at
JSoftware Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@jsoftware.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56, 00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.htv.fi
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.ntl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: alastair.kinloch@btinternet.com
George MacLeod	George MacLeod	37 Newhouse Rd, Bovingdon, Herts, HP3 0EJ, UK. Tel: 01442-831385 Fax: 01442-831388 Email: george.macleod@ntlworld.com
MasterWork Software Ltd	Fraser Jackson	PO Box 58-036, Tawa, Wellington, New Zealand. Tel: +64 (4) 232-4440 Fax: +64 (4) 232-4452 Email: fraser.jackson@xtra.co.nz
Mercia Software Ltd.	Gareth Brentnall	Mercia House, Ashted Lock, Aston Science Park, Birmingham, B7 4AZ, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at danb@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: Ellis@mrtrfm.demon.co.uk
NY Sig	David Siegel	PO Box 2697, New York, NY 10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: rijkse@oasis.nl
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com

Optima Systems Ltd	Paul Grosvenor	1st Floor, 10 Raleigh Court, Priestley Way, Crawley, West Sussex, RH10 2PD, UK. Tel: 01293 562700 Fax: 01293 562699 Email: mailbox@optima-systems.co.uk
Qualedi Inc.	Nicole Schless Georges Brigham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 876-9083. Email: sales@qualedi.com; info@qualedi.com
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 313, Newtown, CT 06470, USA. Tel: +1 (203) 270-9729 Email: rendata@aplbooks.cnchost.com or orders@aplbooks.cnchost.com
Resources & Results	Frank Rhodes	2438 W. Northgate Dr., Irving, TX 75062, USA. Tel: +1 972 523 5117 Email: frank@decision-support.net
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	16 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Sollton Associates	Benoit Paquin	Sollton Denmark, Havsgårdsvej 4, 2900 Hellerup, Denmark Email: sales@sollton.com
SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5061, Oboinsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulthielm	Novator Consulting Group AB, Svårdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CServer: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
Welghaead Systems	Philip Bulmer	Camberley House, 1 Portesbury Road, Camberley, GU15 3RB, UK. Tel: +44 1276 20789 Email: sales@welghaead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP AND WORLD WIDE WEB SITES

IBM APL2 (FTP)	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive (FTP)	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII (FTP)	archive.uwaterloo.ca/languages/apl/workspaces/aplascii
ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
Adatym Limited	www.adatym.com/

AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/apljourn.htm
APL Systems IDC SL	www.apl2000.net
AUSCAN	www.interlog.com/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyadic Systems Ltd	www.dyadic.com/
DynArray	www.dynarray.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Houben (IQL)	www.apl.olap.club.tip.nl
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/ad/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.naska.co.jp/JAPLA/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	http://mackaykinloch.ltd.uk/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualedi.com
Renaissance Data	www.aplbooks.cnchost.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Snake Island Research Inc.	www.snakeisland.com
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

ZARK Newsletter Extracts

introduced by Jon Sandles

Utility Corner: Generalised Searching

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In last issue's LIMBERING UP column, we asked you to design and write utility functions *LEIOTA* and *LTiota* to emulate $(\leq \iota)$ and $(< \iota)$, in which ι is a hypothetical operator that returns an array of indices into the vector left argument where each value in the right argument first relates true according to the relational function operand provided. Whew! That's a mouthful. Here's an example:

```
      38 42 75 84 (=ι) 42 15 38
2 5 1
```

The number 42 is found in the 2nd index of the left argument; 15 is not found, so 5 is returned (one beyond the last index in the left argument); and 38 is found in the 1st position. This is the behaviour of the normal dyadic ι function. Let's try different relational functions:

```
      10 20 35 50 (≤ι) 12 57 35
2 5 3
      10 20 35 50 (<ι) 12 57 35
2 5 4
```

In the first of these two expressions, the first time 35 is less-than-or-equal any value of the left argument is in the 3rd position. In the second, the first time 35 is less-than any value of the left argument is in the 4th position.

Before we discuss the *LEIOTA* and *LTiota* functions that implement these hypothetical primitives, let's get practical. What good are they? If available, how can you use them?

The $(\leq \iota)$ and $(< \iota)$ functions have the useful property of mapping values into ranges of values. For example, suppose you want to classify all employees by age range, where you've defined five ranges:

```

1:      ages≤10
2:    10<ages≤20
3:    20<ages≤35
4:    35<ages≤50
5:    50<age

```

Given a vector of ages, say 12 57 35, you'd like to convert them to the corresponding range indices 1 to 5, in this case 2 3 5. This is exactly what ($\leq$) does. After using ($\leq$) awhile, you'd come to view its left argument as a vector of inclusive range upper limits. Any number in the right argument that is not in any of the defined ranges, i.e., that is greater than the largest upper limit, is assigned one greater than the last index of the left argument.

By a similar line of reasoning, you'd come to view the left argument of ($<$) as a vector of *noninclusive* range upper limits. The only difference between ($\leq$) and ($<$) is whether the limits are inclusive.

Now suppose you define these four age ranges:

```

1:      if 35<age≤50
2:      if 20<age≤35
3:      if 10<age≤20
4:      age≤10

```

You might be tempted to use 50 35 20 10 as the left argument:

```

      50 35 20 10 (<) 12 57 35
1 5 1

```

Unfortunately, the result is worthless. The meaningfulness of the results of ($\leq$) and ($<$) requires that the values of the left argument be in ascending order.

Nevertheless, the shortcomings of these hypothetical primitive functions should not deter us. Since we're the programmers, we can make our utility functions do whatever we want. Let's call the utility functions *UIOTAI* (upper limit inclusive) and *UIOTAN* (noninclusive) instead of *LEIOTA* and *LTiota*.

Here's what we want:

```

      50 35 20 10 UIOTAI 12 57 35
3 5 2
      50 35 20 10 UIOTAN 12 57 35
3 5 1

```

And, while we're at it, let's define two other functions that take lower limits (inclusive and noninclusive):

```

      10 20 35 50 LIOTAI 12 57 35
1 4 3
      10 20 35 50 LIOTAN 12 57 35
1 4 2

```

All four functions (*UIOTAI*, *UIOTAN*, *LIOTAI*, *LIOTAN*) require a vector left argument (as does dyadic *ι*). The values in the left argument needn't be sorted in any special order. They also needn't be distinct, but, as with dyadic *ι*, they usually will be. Finally, the right argument does not need to be a vector. If it's a matrix, the result will be a matrix too.

Now, to the solutions we received.

Gary Logan pointed out that IBM's APL2 Language Reference Manual defines dyadic *iota* with the following identity:

$$L \iota R \quad \leftrightarrow \quad \square IO++/\vee\backslash \sim R \circ . = L$$

If *L* and *R* are simple (non-nested) arrays, *=* can be replaced by *≡*. Taking it a step further, *≡* can be replaced by *≤* or by *<* to produce the behaviour of our hypothetical (*≤ι*) and (*<ι*) functions. In fact, in APL2 implementations that permit user-defined operators, the operator *IOTA* can be defined like so,

```

      □v r' IOTA'

      v Z+L(F IOTA)R
[1]   Z+□IO++/∧\~R∘.F L
      v

```

permitting this syntax:

```

      10 20 35 50 ≤IOTA 12 57 35
2 5 3
      10 20 35 50 <IOTA 12 57 35
2 5 4

```

This solution is direct, elegant, and general. Unfortunately, not all implementations of APL permit user-defined operators. Even if they did, some efficiency can be gained by knowing in advance the function to be used as the operand. For example, instead of using *~R∘.≤L*, you can use *R∘.>L*:

```

      v Z+L UIOTAI1 R
[1]   Z+□IO++/∧\R∘.>L
      v

```

Furthermore, as Henri Brudzewsky noted, the function *∧* (leading) is redundant if the values of the left argument are in ascending order, leading to:

```

      v Z+L UIOTAI2 R
[1]   Z+[]IO++/R°. > L
      v

```

Unfortunately, neither of these solutions gives the result you want when the values of the left argument are in descending order.

Also, they rely on outer product, which can be a resource hog. Since all possible combinations of the values in the left and right arguments are compared, two large arguments can cause the algorithm to bog down, and possibly generate a *WS FULL* message. For example, if the left argument has 100 values (ranges), and the right argument has 10,000 numbers, one million comparisons will be made, and a 1000 by 100 Boolean matrix will be constructed and manipulated.

To avoid the outer product dilemma, Eric Baelen offered a solution that uses grade-up ($\uparrow$) to perform the comparisons. More accurately, his solution uses ranking ($\uparrow\uparrow$). Remember, ranking tells you the relative ranking of each value in its argument relative to the other values. The ranking 1 is assigned to the smallest number, 2 to the next smallest, and so on. For example:

```

      ↑↑27 15 5 36 22 15 33
5 2 1 7 4 3 6

```

Here's Eric's solution:

```

      v Z+L UIOTAI3 R
[1]   Z+[]IO+((pR)+↑↑R,L)-↑↑R
      v

```

The insight that leads to this solution comes from the observation that respective rankings of the values in a vector increase as more values are catenated to the end of the vector. Specifically, they each increase by the number of values catenated that are less than that value. By catenating the limits (L) to the end of the values (R), and comparing the resulting rankings to the rankings without the limits catenated, you can quickly determine how many limits are less than each value. The result follows directly.

In general, the grade functions are surprisingly efficient for large arguments – more efficient than outer product. Unfortunately, this solution performs four grades. If the left and right arguments have 100 and 10,000 elements respectively, two grades of length 10,100 and two grades of length 1000 are performed. That's a lot of grading.

Fortunately, the ranking process can be made somewhat more efficient by taking advantage of the fact that the leftmost $\uparrow$ in the expression $\uparrow\uparrow v$ is operating on a

permutation vector (a vector of some arrangement of the elements $1:pV$). Indexed assignment can be used in place of the second grade-up:

$$R \leftarrow \Delta \Delta V$$

is the same as

$$\begin{aligned} G &\leftarrow \Delta V \\ R &\leftarrow (\rho G) \rho 0 \\ R[G] &\leftarrow 1 \rho G \end{aligned}$$

This leads to the next solution:

```

V Z←L UIOTAI4 R;G
[1] Z←G+ΔR,L
[2] Z[G]←1ρG
[3] R←G+ΔR
[4] R[G]←1ρG
[5] Z←□IO+((ρR)↑Z)-R
V

```

Using Δ twice is faster than using it four times, but once would be better yet. Paul Zocco submitted a solution that does just that. We've left the comments in his solution for obvious reasons:

```

V R←UPPER UIOTAI5 VALS;G;I;M
[1] a Returns the index of the value of
[2] a the vector UPPER that is the
[3] a nearest upper bound (inclusive)
[4] a for each element of the vector
[5] a VALS. Values in UPPER need not be
[6] a ascending or distinct. □IO+ρUPPER
[7] a is returned for elements in VALS
[8] a above the largest value in UPPER
[9] a
[10] a Combine and sort up to use 1st val
[11] a in UPPER if vals not distinct:
[12] R←G+ΔVALS,UPPER
[13] a Flag elts from UPPER in grade vec:
[14] I←(G≥□IO+ρVALS)/1ρG
[15] a Inds into UPPER, in sorted order,
[16] a using "one greater" for values
[17] a above largest in UPPER:
[18] M←(G[I]-ρVALS),□IO+ρUPPER
[19] a Replicate inds for values in
[20] a corresp. ranges:
[21] R[G]←((I,□IO+ρG)-□IO,I)/M
[22] R←(-ρUPPER)↑R a Discard UPPER inds
V

```

Paul's function permits the left argument vector of limits to contain duplicates and to be in any order. This is an advantage over the other functions listed.

However, the logic assumes the right argument is a vector. To make the function more general, the right argument can be ravelled at the beginning, and the result can be reshaped at the end to the argument's original shape.

Let's look at the efficiency of these functions. We timed each of them under APL*PLUS II/386 using a left argument of 5 elements and a right argument of 10,000 elements, and again with a left argument of 100 elements. Both arguments were all integers. The values were uniformly distributed among the ranges defined.

Here are the results; all times were scaled to that of the fastest algorithm:

Function	Algorithm	Processing Time	
		5×10000	100×10000
UIOTAI1	+/\R◊.>L	2.64	35.44
UIOTAI2	+/R◊.>L	1.73	22.29
UIOTAI3	⋈ twice	3.13	2.93
UIOTAI4	⋈ twice	1.74	1.63
UIOTAI5	⋈ once	1	1

As you can see, the outer product solutions go to sleep when both arguments get large.

Since *UIOTAI5* is the fastest function and produces the desired behaviour for a left argument whose values contain duplicates or are not in strictly sorted order, we took the liberty to write the *UIOTAN*, *LIOTAI*, and *LIOTAN* functions in the same style:

```

▽ R←UPPER UIOTAN VALS;G;I;M
[1] R←G+▽VALS,UPPER
[2] I←(G≥□IO+ρVALS)/ιρG
[3] M←(□IO+ρUPPER),G[I]-ρVALS
[4] R[G]←((I,□IO+ρG)-□IO,I)/M
[5] R←(-ρUPPER)⋈R
▽

```

```

▽ R←LOWER LIOTAI VALS;G;I;M
[1] R←G+▽VALS,LOWER
[2] I←(G≥□IO+ρVALS)/ιρG
[3] M←(G[I]-ρVALS),□IO+ρLOWER
[4] R[G]←((I,□IO+ρG)-□IO,I)/M
[5] R←(-ρLOWER)⋈R
▽

```

```

      V R←LOWER LIOTAN VALS;G;I;M
[1]    R←G+4VALS,LOWER
[2]    I←(G≥IO+ρVALS)/ρG
[3]    M←(IO+ρLOWER),G[I]-ρVALS
[4]    R[G]←((I,IO+ρG)-IO,I)/M
[5]    R←(-ρLOWER)+R
      V

```

The comments are basically the same.

Ray Cannon has submitted a new solution for *LIOTTA* and *LEIOTTA* using dynamic functions:

```

LEIOTTA←{>1++/α{α<ω}}(cω)}
LIOTTA←{>1++/α{α≤ω}}(cω)}

```

and an operator solution which can take < or ≤ as the function argument:

```

IOTA←{>1++/α αα{α α ω}}(cω)}

```

Limbering Up: Generalised Searching

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

Suppose you have a "database" of employee information. For each of the 2000 employees, you have four items of information: age, state of residence, type of employee, and a bonus amount received last year.

Here are a few records:

<u>AGE</u>	<u>STATE</u>	<u>TYPE</u>	<u>BONUS</u>
26	FL	E	200
39	MI	S	550
31	NY	O	720
52	NY	S	375

The information is stored in your APL workspace in four simple variables: *AGE*, *STATE*, *TYPE*, and *BONUS*. *AGE* and *BONUS* are 2000-element numeric vectors;

STATE is a 2000-row, 2-column character matrix; and *TYPE* is a 2000-element character vector.

Your job is to summarise this mass of information by grouping the employees by age, state, and type. Complete the following function:

```

      ▽ R+CROSSTAB
[1]  a Summarises employee information
[2]  a stored in workspace globals AGE,
[3]  a STATE, TYPE, and BONUS. AGE and
[4]  a BONUS are numeric vectors, STATE
[5]  a is a two-column character matrix,
[6]  a and TYPE is a character vector.
[7]  a Their elements/rows are in 1-to1
[8]  a correspondence. R is a four-
[9]  a dimensional array whose shape is
[10] a (5 10 3 2):
[11] a
[12] a R[I;;;] The Ith age range:
[13] a     I=1;   0≤AGE≤29
[14] a     I=2;  30≤AGE≤39
[15] a     I=3;  40≤AGE≤49
[16] a     I=4;  50≤AGE≤64
[17] a     I=5;  65≤AGE
[18] a R[J;;;] The Jth state. The glo-
[19] a     bal ALLSTATES is a 10-row,
[20] a     2-column character matrix of
[21] a     the codes for all ten states
[22] a     in which the employees reside.
[23] a R[:,K;] The type 'EOS'[K]
[24] a R[:,;1] The employee count.
[25] a R[:,;2] The total bonus amount.

```

Here's how this function can be used:

```
R+CROSSTAB  a Summarise the data
```

The number of employees age 0 to 29, living in state *ALLSTATES*[7;], with type *S*:

```

R[1;7;3;1]
276

```

Their total bonus amount:

```

R[1;7;3;2]
120474

```

Their average bonus amount:

```

      ÷/R[1;7;3;2 1]
436.5

```

The number of employees by type:

```

      +/[1]+/[2]R[;;;1]
802 460 738

```

The total bonus paid to ages 40 to 49:

```

      +/[1]+/[2]R[3;;2]
325120

```

The idea behind a function like *CROSSTAB* is to process the database once, summarising its information into a single multi-dimensional array. All subsequent analysis can then be easily and efficiently performed on the array, rather than reprocessing the thousands (or millions) or records.

Try to make the function as efficient as possible. Hint: the functions presented in the Utility Corner of this newsletter may be helpful

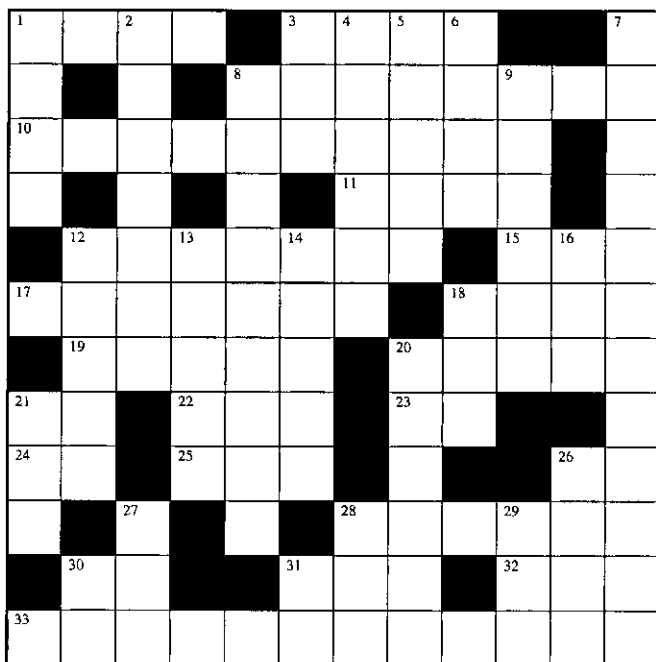
Send your solutions to:

Vector Production
 Brook House
 Gilling East
 York YO62 4JJ
 UK email apl385@compuserve.com

The notable functions and their authors' names will be printed in the next issue of *Vector*. Good luck and happy limbering.

Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA.

New Crossword and Solution for 18.3



Across

1. .01, as a reciprocal
3. $(\sim V \in BT) / V$
8. The index within numeric vector V of the nearest value $\geq$ scalar S
10. Set M to 1 if any of the vector VE is true, or if any of vector C is untrue. Else set it to 0
11. The larger of G or JL
12. $(R < 10 + VL), (R < 10 + VL)$
15. $(\rho M) \rho (E\phi M[1;]), (E\phi M[2;]), (E\phi M[3;]), \dots$
17. An N by N matrix of 0s
18. The location in vector VA where A first occurs
19. $K = (\times / \rho AT) \rho AT$ for scalar K
20. The smallest number in each row of matrix RNT
21. $\circ 1$
22. Bit vector to index vector: BV___BV

23. $\div 10$
24. $(B * 2) * .5$
25. $((- / 5 \ 4 * 2) * .5) \times (1 N) = 1 N$
26. Rank (of an array)
28. $(+ / LM[1;] * A[1], (+ / LM[2;] * A[2]), \dots$
30. Cumulative product of
31. Flag the leading 1s in Boolean V
32. The number of elements in matrix A
33. The number of rows of MAT with more than one occurrence of substring SS

Down

1. $MT[;1] \div MT[;2]$ for two-column MT
2. Is A numeric?
3. $\Delta V 1 E$ for character vector V
4. $S < G \leq 0$ for Boolean S
5. The values of R, with negatives replaced by 0s, and with B on the front
6. $v / J = TV$ for vector J and scalar TV
7. $MAT[\square IO;]$ without using indexing
8. Bits that flag each first distinct value in vector V
9. Let's start from scratch
12. The matrix TB with a column of 12s stuck on its left side
13. $(\phi NT) / \phi N$
14. $(K \rho 3), (L - K) \rho 0$
16. $N, (N - 1), (N - 2), (N - 3), \dots, 1$ (origin 1)
18. $(+ / V) \rho 1$ for positive integer vector V
20. Round the elements of array VS
21. $P \times (9 \div P) - \lfloor 9 \div P$
26. $(\rho L) + (\rho A)$ for vectors L and A
27. $B + B$ for Boolean vector B
28. $R \leftarrow A$ and $R[A / 1 \rho A] \leftarrow S$ or $R \leftarrow ______$
30. Product of
31. All of

Solution to Crossword in 18.3

1	$\perp$	M	A	T		ϕ	V		ρ	∇	N
$\in$		$\vee$	$/$	$\wedge$	$/$	M	$=$	W		$?$	
M	T	N		B			$($	A	$*$	2	$)$
$\in$	A		1	0	0	$-$	V		$/$		S
V	R	$\div$	2		V	$\perp$	Γ	$/$	V		I
		$\backslash$				ρ	0		$+$	$/$	
$($	$\square$	I	0	$+$	ρ	V	$)$	$>$	V	$\perp$	A
ρ	N	$,$	K		ρ		L	N		ρ	M
M	L	$-$	$?$	N	ρ	5	9		Φ		T
$)$		$*$	T	P		5		N	R		$,$
ρ		1	5	$\dagger$	V		$\times$	$/$	R	$,$	N
0	ρ	0		1	∇	M	A	T	R	I	X

J-ottings 32: the Stylish Part of Vector

by Norman Thomson

Browsing through back numbers the other day, it struck me that in spite of being in its eighteenth year, "Vector" has won no awards for outstanding literary merit, has received no Booker prize nominations, and features in no university reading lists for exemplary 20th century English prose. This situation demands remedy, and while the urge burned hot within me, I resolved to take some immediately necessary steps. I had heard recently that if one is allowed to oversee a couple of million monkeys equipped with an equal number of keyboards, it is a near cert that one of the little brutes will eventually outperform Shakespeare. So with my computer thus dedicated to monkey business, here is what emerged.

Start with the distinction between `7?10` and `7(?@#)10` which give 7 random numbers from the set 0,..., 9 with and without repetition respectively. In the former case the left argument must be no greater than the right. The latter phrase uses *copy* (dyadic #) to replicate the 10 seven times prior to *rolling*.

It is easy using indexing and *tally* (monadic #) to extend randomisation to lists such as

```
lets=.(97+i.26){a. NB. lower case alphabetic chars.
lets
abcdefghijklmnopqrstuvwxyz
```

To obtain a single random letter from `lets` say :

```
ran=.{~?@#
ran lets
j
```

Suppose that you want the *number* of random numbers in a selection to be random. Two applications of `?` are needed:

```
10(?~?)7
3 7 6 8 9
```

The hook `10(?~?)7` means `(?7)? 10` that is, the roles of the right and left arguments have been reversed by using the *passive* adverb. Because there can be no repetitions, the right argument should be no greater than the left, since, depending on the luck of the draw you *might* hit

```

7(?~?)10
|domain error: q NB. looks like this ? 10 was >7
| 7 (q~q)10

```

If you want to exclude the possibility of an empty selection use `10(?~q)7`

where

```

q=.:@? NB. random integers from 1 to y.
6 q 7 NB. deal 6 random ints. from 1 to 7
6 2 4 7 5 1

```

A nice little palindromic fork `#(?~?)#` delivers the indices of a random selection of letters :

```

(#{?~?})#)lets
6 17 20 10 15 8 9 22 13 18 19 23 7 24

```

and the addition of *from* makes this into a random selection from the list

```

({~#(?~?})#)lets
xprhwkf

```

In order to obtain the indices of, say, ten random letters *with* repetition say :

```

(10&(?@#)@#)lets
10 15 19 22 8 4 19 18 22 1

```

and to convert these to indices use *from* as before :

```

({~10&(?@#)@#)lets
dpaxjnighg

```

The problem with this as a technique for random words is that words like this are not very beautiful and are almost certain to be unpronounceable. One possible strategy to overcome this might be to use as weights a list of rough relative frequencies of occurrence of the various letters in English :

```

fr=.8 2 3 4 12 2 2 6 8 1 1 4 3 8 8 2 0 6 8 8 3 1 2 1 2 1
#fr
26
+ /fr
106

```

The 16th. letter 'q' has been given the value of 0, on account of its tiresome requirement to be followed by a 'u', a matter which can comfortably be left to a later refinement. A weighted letter list is then

```
wlets=.fr#lets      NB. weighted letters
#wlets              NB. sum of weights
106
  ((~10&(?@#)@#)wlets
iedorthgwh
  ((~10&(?@#)@#)wlets
doakidsoas
```

At least the words look vaguely like words, and some (but only some) are just about pronounceable! Somewhat greater control of the patterns, as opposed to the mere frequencies, of vowels and consonants is desirable, so weight the vowels and the consonants separately:

```
wv=.2 3 2 2 1#'aeiou'
wc=.2 3 4 2 2 6 1 1 4 3 8 2 0 6 8 8 1 2 1 2 1#
      (lets-.'aeiou')
```

Next determine some vowel/consonant patterns, and randomize each component in turn:

```
wpat=.wc;wv;wc;wv;wc
ran&>wpat
terel
wpat1=.wv;wc;wc;wv;wc
ran&>wpat1
ettum
```

The patterns can be extended using \$ to allow for longer words, and at the same time the occurrence of unpleasing single letter words can be inhibited by using

```
q2=.:>:@>:@?      NB. random integers from 2 to y.+1
```

which is used to provide a random argument for *take*:

```
(q2 12){.ran&>12$wpat      NB. word of at most 13 lets
pedetnebec
```

Everything is now in place to define a verb to produce random words:

```
rw=. (q2@[]){.ran&>@$
12 rw wpat
selimnehekja
12 rw wpat1
apsaku
12 rw wv;wc
eromoloner
```

Prefixes and suffixes help, and prototype lists of each are built into the verbs:

```
pre=. (4#<''),'ex';'un';'in';'pre'
suf=. (3#<''),'est';'ist';'s';'ed';'ism'
```

following which:

```
rword=.dyad define
(>ran pre),(x.rw y.),>ran suf
)

10 rword wpat
johism
10 rword wpat1
apmopuphed
```

What about random sentences? Different strategies are available. One possibility is simply to string along a random number of random words with as left argument the maximum word length before inflections, and the right argument the maximum sentence length:

```
rs=.dyad define
s=. '' [ i=.0 [ lim=.?y.
while.i<lim do.
s=.s,(x. rword wpat),' ' [ i=.i+1
end.
)

6 rs 9
intoto sots labist nime prediwoft extatotced sudirts
7 rs 10
undusatbo presalos
```

and of course the global variable wpat is also available for tweaking. A bit short in any attributable meaning (my spellcheck also sees red!) but the likes of Kenneth Branagh might be able to put it across for a fee!

Another view is that a random sentence is a random drawing from an available list of words in the same way that a random word is a random drawing from an available random list of letters. In the case of a sentence the list is called a vocabulary, which can be divided into separate sub-lists of, for example, nouns, verbs, and adjs. Only the start of the definitions of these are shown, the rest should become apparent as matters proceed, and no doubt will provide invaluable psychoanalytic evidence for future diagnosis of my various personality disorders:

```
nouns=. 'cats'; 'degree'; 'philosophy'; ..
verbs=. 'hopes'; 'keeps'; 'flies'; 'steps'; ..
adjs=. 'great'; 'lean'; 'fat'; 'blue'; ..
```

Define a sentence pattern analogous to the consonant/vowel patterns:

```
spat=( <adjs ), ( <nouns ), ( <verbs ), ( <adjs ), ( <nouns )
```

Parts of speech patterns are analogous to the vowel/consonant pattern for words:

```
ran&>spat
+-----+-----+-----+-----+
|faint|boat|enters|empty|light|
+-----+-----+-----+-----+
```

Add a standard idiom for removing blanks:

```
rdb=#~-.@(' '&E.)
rdb,(>ran&>spat),.' '
empty boat eats old feet
```

and here is the basis for some random sentences:

```
rsent=.rdb@,@:(>@(&&' '@:ran&>))
rsent spat
lean tradition treads creepy happiness
```

As with letter patterns, different parts of speech patterns add variety:

```
spat1=( <adj ), ( <adj ), ( <nouns ), ( <verbs ), ( <nouns )
rsent spat1
sneaky greek catalyst steps faith
rsent ( <adj ), ( <nouns ), ( <verbs ), ( <nouns ), ( <verbs )
nice faith mates hope castigates
```

Chomsky, thou shouldst be living at this hour!

Of course things are still in the early stages but there are enough controls in place to make incremental refinements, and I think I can claim to have given my monkeys a head start. Progress so far is thus promising, and I confidently expect "Vector" to be awarded a Nobel prize for Literature before too long. Perhaps our friends in the Swedish APL Association could show the spirit of brotherliness and pull whatever strings are necessary to accelerate this outcome.

And in case you are concerned that I may nothing left to say in J-ottings 33, don't worry, it is already written, as is J-ottings 34,.. and J-ottings 35, ... and J-ottings 36 all the way up to six million in fact. And what is more they are all completely fresh, not a single hint of repetition anywhere!

RECENT MEETINGS

Forests in Finland VI

Acknowledgements

Once again, thanks to our many Finnish friends for the hospitality, sauna, and lots of interesting APL. The only disappointment was the weather (just above freezing and foggy) which was hardly their fault. Conrad took the photos.

APL Germany in Munich

This meeting was notable for the first announcement of IBM's APL2 for the workstation (version 2) – due to the time difference we heard it about 3 hours before New York did! Thanks to Dieter Lattermann for inviting me to talk at the meeting.

Forests in Finland VI 7th-9th February 2002

reported by Adrian Smith

Welcome at Krapihovi

Once again, Arto Juvonen welcomed an enthusiastic group of APLers (33 of us this time) to a hotel in the Finnish forests. This one was a little less remote, and was nicely placed within easy walking distance of a large nearby lake. We think the *Krapi* part of the hotel name translates as *Crayfish*, which would make sense, given the location.



As always, the accommodation, food and company were excellent. The normal format of a 2-day meeting with one overnight was extended a little, as we started quite early on the first day. However the Finns know the value of sensible breaks, and the schedule was not overcrowded.

The only severe let-down was the weather. The week before the meeting, the temperature had been around -17°C with clear bright air and snow. Just before we arrived, temperatures soared to $+4^{\circ}$ and a cold fog settled

around Helsinki. Even Finns object to being thrown into icy lakes in these conditions, and the rest of us were content to discover just how slippery wet ice can get. Never mind, we will be back next year, hoping for better things.

The Talks

Three of the talks are included in this Vector, so I will make a very brief summary of my notes. Conrad Hoesle-Kienzlen showed a substantial reporting subsystem which he had written to support a client who wanted 'printout' from Dyalog APL via Excel. This was all created in Dyalog 8.2 ("version 9 sits in its package") and

was most interesting for the unexpected limitations of the Excel object model, for example you cannot choose the number of copies to be printed. There are also plenty of Microsoft annoyances, for example you must be careful to set the state of the spreadsheet to 'saved' before you quit, to avoid Excel popping up a helpful "Save changes?" reminder. I remember going some way down the same road back in 1993 using DDE, and I think that all this 'Object model' stuff may actually have made life harder. At least with DDE_Execute you simply ran commands off menus and getting and setting data by range was really easy.

Kimmo Kekäläinen showed a similar application, this time driving MS Word, and again produced this way because the client wanted something similar to the Word "Merge to Document" capability. He did comment that NewLeaf is much more efficient if you just want bulk output, and that for production code it is much better simply to generate an RTF merged document rather than fight the Word COM object.

Arto Juvonen illustrated many old and new utilities from the FinnAPL idiom library (some excellent ones for date handling) and Jukka Vajärvi showed the macro capability for "Saurus" the Dinosoft product. This seems to have a lot in common with the VoxProxy approach in adding scripts to PowerPoint.



Day-2 opened with Veli-Matti Jantunen running a very nice PowerPoint show on Dyalog's dfns to wake us all up, then we had demonstrations of major APL applications from Reima Naumanen and Richard Eller.

Conrad showed how the web-browser technology can be used with APL2 on the mainframe to modernise old AP124 interfaces, and I waved my arms about in an attempt to explain the inner secrets of PDFs.

A good meeting, and well worth the trip. Thanks, as always, to the organisers, hosts and hotel staff. Here are three of the papers for you to enjoy.

Dynamic Functions in Practice

by Veli-Matti Jantunen (seminar given at Forests in Finland VI)

[ed] These notes have been adapted from a PowerPoint original, with the addition of some commentary and consolidation of several of the bullet points.

Last summer at the Yale conference I tried to convince Bill Rutiser that they should implement Dynamic functions in APL+Win, too. Bill asked dryly: "Where do you need them?".

I don't know how convincing I was (under the scorching Yale sun), but I vaguely remember giving estimates that up to 30 percent of all the code I write is dynamic, and my productivity has increased in proportion to that. Besides, dfns have given me back the delight, i.e. the same delight I had when I first met APL aeons ago.

Now I should change that percentage to, say, 45.

A Small APL Teaser

- change the nested vector:

```
,A+'Arto' 'Conrad' 'Kimmo' 'Adrian' 'Jukka'
Arto Conrad Kimmo Adrian Jukka
```

to ...

```
Arto darnoC Kimmo nairdA Jukka
```

- fewer than nine APL characters, please
- you *are* allowed to use dynamic functions ;)

APL Difficulties

i.e. APL design flaws.

- only one function type

We have to use the same function type with small utilities and huge applications

- local variables in the header line

Nowadays this should be vice versa <sigh!>.

- local functions are hard to handle
- inefficient argument handling

You never know when the *WS FULL* appears..

- functions' loading time backlash
- especially with small functions

When the function is opened for use, the code is parsed especially the header line and labels among others (e.g. possible control structures). This can be dramatic with e.g. *DeleteTrailingBlanks* " *HUGE_NESTED_VECTOR*.

Direct Definition

There have been some proposals how to circumvent some of the limitations.

- as seen in the (Sharp) APL literature..

I can still remember reading the first APL Conference Proceedings, and testing those examples with VS APL ... they didn't work.

```
vec:  ,w
sum:  +/w
mvec: (sum vec w)÷pvec w
mean: w : 0÷pw : mvec w
```

- implemented at least in I-APL
- no practical use

Dyalog Peculiarities

Dyalog APL has some interesting extensions to the APL language.

- specific functions ($\vee\Box\#$) and operators ($\circ\sim\&$)
- function assignment

```
sum←+/
rank←p÷p
fn←very_long_function_name
```

This last one is quite handy: if you need, say, a utility function *DoSomethingUseful* from the namespace *#.utils* repeatedly in one function,

you can make a local assignment `dsu+#.utils.DoSomethingUseful`, and use that (function pointer) instead of the original name monster.

- namespaces
- syntax differences (e.g. shy result)
- control structures (`:If`, `:For`, `:Trap`, ..)
- dynamic functions (first seen at Lancaster 1996)

Dynamic Function

- delimited with braces { }

... where the braces contain legal APL expressions!

- doesn't need to return a result, and it can include another D: function
- may call traditional APL functions and operators as well
- uses D: language, which differs slightly from traditional APL
- laconic syntax
- D: behaves like functional languages (e.g. static variable scope)

Arguments

- D: functions are ambivalent
- the right argument symbol is w
- the (possible) left argument symbol is α
- arguments are pointers, thus one cannot modify the arguments

There's one exception to this rule

- it is not necessary to use the arguments

Efficiency

- D: is pretty simple for the interpreter
- delimited function structure

no branches, labels, control structures, headers..

- no extra data copying behind the scene
 - effectively used with operators
- => usually 10-20% time saving

Some Examples

```

    {w*10} 1+2
30
    {1+2} 'hello world'
3
    'vege' {α,w} 'table'
vegetable
    {{w*0.5} +/w*2} 3 4
5
    root+{w*0.5}
    square+{w*2}
    root +/square 5 12
13

```

Basic Idioms

```

{}          eat
{w}         dex
{α}         lev
{w w}       duplicate
{w/1p w}    indices of a binary vector
{w~' '}     blank removal
{α[w]}      controlled indexing

```

This is extremely useful with looping and sparse indexing!

```

{w[Δw]}     upgrade vector
{w[Δ>w]}    upgrade nested vector

```

Did I mention that I'm an APL2 bigot?

```

{'.'WS'Cursor'w} GUI control

```

Structure

D: function may have line separators ♦

- result = the value of the first non-assigned expression
- every assigned name is local

D: functions are edited with the APL editor

- empty lines and comments are OK
- one-liners can be edited in the session, too

Pretty Obvious

- global variables are editable

.. but only if you create them beforehand

```
a←b←13 ⋄ {}{(a←w)(b[ ]←w)} 7 6 5
a b
1 2 3 7 6 5
```

- D: function can be assigned to a name

n.b. syntax and name class differences!

```
mean←{(+/w)÷ρw} 1 2 3
mean←{(+/w)÷ρw}
SYNTAX ERROR
mean←{(+/w)÷ρw}
^
```

Variable Scope

- D: functions have a static scope (traditional APL has a dynamic scope)

```
type←{way←'static'
      FN1←{way←'dynamic' ⋄ FN2 w}
      FN2←{way w}
      FN1 w}

type 'scope'
static scope
```

Guard

- the only control structure of D; symbol :
- format: *condition:expression*
- if *condition* = 1, the *expression* will be executed and the D: function is closed, otherwise the next line will be executed

```
{2|w:'odd' ⋄ 'even' ⋄ 'never'} 313
odd
```

- even the guard may contain a D: function

```
{ ..multiple lines producing
  a boolean result.. } w: { .. }
```

Default Argument & Shy Result

- the default value of the left argument can be assigned: $\alpha \leftarrow \text{default}$
- the assignment takes place only if the left argument doesn't exist

```
root←{α+2 ∘ w÷α}
3 root 64
  4
root 64
  8
```

- shy result can be attained with guard

```
1:dummy←result
```

Recursion

Recursion and operators are much more used with dynamic functions than with traditional APL.

- D: function may call itself
- the function symbol is ∇

```
fact←{w=1:1 ∘ w×∇ w-1}
```

- pretty efficient with tail call

```
fact←{1{w=0:α ∘ (α×w)∇ w-1}w}
fact←{α+1 ∘ w=0:α ∘ (α×w)∇ w-1}
```

- going through a nested structure

```
{1<=w:∇''w ∘ fn w}
```

Dynamic Operators

- "functions with four arguments"
- operands $\alpha\alpha$ and ww , recursion uses $\nabla\nabla$
- the mere existence of $\alpha\alpha$ makes D: an operator

```

    else+{α:α w ◊ ww w}
    1 + else - 22
22
0 + else - 33
-33

```

Multiline Idioms

```

{1<=w:,w ◊ ,c,w}    ensure vector of vecs
{2=NC w:±w ◊ def}    default value
⊃{rowfn w}""mat      process matrix lines
{IO+0 ◊ α+1+w-α}     integers α..w

```

D:-functions can be used as capsules, thus avoiding the mess with wrong system variables etc.

```

{ML+3 ◊ ew}          enlist
{ML+0 ◊ ew}          type
{ML+3 ◊ (w≠α)cw}     APL2 partition
{ML+2 ◊ (1,1+α)cw}   binary partition
{RL+/-3+TS ◊ w[(pw)?pw]} randomise vector

```

Conditional Structures

you can make your own controls

```

:If
    {if_cond:expr_1 ◊ expr_0}

:Select
    {case1_cond:expr_1
     case2_cond:expr_2
     ..
     expr_0}

```

Looping Structures

```

:For (no connection between loops)
    {expression}""in

:For (variables updated between loops)
    †{expression}/npx

condition{(b z)+w ◊ α:z

    ..algorithm..

a ∇ b z} var init

```

```

:While and :Until
α storage, w counter
gcd+{w=0:α ◊ w ∇ w|α}

```

Error Trapping

- errors can be trapped with error codes (0=all errors), symbol :

```
errcode(s)::err_expr
```

- when an error occurs, the `err_expr` will be executed (unwind)
- nested traps are possible

```

fopen+{0::0
      22::w □FCREATE 0
      24 25::w □FSTIE 0
      w □FTIE 0
}

```

Possible Uses

- file handling

```

open+{0::0 ◊ 22::w □NCREATE 0 ◊ w □NTIE 0}
isfil+{0::0 ◊ {1}□NUNTIE w □NTIE 0 0}
isdir+{0::0 ◊ {1}□NUNTIE(w, '\nul')□NTIE 0}

```

These aren't by far the most useful routines for file handling purposes, but serve as good examples how you can use ad hoc error trapping.

- non-resulting expression giving a result ;)

```

'fm' □WC 'Form'
'fm' {0::w ◊ {w} α □WS 'BCol' w} 255 0 0
255 0 0

```

Unwinding

- error trapping brings us back to the trapping line, and that environment comes back, too

```

beartrap+{0::'erroneous divider:' q
          q+w
          α÷q}

```

```
1 beartrap 2
```

```
0.5
```

```

1 beartrap 0
VALUE ERROR
beartrap[1] 0::'erroneous divider:' q
      ^

```

Creating Globals

- it is possible to make globals in D:
- according to the guide:

```
{_ 'GLOB+w' } 1 2
```

- or with the compose operator:

```
{GLOB◦+w} 3 4
```

- or with the full path name:

```

{# .GLOB+w} 5 6
{(_ ' ' _NS ' ').GLOB+w} 7 8

```

Tricks

Most of these are copied from dfns@dyadic.com mail group.

- does α exist?

```

 $\alpha+1$  1 1p AV ◊  $\alpha=1$  1 1p AV:.. so-so algorithm
0=NC 'α':.. version 9
 $\alpha+\{w\}$  ◊ 1=α 1:.. works everywhere

```

- an array of constants, the structure of which is copied from another array

```
{const}"array
```

- "modify" the argument

```
{q+,w ◊ ((q='^-')/q)+'- ' ◊ (pw)pw}
```

- initialise the left argument

```
1◊{w=0:α ◊ (α×w)∇ w-1}
```

- a quick alternative to *sink+*

```
{0)expr:                    a sink + expr
```

- chaining (to avoid intermediate variables)

```

{ expr
}{expr
}{expr
}data

```

- passing D: function to its sub-D: function

```

fun+({fun w}w)  a hardcoded name
fun+({v{αα w}w}  a via operand!

```

Where Is D: Appropriate?

- idioms!

There's no need any more to have unnecessary intermediate variables

- ad hoc algorithm testing within session

This is one of the real strengths: you can test and retest your code without messing the workspace with temporary variables and functions.

- coding of clean algorithms

logic "shines through"

- local functions

on the fly.

Let's assume that you have a traditional *DeleteTrailingBlanks* function (one-liner, perhaps). You can change it pretty easily to a dynamic function with the same name. But you can as well (if you need it only in one processing part) make it as a local function within that routine. Or, as I have done it with one application, make a *DynInit* function, which creates a toolbox of (semiglobal) D: functions, which only exist during the run time. I have found this approach quite useful for my needs.

- speeding up your ol' toolbox

The most part of the old workhorses can be very quickly changed to D: functions.

- APL communication

Because D: code is straightforward, you can show APL code easier than before (e.g. working APL code can be fully shown in PowerPoint presentations).

- having fun

This is the part I like most ;)

Where Is D: Inappropriate?

- GUI callbacks :(

This may change in the future!

- BIG applications

(strongly a personal opinion)

Maybe this is just an old-dog-new-tricks syndrome, but I think that the old function syntax with control structures et al. shines here.

- BTW: `⌈CS` is not supported

Sources

- Dyalog APL Manual
- www.pyr.fi/apl/texts/fapliddf.zip
- dfns@dyadic.com ["subscribe"]
- www.dyadic.com [product]
- www.dyadic.com/download/dfns.zip (dfns.rtf)
- www.dyadic.com/download/dsamples.zip (dfns.dws, min.dws, max.dws, tube.dws)
- Vector 17.4: Scholes: "D: a Functional Subset of Dyalog APL"

Packing: General

This part of my presentation was meant to be the "practical" part just as to fulfil the promises of the original header line. The idea came last autumn when there was a short but lively discussion about packing in the dfns mailgroup.

Huffman coding

- the most common item is replaced with the shortest binary index
- decoding is very s l o w in APL

Lempel-Ziv-Welch packing

- a "running" algorithm: if the next part to be handled is already found in the dictionary, it can be replaced with the pointer only -> efficient
- many variations have patents pending
- inefficient to code in APL

I know that someone has tried this ;)

APL Packing

Although these methods are not usually comparable with the professional ZIP/ARJ/RAR packers, they have their own uses within the workspaces.

- traditional simple zero-packing

```
zapN←{(ρw){,w≠0}{(,w≠0)/,w}}
unzN←{⊖IO+1 ∘ (1≡w)ρ(2≡w)\3≡w}
```

This is pretty quick, and especially useful when handling sparse multi-dimensional APL arrays.

- running length packing (RLE) is slightly more complicated

```
zapR←{b←{0∈ρw:ω
      {-2-/ω/1ρw}{1,2≠/ω},1
      },ω
      ⊖IO+1 ∘ (ρw)b((,ω)[+b])}
unzR←{⊖IO+1 ∘ (1≡w)ρ(2≡w)/3≡w}
```

Quite useful with bitmaps.

- unique packing is quite handy

```
zapU←{⊖IO+0 ∘ u+ϕ,ω ∘ (ρw)u(u1,ω)}
unzU←{⊖IO+0 ∘ {0≡w}ρ(1≡w)[2≡w]}
```

This works nicely where there are only a small set of unique items in a big variable.

- outer resource: free zlib.dll
- Vector 18.1: Lanzavecchia/Smith

Pros: universal, free, efficient. Cons: you cannot bundle them within the workspace, you need to have extra code to ensure that those resources exist.

We need a couple of D: functions (because .dll handles only 8-bit integer vectors).

```
zapZ+((p w)(32768 ZLIB.DeflateIV 83 □DR,w))
unzZ+((1>w)p 323 □DR 32768 ZLIB.InflateIV 2>w)
```

Problem: Bitmaps

Nowadays GUI apps contain lots of bitmaps: company logos, backgrounds, icons, pictures. Toolbars contain toolbuttons with lots of icons, both standard (which are easily copied from system resources) and hand-crafted (which are buried within the workspace). If you put these parts outside the application, there's the controlling problem, if you enclose them inside your ws, it will grow exponentially, if you don't squeeze them tightly.

Let's Test ...

This is my first test I made after writing a couple of pack-unpack routines.

```
'bmp' □WC 'Bitmap' 'D:\bmp\lion'
cb+ 'bmp' □WG 'CBits'
(pcb)(□SIZE 'cb')
480 568 1090580
```

First we read a bitmap (in this case we could have read the Bits and CMap properties to save space, but that would have spoiled the results ;)). We need a tool which tells us the time used with packing/unpacking, the size of the packed data and the percentage saved. Besides, it should test if the data is unpacked unchanged. A dynamic operator is quite suitable for this purpose:

```
Tst+{
  ai+□AI ◊ q+ww w
  w#αα q:'wrong result'
  sz+ε{s+w ◊ □SIZE's'}"q w
  (2>□AI-ai)(1>sz)('%', "0*100×÷/sz)
}
```

Results

```
unzN Tst zapN cb
90 1124720 103%
```

Quick! But the compressing ratio is not nice...

```
unzR Tst zapR cb
240 822148 75%
```

Chops one quarter off the size, not bad!

```
unzU Tst zapU cb
180 272784 25%
```

... but this seems to be an approach worth using!

```
ZLIB.Init
unzZ Tst zapZ cb
1132 136692 13%
```

This works as expected: best compression this far.

```
unzB Tst zapB cb
731 116440 11%
```

... but this method seems to outperform (in this case) even the ZLIB!!

Let's have a closer look at the last function pair. Actually these are nothing but enhanced unique packing, which works surprisingly well in this case. I have tested with many bitmaps, and usually ZLIB seems to be the winner, but for my purposes these two functions have given a sufficient compression. And hey, you can easily write this algorithm with traditional APL, too!

zapB←{a BMP packer, opened with unzB

```
⊂IO←0 ∘ u←ϕ,w      a unique items
i←u1,w             a indices
w←⌈2⊙pu            a max. width
b←1,2÷/i           a repeats
j←0v,(wρ2)⌈b/i     a binarise
((ρpw),(pw),u)b j  a package

}
```

unzB←{a Open a zapB'd package

```
⊂IO←0 ∘ q←0>w ∘ s←0>q a get vars
r←sp1+q ∘ u←(s+1)+q   a p, unique items
w←⌈2⊙pu               a index width
e←{~2-/ (w,1)/1+pw}1>w a expander
a create indices from bit vector
i←21w{(α,⌈(pw)+α)pw}2>w
rpu[e/i]              a open

}
```

These functions, among others (e.g. Shanno-Fano packing), are included in the newest dfns.dws workspace.

Using OLE with Dyalog APL and Excel

by Conrad Hoesle-Kienzlen (chk@hoesle-kienzlen.de)

Introduction

One of the specifications of a rather large APL project was to print complicated forms via Excel. That was the reason for me to investigate in depth the possibilities of the OLE interface built into Dyalog APL. Of the three major APL dialects i.e. APL2, APL 2000 and Dyalog APL, the latter offers the best support, which makes it comparatively easy to get this difficult interface to do what one wants.

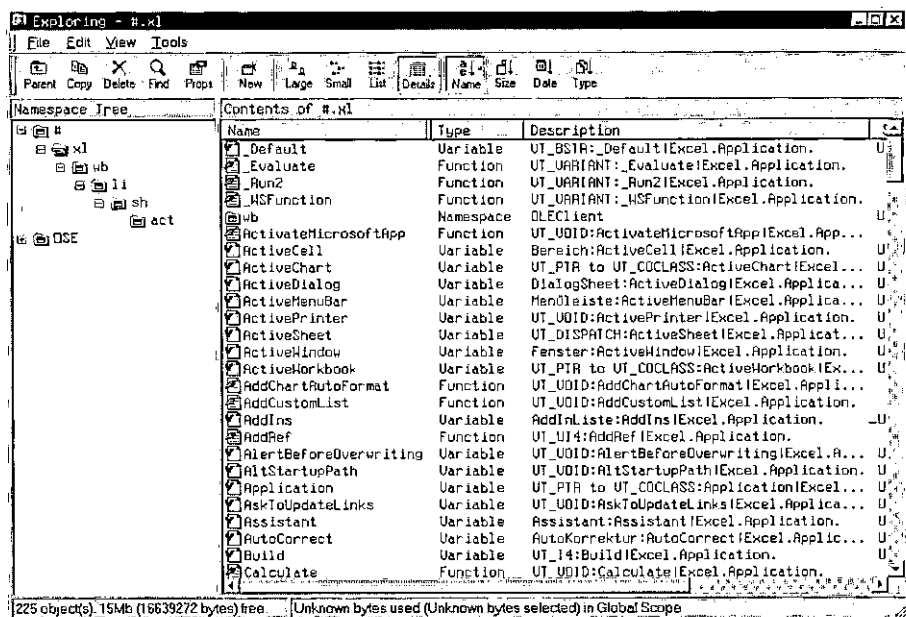
This paper describes how to use Excel as an OLE client with APL as the server. The paper was written at a time when Dyalog APL Version 8.3 was on the market, later versions may differ from the screenshots included here, I have no doubt that the current Version 10 is again improved and even easier to use.

OLE-Support by Dyalog APL

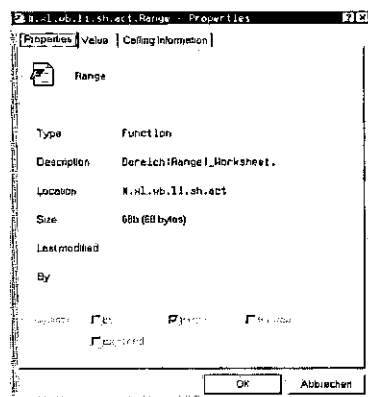
How does the OLE support of Dyalog APL look and feel? First of all, it is very easy to activate Excel (or other OLE applications) from within APL as a client. The code line:

```
'xl' ⌊WC 'OLEClient' 'Excel.Application'
```

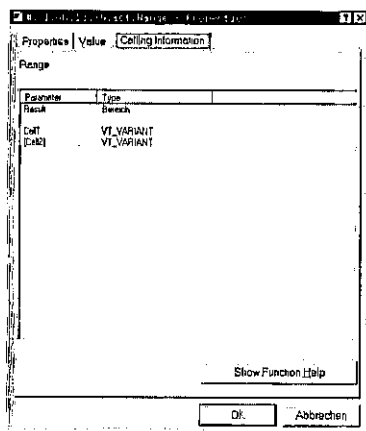
is all that is needed. It creates a new namespace with the name "xl" (the name can be chosen at will). As with all namespaces, one can browse the content with the explorer. The following screenshot shows such a namespace. It contains other namespaces which will be explained later on.



As one can see, one gets practically the same information about this "strange" namespace, as one gets about an APL namespace. This alone would be quite helpful but there is even more. If one clicks on one of the names with the right mousebutton, one can inspect the properties of that object ...



The first page of the properties does not reveal anything new, but the pages "Value" and "Calling Information" are very helpful. If the object is a variable, the "Value" page displays – not surprising – the content of the variable. If the object is a function, the value page is not so interesting but the page "Calling Information" is ...



This page tells us, that the function "range" returns a result of type "range" (the German version I use displays "Bereich" for range) and that it uses one, optionally two parameters of type VT_VARIANT and the names "Cell" and "Cell2". This is very concise, but enough information for the insider.

For those who want to get extensive information, there is the button "Show Function Help". If one clicks it, one is transported to the place in the online help for Microsoft Visual Basic for Applications where the function is described in detail (most of the times, that is). That this is invaluable becomes

clear after a very short time of working with OLE, VBA and Excel, because the programmers of Microsoft seem to be totally ignorant of conventions and rules.

One of the earliest experiences one makes is, that one should not trust the information about the type of an object (variable or function). Quite often one finds out that an object of type "variable" does not behave like one, that is, cannot be queried or set by assignment but has to be handled like a function. Whether a variable really is a variable is seen easiest by looking at the "value" page. If it is a true variable, one can see the content there. If it is a "function in disguise", the value page displays the name of the object.

A special kind of object are the ones which return a result of type VT_RANGE, VT_PTR (a pointer) or VT_DISPATCH. Those results are not values which one can use directly, but new namespaces, a fact that can be very frustrating. In this case one has to place the new object namespace somewhere in the namespace hierarchy. Sometimes one has the choice where to place it, sometimes it will only work if it is a sub namespace of the parent object. This done, one has to explore the new namespace for a function or variable which gives the desired result one expected because of the name of the object. Sometimes one does not find the desired result at all because there just is no fitting object.

Working with a Spreadsheet

```
'xl' =WC 'OLEClient' 'Excel.Application'
```

starts Excel as an OLE-Client. If one executes this line, a new namespace with the name "XL" is created but nothing more happens. The reason is, that Excel wants

to be told which spreadsheet of the folder has to be opened. So the question is, how to open a spreadsheet and connect to it. The following function listing shows the necessary steps:

```
[ 0] {sheet}xlopen tab;s
[ 1] ap Open an Excel-Folder as OLE-Client
[ 2] ao CHK 17.01.1999 <chk@hoesle-kienzlen.de>
[ 3] ad sheet: The name of the spreadsheet in the folder
[ 4] ad tab : The name of the Excel file on the harddisk (including path)
[ 5]
[ 6]  $\rightarrow (2=\square WC'sheet')/\square LC+1 \circ sheet+1$  a use 1st spreadsheet if missing
[ 7] 'xl'\square WC'OLEClient' 'Excel.Application' a Create OLEClient Namespace
[ 8] xl.Visible+1 a Visible=1 for testing
[ 9] 'xl.wb'\square WC xl.Workbooks a create empty workbook
[10] 'li'xl.wb.Open tab a open folder in workbook
[11] 'xl.wb.li.sh'\square WC xl.wb.li.Sheets a create spreadsheet namespace
[12] 'act'xl.wb.li.sh.Item sheet a make spreadsheet active
[13] xl.wb.li.sh.act.Select 1 a select spreadsheet
```

As one can see, there is quite a lot to do. We already know line [7]. Line [8] is interesting for two reasons. For once one can control the visibility of the whole process. If set to one, Excel will not only be loaded but also displayed on the screen and one can follow the whole process as in a cinema. If visible is set to 0 one can see nothing and everything runs a little bit faster. One shouldn't expect wonders of speed in any case though.

Secondly line [8] shows a "real" variable and the handling of such an object: it is a variable just like any old APL variable, it can be set by an assignment or queried by its name. If one looks for interesting difficulties, one can even erase it.

In case one has set Visible to 1, an empty Excel is displayed. To see the desired spreadsheet one has to run through steps [9] to [13]. Only then can the spreadsheet be seen and worked upon (even by typing, if one wants!).

Lines [9] and [10] show another interesting aspect of VBA programming: The object "Workbooks" is a "fake" variable. There is no "Calling Information" page in the properties dialog and the "Value" page only shows the name of the object. That one has to use $\square WC$ with it can only be found out by trial and error or – much later – by experience.

"Open" in line [10] on the other hand is a function with a result type "VT_PTR to VT_COCLASS", it has to be called with the filename of the Excel workbook as if it were a normal APL function. That it needs a left argument, namely the name of the new namespace it creates can only be guessed somehow, it is not documented anywhere. The new namespace has to be created as a child of the parent namespace, it will not work anywhere else.

"Sheets" in line [11] is a variable like "Workbooks". One could place the namespace "sh" somewhere else and not under "xl.wb.li" and save a lot of typing later but it is at the logically correct place in the object hierarchy here, so let's be orthodox.

The function "Item" in line [12] again places its namespace at the end of the object hierarchy. Contrary to "Open" it returns a result of type "VT_DISPATCH", but this does not make any visible difference, both are APL namespaces. For an argument it takes a parameter "index" of type "VT_VARIANT". This can either be the name of the desired spreadsheet, or the numeric index. Dyalog APL provides the necessary type conversion.

After *xlopen* has completed, one can admire an open Excel spreadsheet on the screen.

The Excel-Spreadsheet

If one wants to, one can now work with the open spreadsheet without further preparations. One can read values from it into the APL workspace or write values into cells in the sheet. But this means that one knows exactly where to read or to write because one has to address the cells like in an APL matrix by row and column (in Excel one has to specify those by a letter-number combination like "C8"). But if somebody has inserted or deleted a row or a column, this will lead to difficulties. It is much safer and more flexible, if one uses named cells or areas and works with those names.

Let us look at the sample spreadsheet more closely. It is a template for a list with a fixed header area which will be repeated on every top of a page, and an arbitrary number of lines of data below.

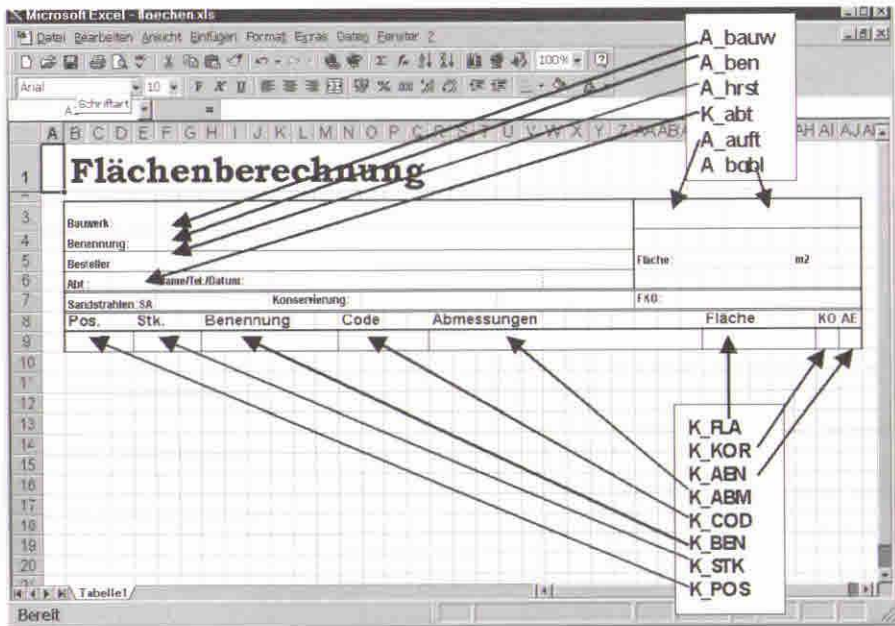
Finding the place, where that header area has to be defined in Excel has cost me more than an hour. I wonder whether Microsoft has a special department which finds new and unlikely places for important options for each new version of their products. My guess is, that they deem this to be educational. Users should be educated to become more flexible. Anyhow, in my German version of Excel 97 this header area always has the unalterable name "Druckbereich". You will have to find out what the name is in your localized version.

In addition to "Druckbereich" one should allocate a name to every cell or range of cells that one wants to access from APL. BEWARE: this name has to be unique, not only within the spreadsheet but within the whole folder! It is hard to

understand why the names do not have their own namespace in each spreadsheet since each spreadsheet has its own name itself, but it can't be helped.

If one wants to access a rectangular range of cells it is normally sufficient to name the cell in the upper left hand corner since the number of rows and columns is defined by the size of the APL matrix. In case the size is not known beforehand one can name the lower right-hand cell as well.

The following screenshot shows our template with the names pasted in:



The names can be freely chosen and Excel does not distinguish between upper- and lowercase. But the case is preserved and since APL does distinguish between the two cases one should take this into account.

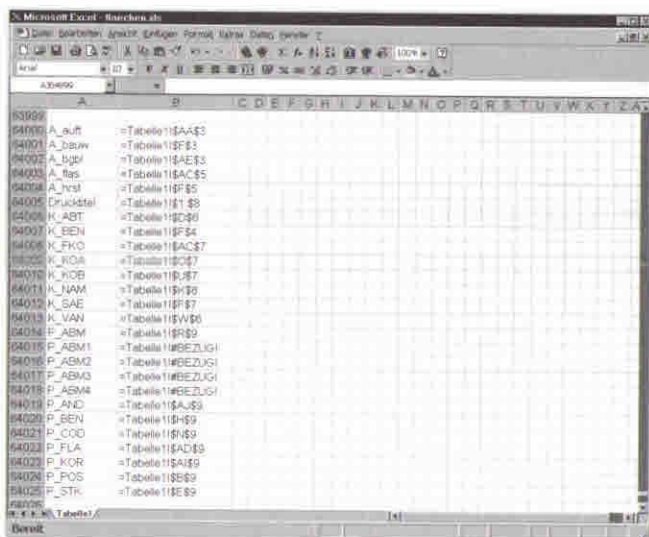
Reading from a Spreadsheet

The first question now is: If I want to read something from that spreadsheet, how do I know the names used there? One could, of course, hope for the best and use hard-coded names in the APL program. This is the simple but inelegant and error-prone method. An infinitely more elegant solution is, to read the names in from the spreadsheet. So we use the explorer and shout hooray, because we find an

object called "ListNames". Right after using the method with the promising name, we groan. Microsoft programmers have a queer sense of humour, because "ListNames" writes the list of names into the spreadsheet itself! But we are APL programmers and we always find a solution, as can be seen in the listing below:

```
[0] r←xlgetnames
[1] ap Read in all defined names from an Excel spreadsheet
[2] ao CHK 17.01.1999
[3] ad The spreadsheet has to be opened with xlopen beforehand
[4] ad "ListNames" writes the namelist into the sheet and not to APL!
[5] ad therefore we have to read them in from the sheet.
[6]
[7] 'now'xl.wb.l1.sh.act.Range'A64000' 'B64999' a define range for list
[8] xl.wb.l1.sh.act.now.ListNames a write list there
[9] r←xl.wb.l1.sh.act.now.Value a read from there into APL
[10] xl.wb.l1.sh.act.now.Delete 0 a delete range
[11] r←((0.5×pr),2)pr a create two-column Matrix
[12] r←(0×pr)r[;1]r a delete empty rows
[13] r[;2]←(r[;2])+'$')+r[;2] a delete spreadsheet name
[14] r[;2]←r[;2]~'$' a delete Dollar signs
[15] r[;2]←xlnumadr r[;2] a numeric column/row index.
```

The following screenshot shows the result of "ListNames". To get this list, one has to define a range where the function can write its result, as line [7] shows. To be out of the way of other contents, we chose a range right at the bottom of the sheet. Line [8] calls the function and line [9] reads the result from there into APL. Here is, how the result looks:

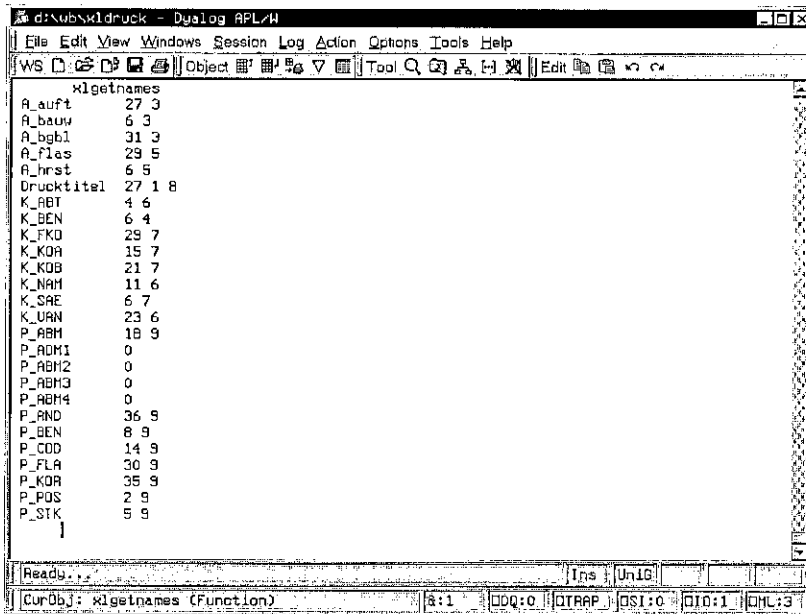


As one can see, reading data from a spreadsheet is a very simple thing: One creates a namespace with the desired range (line [8]) and uses its Value-method like an APL variable (line [9]) – that is all! The result occupies two columns. The first column contains the names themselves and the second column is the address of the respective cell. The function “Value” returns a two-element enclosed character string with the content of the two columns. This enclosed string has exactly the content one can see in the spreadsheet. Therefore we have to convert this information into a form which is easier to process in APL. Lines [11] to [15] do this cleanup. The subroutine *xlnumadr* converts Excel addresses like AD9 into numeric indices (30 9).

Since there is no way to query beforehand the number of names, one has to define a rather large area and delete the unused rows later. There are also lines with names where the address is invalid, here marked with #BEZUG! Furthermore we see a different kind of address in the line “Drucktitel”, it is not the address of a single cell but the address of a range of cells, namely \$1:\$8. Those unusable definitions are converted by *xlnumadr* into 0 since we do not need them anyway.

Line [10] deletes our auxiliary range so that the spreadsheet is “clean” again. If one looks at the description of the function “Delete” in the online help, one sees that this function returns a result of type VT_VOID, meaning that there is no result. Delete can be called without or with one parameter. APL does not provide a means for calling a function without or with a right parameter, so Dyalog APL uses a “Zilde” (∘) as a way of defining “this function is called with no parameter”. If the parameter is given, it defines the way in which the neighboring cells are treated.

The following screenshot shows the result after conversion in APL in column-row order:



Writing to a Spreadsheet

The last section described how to read data from a spreadsheet into APL. It is a relatively simple process. One defines a range of cells as a new namespace and uses its Value-method like an APL variable. Writing is the exact opposite:

```
[0] data xlwrite range
[1] ap Write data from APL into an Excel spreadsheet
[2] ao CHK 17.01.1999
[3] ad data : Character vector or matrix with the data
[4] ad range: Either the name of a cell or range of cells
[5] ad         or a two element range definition like 'A3' 'C7'
[6] ad If "range" covers a range of several cells, "data"
[7] ad has to be a matrix with so many lines and columns
[8] ad as "range" defines, even with ranges of one line!
[9]
[10] 'now' xl.wb.li.sh.act.Range range      a create range namespace
[11] xl.wb.li.sh.act.now.Value+data         a write data into range
```

As stated above, writing is the exact opposite of reading and equally simple. One defines the desired range of cells as a new namespace and assigns the data to the Value-method. The format of the data is described in the comments of the above function listing. It has to be an enclosed character matrix with exactly as many lines and columns as the defined range. Only if the range is a single cell the data may be a character string. If the data for one cell is wider than the cell size, it is displayed exactly as if it was keyed in manually: If the neighboring cell is empty, the data fills the adjacent space. If there is data in the adjacent cell, the content is cut off in the display.

Copying a Range of Cells

In our template sheet we had defined line 9 as the data area. Since we do not know beforehand, how many lines of data will have to be filled in, we have to multiply those data lines "on the fly". For this we use the function *xlcopy*. The left argument defines the source range from where to copy, the right argument is the target range. In principle one needs only the address of the starting cell of the target area since the range is already known by the size of the source range. If, however, the source range should be copied several times at once (as we want to do here), one has to give the full range into which the source has to be copied. Care has to be taken in defining an exact multiple of the source range, otherwise the operation will result in an error message which is displayed in a great big window on the screen and the process is aborted. The format of the source cells (frames, colors, patterns etc.) is copied as well, the names of cells in the source area are not copied.

```
[0]  from xlcopy to
[1]  ap Copy a range of cells to a new target area
[2]  ao CHK 17.01.1999
[3]  ad The spreadsheet has to be opened beforehand with xlopen
[4]  ad from: Source range, e.g. 'C1' 'D9'
[5]  ad to   : Target range, e.g. 'E1' (address of starting cell)
[6]  ad BEWARE: the target area is inserted, not overwritten!

[7]
[8]  'now'xl.wb.li.sh.act.Range from  a create source range namespace
[9]  xl.wb.li.sh.act.now.Copy 0      a copy to clipboard
[10]
[11] 'now'xl.wb.li.sh.act.Range to    a create target namespace
[12] xl.wb.li.sh.act.now.Insert 1     a insert content of clipboard
[13]
[14] 'now'xl.wb.li.sh.act.Range'A1'  a namespace of first cell
[15] xl.wb.li.sh.act.now.Copy 0      a empty clipboard
```

Interesting here are lines [14] and [15]. A small range (only the first cell) is defined and copied to the clipboard. If one does not do this, Excel (or rather Windows) opens a dialog box and inquires what the user wants to do with the large amount of data in the clipboard. Since we do not want to answer silly questions, we prevent this nuisance with that simple trick.

Printing the Spreadsheet

We have finally arrived at a spreadsheet, beautifully formatted and filled with highly interesting Data. The logical step that follows is, to print this out for the higher management, because, as we all know, higher management does not know how to handle computers, they prefer paper. The following simple little function automatically prints our wonderful sheet on the standard printer defined in Windows. Unfortunately one cannot specify the number of printouts here, since this is a parameter of the printer driver. But one can, of course, call this function as often as needed. The name of the standard printer can be queried from the variable "xl.ActivePrinter".

```
[0] xlprint;i
[1] ap Print the active Excel worksheet
[2] no CHK 17.01.1999
[3]
[4] i+xl.wb.li.sh.act.Index      a Index of the active sheet
[5] xl.wb.li.sh.act.PrintOut i  a print this out
```

One could as well call up the printer dialog, but we want to use this function as a subroutine within a bigger context, therefore we just use PrintOut as often as necessary. Calling the printer Dialog via OLE unfortunately is not possible.

Closing Excel

After one has filled in and printed out the spreadsheet, one should free the used resources and close Excel. The following function shows what is needed for this action.

As one can see it is not sufficient to delete the namespace "xl". Excel wants to be closed in a civilised way. Line [5] makes Excel believe that the spreadsheet, which had been changed, has been saved. Otherwise Excel would ask, if the changed spreadsheet should be saved before it is closing down. Finally we can tell Excel in line [6] that it is no longer needed and it will shut itself down without a murmur. Then, and only then should we delete the namespace.

```

[0] xlclose
[1] ap Close an Excel spreadsheet and delete its namespace
[2] no CHK 17.01.1999
[3]
[4] :Trap 0
[5]   xl.wb.li.Saved+1      a prevent "Save" dialog
[6]   xl.Quit              a shut down Excel
[7] :EndTrap
[8] []EX'xl'              a delete namespace

```

The Workspace XLCLIENT

This workspace, which can be downloaded from my website (see address at the end of this paper) contains all the functions listed in this paper and some more. The main function *PrintDemo* calls all functions needed to fill in and print out the above shown spreadsheet. Most of the work is done by the function *xlfillsheet*; the listing is not printed out here because of the space needed. But there are loads of comments in that function which explain exactly what is going on in each line. The function reads in the names from the spreadsheet as shown here, looks for variables in the workspace with the same names and writes their content into the sheet.

Special Functions

In principle, one can reach all functions which Excel offers in menus via OLE. As examples I have listed three of them here for good measure. The first fills the background of a range of cells with a pattern, the second changes the frame around a cell and the third defines new colors for the background and content of a cell. It is interesting to see what roundabout ways one has to go for apparently very simple effects.

To fill the background of a range of cells with a pattern, one has to create an "Interior" object first:

```

patt xlpattern range
ap Fill the background of a range of cells with a pattern
no CHK 17.01.1999
ad range: Either the name of a cell or range of cells, or
ad         an explicit range definition like 'A3' 'C7'
ad patt : A number that defines the style of the pattern

'now'xl.wb.li.sh.act.Range range      a create range namespace
'xl.int'[]WC xl.wb.li.sh.act.now.Interior a create interior object
xl.int.Pattern+0[18]patt              a define pattern 0..18
xl.int.PatternColor+1                 a set color to black
[]EX'xl.int'

```

Patterns are defined numerically and may have values from 0 to 18. Just try out, what the different numbers mean!

The frame of a cell is not, as one could assume, a property of that cell, but a separate object. The width of the frame influences the type of the frame, dashed frames are only possible with width 0:

```

style xloutline range
  ⚡ Define the frame around a range of cells
  ⚡ CHK 17.01.1999 <chk@hoesle-kienzlen.de>
  ⚡ range: Either the name of a cell or range of cells, or
  ⚡         an explicit range definition like 'A3' 'C7'
  ⚡ ad style: A number that defines the style of the frame

'now'xl.wb.li.sh.act.Range range  ⚡ create range namespace
'xl.bdr'␣WC xl.wb.li.sh.act.now.Borders  ⚡ create border object
xl.bdr.Weight+1[style             ⚡ set the frame style
:If style=0                       ⚡ if style = 0
  xl.bdr.LineStyle+0              ⚡ set style of line to "thin"
:EndIf
␣EX'xl.bdr'
```

Changing the colour of a cell is very complicated, it is not enough to create a text object of the cell, one also has to create the font object of the text object:

```

col xlcolor range
  ⚡ Change text- and background color of a cell
  ⚡ CHK 17.01.1999
  ⚡ ad range: range of cells, e.g. 'B3' 'C8'
  ⚡ ad col : 2-elem. vector with number of color for text & background

'now'xl.wb.li.sh.act.Range range  ⚡ create range namespace
'char'xl.wb.li.sh.act.now.Characters  ⚡ create text object
'xl.fnt'␣WC xl.wb.li.sh.act.now.Char.Fnt  ⚡ and its font object
xl.fnt.ColorIndex+1[col           ⚡ color of font
'xl.int'␣WC xl.wb.li.sh.act.now.Interior  ⚡ interior-object
xl.int.ColorIndex+2>2+col         ⚡ color of background
```

Finally

Well I hope you liked this excursion into the OLE jungle and perhaps one or the other of the participants have seen just enough on this trip to come back and investigate it further. If so, I hope to be informed of the results.

Those who want to experiment further can download the Dyalog APL Workspace (Version 8.2!) together with the Excel spreadsheet from my website:

<http://www.hoesle-kienzlen.de/projects.htm>

A PDF Primer for APLers

by Adrian Smith (adrian@causeway.co.uk)

notes to accompany seminar given at Finnish Forests VI and APL Germany

Background

Adobe's "Portable Document Format" is now nearly 10 years old and has become a very well-supported way of transmitting a document between any two computers so that "What You See is What They Get". In this way it differs fundamentally from HTML, where the person viewing the document has a great deal of control over the appearance. A PDF will look the same on a PC, a Mac, or a whole variety of Unix boxes.

Almost everyone seems to have downloaded Adobe's free PDF Reader application, so there is rarely a problem in posting documents on your website in this format, for example to provide a 'printed manual' in a downloadable form. However, PDFs have one very strong selling point just now – the format is sufficiently simple that there is nowhere to hide a virus. You can safely mail them as attachments without endangering your friends.

There are plenty of tools out there which can take a PostScript file and magically 'distil' it into the PDF format. Adobe Acrobat is by far the best and does an extremely competent job. However it is quite expensive and adds considerably to the complexity of any batch-production process (for example you might want to email a few hundred reports) where you need to create lots of little PostScript files and wait around for Distiller to do its magic.

What is less well known is that PDF is actually a published format, and can be written as pure ASCII text. The construction process is by no means as easy as HTML, but (in principle) any environment which can create a text file can generate a valid PDF. Naturally, this includes APL+Win, as this short session will demonstrate. What I cannot do in 45 minutes is to show you everything you might need to construct PDF files entirely for yourself. If you really want to implement this in production code, you will still have to do some serious research. Hopefully, I can help you to look in the right places.

Let's Take One Apart ...

There is no better way to learn any format than to take it apart, and that is what the rest of this session will be about. What I have done is made the simplest PDF I possibly could using Causeway's NewLeaf workspace and some code like

```
nlUse letter
nlPlace 'Hello, world'
PG+nlClose
pdf+MakePDF PG
```

This writes 'Hello, world' in 12pt Times-Roman at the top of a sheet of US Letter paper. It then converts the NewLeaf intermediate format into a simple character vector of PDF code, which is actually a mere 991 bytes long. I have intentionally used linefeed-delimited vectors for the final result, as they are easy to browse with jedit and can be written directly to file in this form. To see it on screen, simply save it to file and load it into the Adobe viewer.

HelloWorld example

```
%PDF-1.1
%Causeway

1 0 obj  % document info
<<
  /Creator {Causeway NewLeaf}
  /Producer {Causeway PDF Filter}
  /CreationDate {D:200110071818}
>>
endobj

2 0 obj  % Root node
<<
  /Type /Catalog
  /Pages 4 0 R
>>
endobj

% ===== Resources =====

3 0 obj  % Font
<<
  /Type /Font
  /Subtype /Type1
  /Name /F1
  /BaseFont /Times-Roman
  /Encoding /WinAnsiEncoding
>>
endobj

% ===== Content =====

4 0 obj  % Page list
<<
  /Type /Pages
  /MediaBox [ 0 0 612 792 ]
  /Count 1
  /Kids [ 5 0 R ]
  /Resources
  <<
    /ProcSet [ /PDF /Text ]
    /Font << /F1 3 0 R >>
  >>
>>

>>
>>
endobj

5 0 obj  % Page 1
<<
  /Type /Page
  /Parent 4 0 R
  /Contents 6 0 R
>>
endobj

6 0 obj  % Stream
<< /Length 62 >>
stream
1 J 1 j
BT
/F1 12 Tf
1 0 0 1 72 705.6 Tm (Hello, world) Tj
ET
endstream
endobj

xref
0 7
0000000000 65535 f
0000000021 00000 n
0000000155 00000 n
0000000252 00000 n
0000000405 00000 n
0000000571 00000 n
0000000649 00000 n

trailer
<<
  /Size 7
  /Info 1 0 R
  /Root 2 0 R
>>
startxref
772
%%EOF
```

As you can see, PDFs tend to be quite long and thin, which is why I have switched to double-column to show it! Have a careful look at the overall structure (a highlighter pen could be very handy here) and some things will slowly begin to make sense. Some things I will just have to explain, like that xref block at the end. Ignore this for the moment, and observe that there seem to be lots of logical blocks delimited by “# 0 obj endobj” markers, where # is an identifying number that starts at 1.

Let's begin at the beginning and explain the structure as we go. Actually, we should perhaps begin at the end (PDFs are actually designed to be read from the back) but that would be too confusing. So from the top it is:

```
%PDF-1.1
%Causeway
```

That first line is mandatory and tells the reader that we are working within quite an old standard. This will all work fine in Acrobat Reader 2, and I normally use 4, although 5 is fine too. The second line has a gratuitous 'hibit' character which is designed to stop mail programs from treating the file as text, and generally trashing it.

```
1 0 obj  % document info
<<
  /Creator  (Causeway NewLeaf)
  /Producer (Causeway PDF Filter)
  /CreationDate (D:200110071818)
>>
endobj
```

This section is entirely optional, but it shows up in the 'document properties' screen and I always feel it is a good idea to populate these things. It also illustrates several other points:

- Each object has a unique number and a version number (always 0). Comments are preceded by % symbols and the viewer ignores from the comment to the end of that line.
- Objects typically consist of dictionaries which are delimited by double angle-brackets. A dictionary consists of one or more name-value pairs; names are case-sensitive and always have a leading slash symbol. This is very easy to forget!
- Strings (any text) are placed between parentheses. If you want parentheses within a text string, you must escape them with a backslash. Of course the same applies to the backslash itself!
- The layout is quite immaterial, but I find it helps to indent things like this.

I suppose I should show you some APL code ... here are the functions which created that info block:

```

      v pdfΔMakeInfo;obj
[1]  ⍝ Make document info object
[2]  obj←('/Creator' '/Producer'),[1.5]'{Causeway NewLeaf}'
      '{Causeway PDF Filter}'
[3]  ⍝ Date in ANSI standard format (local time)
[4]  obj←obj, '/CreationDate'('(D:',(,'ZI4,ZI2,ZI2,ZI2'⎕FMT
      1 5p⎕TS),')')
[5]  'document info'pdfΔcatobj pdfΔMakeDict obj
      v

      v dict←pdfΔMakeDict pvm
[1]  ⍝ Make a dictionary out of content (2-column matrix of pv pairs)
[2]  pvm[;2]←' ',pvm[;2] ⋄ dict←c[2]pvm
[3]  dict←ε(cnl,' '),dict
[4]  dict←'<<',dict,nl,'>>',nl
      v

```

Now for the ancestor of the rest of the document ...

```

2 0 obj  ⍝ Root node
<<
  /Type /Catalog
  /Pages 4 0 R
>>
endobj

```

If you turn back to the complete listing, you will see that this is pointed to from the trailer section of the file – as I said, these are designed to be opened at the back, like any good newspaper!

What this is telling us is that it is the 'catalog' for the entire file and that the only other thing of interest is some pages, which we should look for in object number-4. There are plenty of other things that might be catalogued, such as outlines, but I did say this was a simple one to get us started! On to the next

```

% ===== Resources =====

3 0 obj  ⍝ Font
<<
  /Type /Font
  /Subtype /Type1
  /Name /F1
  /BaseFont /Times-Roman
  /Encoding /WinAnsiEncoding
>>
endobj

```

Usually this would just list all the fonts which you are using in the document – here there is just one. This block needs explaining in detail, as fonts are crucial to the success of PDF publishing.

- It is a font (obvious), but specifically it is an Adobe Type-1 PostScript font. We shall refer to it with the mnemonic 'F1' from now on and the Reader can use Times-Roman to render it. This is a very important feature of PDF documents – if you restrict yourself to the 14 in-built fonts (Times, Helvetica, Courier, Symbol, ZapfDingbats) then your document will be easily readable by anybody. What is more, you do not need to include the definition of the font (typically around 40K each) so your PDF is very small.
- It is encoded to the Windows standard. This is not the default, so we should always include this entry too.

If you have used several fonts, there will be several very similar objects to define them. You can (of course) bundle in PostScript fonts here, but do not even think about bundling TrueType fonts. They go into the PDF as bitmaps and look terrible in the viewer. To be avoided at all costs!

On to the real meat now!

```
% ===== Content =====  
4 0 obj  % Page list  
<<  
  /Type /Pages  
  /MediaBox [0 0 612 792]  
  /Count 1  
  /Kids [5 0 R ]  
  /Resources  
<<  
    /ProcSet [/PDF /Text]  
    /Font << /F1 3 0 R >>  
  >>  
>>  
endobj
```

This lists all the pages in the document, and defines any resources which are common to them. You can insert these settings for each page, but I cannot see that the paper-size will change mid-stream for anything I will generate, so I put them here to save space. Again, working through the detail:

- The *MediaBox* is what most of us would call the paper-size. Divide by 72 and you will recognise US Letter. Note the notation for an array – you delimit it with square brackets. PDF has heterogeneous and nested arrays, just like APL.
- The *Count* is the page count, and the *Kids* array is a set of object references to each page. This entry can grow quite large for a big document! Each reference gives the object number and version followed by R which is the Reference operator.
- The *Resources* entry is itself a dictionary (these can be nested too) and lists the PDF procedure sets which will be needed to render our pages (these are

very standard) and any fonts which will be needed (again referencing the font objects by number)

So now we have almost reached the metal! You might confidently expect to see 'Hello, world' somewhere in the next object, but not quite. This onion has yet one more layer to peel:

```
5 0 obj    % Page 1
<<
  /Type /Page
  /Parent 4 0 R
  /Contents 6 0 R
>>
endobj

6 0 obj    % Stream
<< /Length 62 >>
stream
1 J 1 j
BT
/F1 12 Tf
1 0 0 1 72 705.6 Tm (Hello, world) Tj
ET
endstream
endobj
```

The *Page* object connects back to its parent (the *Pages* object) and points forward to a sibling which actually has the content we are looking for. This starts off with a dictionary to say how long it is (exact byte-count, including the line-feed characters from *stream* to *endstream*, but not including these two words) and then we have some marvellous mumbo-jumbo that actually puts the marks on the paper. Let's pause for breath and consider this piece very carefully indeed!

- The first thing I do is set all line-ends and line-joins to 'rounded' which fits with the way Windows does it and in general makes for nicer looking tables and line-graphs. In this particular PDF, it has no effect whatsoever! Note that we need to think left-to-right here – all the PDF functions take their arguments on the left, so **J** and **j** are the functions.
- **BT** is a niladic function which sets us into text mode. This is one of the slightly strange things about PDF which must be a viewer optimisation – you can't mix text and graphics in the same block.
- **Tf** is the function to set the working font. Here it is given two arguments, the font name and font size (in points). Remember that we named it in that font resource object a little while ago.
- **Tm** positions us on the page. Ignore the 1 0 0 1 (this sets stretch and skew) and notice the last two numbers 72 705.6 which are the (*x,y*) co-ordinates of the start of the text baseline. These are in points, measured from the *lower* left-hand corner of the paper. NewLeaf has a default left margin of 1" which leads directly to the number 72 here. The top margin is also 72pt, and if you

fire up an APL interpreter and try $(792-72) - 12 \times 1.2$ you will discover where that second number came from – 792 is the height of the paper, and 1.2 is the default ‘leading’ so a 12pt font is vertically spaced on rails 14.4pt apart.

- Tj actually writes the text. Its argument is a simple string, in this case the text we started with. Do remember to ‘escape’ parentheses in all strings here!
- ET ends the text block, which would be necessary if there were any drawing operations to follow, for example table boundaries or even text underlining.

That is really the end. Now we have some special stuff which is purely there to help the viewer handle large files efficiently,

```
xref
0 7
0000000000 65535 f
0000000021 00000 n
0000000155 00000 n
0000000252 00000 n
0000000405 00000 n
0000000571 00000 n
0000000649 00000 n
```

After the xref line you get a count of the number of entries followed by a table in a very fixed format. Ignore the ‘zeroth’ row which is some kind of placeholder. What we then have is the exact byte-offset into the file (or vector in the workspace) of each object. For example:

```
      12+252+pdf
3 0 obj %
```

... leads us directly to object number 3. Note that the objects need not be in order here, it was just convenient for me to make the file this way.

Finally, we end the file with a trailer which a PDF-reader application can use to find its way quickly to the xref and the root note of the data:

```
trailer
<<
  /Size 7
  /Info 1 0 R
  /Root 2 0 R
>>
startxref
772
%%EOF
```

Here is the APL code which formatted it:

```

v pdfΔMakeXref;xref;xp;mat
[1]  a Finish off the xref table
[2]  a We are using LF-delimited data here so we need a trailing blank.
[3]  xp+pdfΔCurrSize o xref+'xref',nl,'0 ',(v1+pXREF),nl
[4]  a This is the only place where the formatting is very fussy!
[5]  mat+0 65535,[1]XREF,[1.5]0
[6]  mat+1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 0 0 0\ZI10,ZI5'FMT mat
[7]  mat[;18]+'n' o mat[1;18]+'f'
[8]  pdfΔcat xref,,mat,nl
[9]  pdfΔcat'trailer'
[10] pdfΔcat '~1+pdfΔMakeDict'/Size' '/Info' '/Root',[1.5]
      (v1+pXREF)'1 0 R' '2 0 R'
[11] pdfΔcat'startxref',nl,(vxp)
v

```

Each line in the xref must be exactly 20 bytes long (including the newline), hence the very careful use of format and expand.

This was Just Scratching the Surface

As I said at the start, this is all firmly in the category 'straightforward but very tedious'. Writing the PDF driver for NewLeaf was not particularly hard, as NewLeaf has already done the tough part, and has worked out where all those words and lines physically go on the paper. I added some extra stuff to do the bookmarking (and hence generate the outlines automatically) and left it at that. There is lots more you can do with this format, like fancy gradient fills and links to external resources like webpages. If you want to make PDFs and are feeling lazy, have a look at NewLeaf and RainPro and see if they fit in to your existing reporting engine. If you are feeling brave, dig out the PDF specification from the Adobe site, and start from where these notes end.

If in doubt, call the number below!

Adrian Smith
 Email:adrian@causeway.co.uk
 Tel: +44 (0) 1653 696760

APL Germany in Munich

30th April 2002

notes by Adrian Smith

Announcement:

APL2 for the Workstation, Version-2

Full details of the product are best researched on IBM's website, but this was a piece of APL news well worth hearing. The new release comes on a single CD which installs on AIX, Linux, Solaris and Windows. ("If anybody asks about OS/2, well OS/2 isn't there anymore"). Some of the key features are:

- full support for Namespaces, implemented in the same way as in mainframe APL2. Namespaces can be created in any of the environments and can be run using a supplied runtime packager.
- The runtime library (for all platforms) is FREE. This allows you to develop your application on the platform of your choice, move it to any other platform via)OUT and)IN and then create an appropriate runtime application from it. Obviously, you will need to install your APL2 on each target platform to do this, but see the licence conditions below.
- an interface to TCL/TK is included, so that you may build cross-platform Gui applications using an established Gui toolkit. Your TCL front-end can include callbacks to run APL2 functions.

What is more, this APL2 release is included in the IBM "Scholars Programme" which means it is free for educational use. For the rest of us, the price is \$1,500 per developer (note this means what it says, so you are allowed to work on multiple machines or operating systems for the price of a single copy).

This APL looks like a serious contender for writing the kind of "lights-out" number-bashing code which APL is so good at, and for deploying the results on any of the major Unix platforms. The provision of namespaces is an excellent step forward, and I am glad to report that at least some voices (maybe even a majority) were raised in favour of Control Structures as the next likely GSE requirement. Meanwhile, the BAA has begun the process of finding out just what is involved in signing up a UK school for the free educational licence.

APL2000 Inc

APL+Win 4.0 for Windows 95/98/NT/ME and 2000

- ActiveX APL Grid Object
- ? Button for "What's This?" help
- Noredraw property for visible controls
- New connection with printer dialog boxes
- Combo box supports images and indents
- Greatly increased maximum file size
- Calling syntax for ActiveX objects
- Full compliment of printed documentation

APL+LinkPro 3.0

- Link between APL+Win and non-APL databases
- Open Database Connectivity (ODBC)
- Wide variety of databases on different platforms

APL+UNIX 5.3

- Control Structures (compatible with APL+Win)
- Interface for TCP/IP communications using sockets
- Partition function (APL2 compatible)
- ⌈AT – Attributes (APL2 compatible)

APL

APL Systems IDC SL

Alfredo Marquerie, 12 – 2F
28034 Madrid, Spain
+34 91 730 7008 Voice
+1 775 743 6131 Fax
+34 60 680 5949 Cell
uksales@apl2000.net
uksupport@apl2000.net

Coming Soon! **APL+Web Services**

GENERAL ARTICLES

This section of Vector has articles of general interest, usually with relatively little in the way of computer code or complex algorithms. Our hope is that this section can be read with enjoyment by anyone who happens to pick up the magazine, even if they have never heard of APL or J.

Playing with the Beast Number with J

by Michel Dumontier (michel.dumontier@free.fr)

Summary

In the end of the bible, there is the book of numbers.
We can make the numbers tell anything... but with J it is more entertaining!
It can be also the occasion to learn J through playing.
My purpose: the reader has fun with J.

Palindromes and Anti-palindromes

Viewed in its decimal form, the beast number: 666, nothing is blindingly obvious!
but just it is a palindrome.

Let us calculate 666^{666} and the number of digits when written in decimal form.

```
b=:666x
bp=:b^b
digits=: #@":
digits bp
1881
```

The number of digits in decimal form of 666^{666} is still a palindrome!
Putting b in binary form

```
]bn=:#:b
1 0 1 0 0 1 1 0 1 0
```

It is anti-palindromic

```
#bn NB. Counting the number of bits.
10
```

verifying it is an anti-palindrome:

```
hbn=.5{.bn
^/(hbn,|.hbn)=bn
1
```

The other previous anti-palindromic numbers are: 2 10 12 38 42 52 56 142 150 170
178 204 212 232 240 542 558 598 614

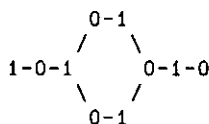
So as not to use too much space, let us use transpose and read the numbers vertically

```

|: #:2 10 12 38 42 52 56 142 150 170 178 204 212 232 240 542 558 598 614
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 1 1
0 0 0 1 1 1 1 0 0 1 1 0 0 1 1 0 1 0 1 0 1
0 0 0 0 0 1 1 0 1 0 1 0 1 0 1 0 1 1 0 1 0
0 1 1 0 1 0 1 1 0 1 0 1 0 1 0 1 0 1 1 0 0
0 0 1 1 0 1 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1
1 1 0 1 1 0 0 1 1 1 1 0 0 0 0 1 1 1 1 1
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

```

To see better the anti-symmetric form of the beast number, let us put it under a chemical-like form (a la Kekule)



Can one guess why 666^{666} represents the total destruction?

Think to world and anti-world...

The left middle is 1 and has "valence" 3 with 0s as atoms and the right middle is the contrary, a 0 with "valence" 3 with 1s as atoms.

The first "beastly palindromic prime" of the form $1[0\dots 0]666[0\dots 0]1$ is 16661
It is the 1928th prime:

```

p:1927 NB. J works in origin 0
16661

```

But, $1+6+6+6+1=1+9+2+8$

The next prime of this form has 13 zeros twice, that is:

```
10000000000000666000000000000001
```

Harvey Dubner determined that for the 7 first primes of this form the number of successive 0s is: 0, 13, 42, 506, 608, 2472, 2623.

If we write bp in binary form, it contains 6247 binary digits:

```

$bbp=:#:bp
6247

```

how many times does the binary form bbp contain the beast number b?
Remember that bn was the binary form of b, then

+ /bn E. bbp
11

and 11 is 66%6

Note that 11, 111... are numbers derived from the beast number.

Other curiosities:

b^6 NB. there are 6 6s in 666⁶
87266061345623616

Note that in the previous sentence there are also 6 6s

In classical notation $1^6 - 2^6 + 3^6$

- /(>:i.3)^6
666

(+ /6 6 6) ** / (6 1)^2
666

(!6) * (+ / (6 1)^2) % + / (6 2)^2
666

+ / (p:i.7)^2 NB. the sum of the squares of the 7 first primes
666

The sum of the 144 (= (6+6)*(6+6)) digits of Pi.

12 12\$pi144 NB. 144 first decimals of Pi.
1 4 1 5 9 2 6 5 3 5 8 9
7 9 3 2 3 8 4 6 2 6 4 3
3 8 3 2 7 9 5 0 2 8 8 4
1 9 7 1 6 9 3 9 9 3 7 5
1 0 5 8 2 0 9 7 4 9 4 4
5 9 2 3 0 7 8 1 6 4 0 6
2 8 6 2 0 8 9 9 8 6 2 8
0 3 4 8 2 5 3 4 2 1 1 7
0 6 7 9 8 2 1 4 8 0 8 6
5 1 3 2 8 2 3 0 6 6 4 7
0 9 3 8 4 4 6 0 9 5 5 0
5 8 2 2 3 1 7 2 5 3 5 9

+ / p144
666

1998 is a multiple of the beast number: it is 3 times b.

We are the privileged guys who have known 2 successive palindromic years i.e. 1991 and 2002 containing a Beast year: 1998. (And also exceptionally a Leap year 2000, that is also a passage into a new millennium and century).

```

      v Z+LP X
[1]   Z+≠/0=4 100 400 4000|X
      v

```

(Note that ≠ can be replaced here by -)

(A year is alternatively leap or not if it is divisible by 4, 100, 400, 4000)

(1900 was not a leap year, 2000 is). J version below.

```
lp=:3 : '-.@=/0=4 100 400 4000|y.'
```

(Note that -.@= can be simplified by ~:)

```

dif=: 3 : '{1}. y.) - _1}. y.'
]an=:4 2 $ 1991 2002 2992 3003 3993 4004 4994 5005
1991 2002
2992 3003
3993 4004
4994 5005
,dif "0 1 an
11 11 11 11

```

Those living in these intervals can hope to see two successive palindromic years. The others have to wait 110 years!

```

dif 2002 2112
110

```

In the early years (before 1000) you could see several!

```

dif 666 676 686 696 707 717
10 10 10 11 10

```

Frank Fiederer points out that the age of the United States in 1998 (which is 3 times the Beast number) is also related to 666 since

```

(1998-1776)=666%3
1

```

Yet another property on 1998 related to 666 by Monte Zerger.

If we associate 3 with the letter A, 6 with the letter B, 9 with the letter C...

The sum of the letters of the year NINETEEN NINETY EIGHT is equal to 666

```

NB. The letter A is at the index 65.
+/3*(_64+a.i.'NINETEENNINETY EIGHT')
666

```

This digression on 1998 is just to point out that I intentionally wrote my 2nd paper on the Beast number in the Beast year 1998.

666 is the sum of two successive palindromic primes:

Successive palindromic primes: 2, 3, 5, 7, 11, 101, 131, 151, 181, 191, 313, 353, etc...

$$\begin{array}{r} +/313 \ 353 \\ 666 \end{array}$$

The triplet (216, 630, 666) is a Pythagorean triplet:

$$\begin{array}{r} (6*6*6)^{\wedge} \ 1 \ 2 \\ 216 \ 46656 \\ (666-6*6) \ ^{\wedge}1 \ 2 \\ 630 \ 396900 \\ 666^{\wedge}2 \\ 443556 \\ +/(216 \ 630)^{\wedge}2 \\ 443556 \\ b^{\wedge}2 \ 3 \\ 443556 \ 295408296 \end{array}$$

Defining $Sk(n)$ the sum of the k th powers of the digits of n

We have $S1(666)+S3(666)=666$

$$\begin{array}{r} s=:6 \ 6 \ 6 \\ (+/s)++/s^{\wedge}3 \\ 666 \end{array}$$

We have also $S3(666^{\wedge}2)+S1(666^{\wedge}3)=666$

In other words: the sum of the cubes of the digits of the square of 666 plus the sum of the digits of the cube of 666 is equal to 666.

$$\begin{array}{r} (+/(4 \ 4 \ 3 \ 5 \ 5 \ 6)^{\wedge}3)+(2+9+5+4+0+8+2+9+6) \\ 666 \end{array}$$

Proving: let us calculate the number of digits of $b^{\wedge}2$ and $b^{\wedge}3$

$$\begin{array}{r}]d2=:digits \ b^{\wedge}2 \\ 6 \\]d3=:digits \ b^{\wedge}3 \\ 9 \\ (+/(d3\$10)\#:b^{\wedge}3) \ +/((d2\$10)\#:b^{\wedge}2)^{\wedge}3 \\ 666 \end{array}$$

Another number having this property is 2583:

$$\begin{array}{r} c=:2583x \\ d2=:digits \ c^{\wedge}2 \\ d3=:digits \ c^{\wedge}3 \end{array}$$

```
(+/(d3$10)#:c^3) ++/((d2$10)#:c^2)^3
2583
```

By P. De Geest we have also $S1(666^{47})=S1(666^{51})=666$

```
d47=:digits b^47[d51=:digits b^51
+/(d47$10)#:b^47
666
+/(d51$10)#:b^51
666
```

Note that: $(4+7)*(5+1)=66$

P. De Geest and G. L. Honaker Jr. have found the following:

If we denote by $SP(n)$ the sum of first n palindromic primes,

Then $S3(SP(666))$ is equal to 3 times 666

Note that 3 and 666 appear on both sides of the equation.

Here is a function that calculates the n first palindromic primes.

```
palp=: 3 : 0
l=:0$0[i=:0
while. y. > $l do.
p=:x: p:i
i=:i+1
if. p= x: ". |. ": p do.
l=:l,p
end.
end.
)
t=:palp 111
+/t NB. The sum SP(111)
4641020
```

The function calculates easily the 113 first ones but spends a long time to find the 114th, then I abandoned to verify the above formula.

Challenge: write a function which proves that $S3(SP(666)) = 3 * 666$

Definition: two consecutive integers having the same sum of prime factors are called Ruth-Aaron pairs.

By Carlos Rivera: 20772199 is the smallest integer with the property that the sum of the prime factors of n and the sum of the prime factors of $n+1$ are both equal to 666.

```
n=:20772199
(+/q:n), +/q:n+1
666 666
(q:n) ; q:n+1
+-----+
|7 41 157 461|2 2 2 5 5 283 367|
+-----+
```

Then (20772199, 2077220) is the smallest beastly Ruth-Aaron pair.

The sum of the 666 first primes contains 666:

```

q:+/p:i.666
23 191 349
191*349
66659

```

Three funny sums!

```

+/1 2 3 4 567 89
666
+/123 456 78 9
666
+/9 87 6 543 21
666

```

A Smith number is an integer in which the sum of its digits is equal to the sum of the digits of its prime factors. 666 is a Smith number.

```

q:666
2 3 3 37
(+/6 6 6)=+/2 3 3 3 7
1

```

The sum of the 6 first Roman numerals is 666

```

D=:500[C=:100[L=:50[X=:10[V=:5[I=:1
+/D,C,L,X,V,I
666

```

A chronogram in English for 666 for instance is what you should do when you encounter the number 666: "Expect the Devil" (+/X,C,D,V,I,L).

(the value of the chronogram is the sum of the roman letters encountered in the sentence, U,V and I,J are considered the same letter).

The sum of the n first integers is named the nth triangular number

In classical notation: $T(n) = n(n+1)/2$

Then the 36th triangular number is 666

```

+/>:i. */6 6
666

```

Definition: a repdigit is a number consisting of a single repeated non zero digit as 666 or 22 or 33.

In 1975, Bailey and Weger proved (see J. Rec. Math, Vol. 8, No 2): that 666 is the largest triangular number that's also a repdigit.

Then the golden number can be written as

$-(\sin(666)+\cos(6*6*6))$

$-(10.666*0.180)+20. (*666)*0.180$

1.61803

gr

1.61803

The evil eyed numbers family

Considering the beastly evil eyed number which is 60606

q:60606

2 3 3 7 13 37

* / 2 3 3 37

666

Then 60606 contains 666, 60606 is equal to $7*13*666$

60606 is a 7-gonal number:

7 Polyg 156

60606

It can be put under the following form:

$(>:6) \text{ Polyg } * / (+/66) * >: (+/66)$

60606

We have seen above that 666 was the sum of two successive palindromic primes

It is the same for 60606:

$(+/313353), +/3020330403$

666 60606

60606+1 is a prime number.

q: >: 60606

60607

The period length of the decimal expansion of $1/60607$ is 60606

(challenge: write a J expression that proves it)

Patrick De Geest points out that 666666 is a palindrome both in base 10 and in hexadecimal, proving it:

`hfd=:16&#.('@0123456789ABCDEF'&i.)^:_1`

`hfd 666666`

A2C2A

He also notes that in base 31 it is MBMB, which is like 666666 formed with two consecutive identical parts (MB). Perhaps MB = Multiple Beast? (666 666)

```

b31=:31&#.@('0123456789ABCDEFGHIJKLMNQRSTU'&i.)^:_1
      b31 666666
MBMB

```

As a gift, I give you the possibility to try with greater numbers: here are two functions I wrote 15 years ago in APL and I have translated them directly in J. They give the possibility to transform a decimal to any base until a limited base L and the inverse.

```

CH=: '0123456789ABCDEFGHIJKLMNQRSTUvwxyz[\]^_`abcdefghijklmnopqrstuvwxyz~'
DFGREP=:3 : (';' 'x. #. (CH i. y.)')
GREPFD=:3 : (';' '(((x. DGTS y.)$x.)#;y.){CH')

```

The DGTS function calculates the numbers of digits necessary to represent a decimal number y. in base x.

```

DGTS=:3 : (';' '>:<.x.^.y.+y.=0')

```

Let us try:

```

      31 GREPFD 666666
MBMB
      31 DFGREP 'MBMB'
666666

```

NB. Note that using GREPFD, you can't use a base greater than 70

```

      $CH
70
      69 DFGREP 'Jz'
1378
      69 GREPFD 1378
Jz

```

Here are the APL functions:

```

v Z←X GREP Y;⌈IO
[1]   ⌈IO←0
[2]   Z←'0123456789ABCDEFGHIJKLMNQRSTUvwxyzabcdefghijklmnopqrstuvwxyz~'
uvwxyz€_@#'[((X DIGITS Y)ρX)↑Y]
v
v Z←X DIGITS Y ;⌈IO
[1]   ⌈IO←0 ∘ Z+1+⌊X*Y+Y=0
v

```

A challenge: try to write both functions DFGREP and GREPFD in J that allow us to work on base > 62, at least until 66.

Better, write them to allow extended integers.

Still better, look at the elegant writing of dfh and hfd and also hex that allows us to make operations on hexadecimal numbers.

```
dfh=:16&#.@('0123456789ABCDEF'&i.)
hfd=:dfh ^:_1
hex=:&.dfh
      'AB' *hex 'CD'
88EF
```

NB. (in French 8 is pronounced as 'OUI'=YES and read the result above!)

Still better I said: try to write DFGREP and GREPFD extended to 66 base, in this J style

We shall read carefully the 'technical correspondence' column of Vector
As a reward, here is the other APL function.

```
▽ Z←X DFGREP Y; IO
[1]   IO←0
[2]   Z←X⊥'0123456789ABCDEFGHIJKLMNPOQRSTUVWXYZabcdefghijklmnopqrstuvwxyz_@#'\Y
qrstuvwxyz_@#'\Y
▽
```

A last one, just for fun:

If you assign 6*6 to A, B=A+1, etc...the sum of the letters of the following word is 666

```
+ / 36 + ( _ 65 + a. i. 'SUPERSTITIOUS' )
666
```

I could not find a better formula for the French equivalent word!

```
+ / < . 3 . 25 * ( _ 64 + a. i. 'SUPERSTITIEUX' )
666
```

But it looks like a fixing thing!

Another better one in French (after many tries!):

```
+ / 66 + ( ( * / 5 5 ) - a. i. 'VOICI LE NOMBRE DE LA BETE' )
666
```

Conclusion

We can make the numbers to tell all what we want!

Exercises

- 1) Try to rewrite the functions of the text above in a more elegant form
- 2) Defining $Pi(n,d)$ as the d consecutive decimal digits of π starting at the n th digit after the decimal point: proves that $Pi(666,3)=7*7*7$

* / 666 3.66366366366366
2440

Prove that $Pi(2440,3)=666 \Leftrightarrow Pi((666*3.66366366366...),3)=666$

- 3) Theorem (found by Jud McCranie) every positive integer occurs as the period length of the reciprocal of some prime.

Found in June 1998: The smallest prime p such that $1/p$ has the period 666 is 90365999773. Write a J expression to prove it.

(from Simon Whitechapel) the beginning of the sequence of primes such that $1/p$ have the period $p-1$ is

7, 17, 19, 23, 29, 47, 59, 61, 97, 109, 113, 131, 149, ...

The last one, $1/149$ has a period 148.

Verify that the sum of those digits is 666.

If we build a 148 148 matrix which rows are composed of the decimal digits of $1/149, 2/149, 3/149, \dots, 148/149$, the result is a pseudo magic square in which every row and column sums to 666.

- 4) The alphabetic below has a unique solution: find it.

```

  SIX
  SIX
  SIX
+ BEAST
-----
  SATAN

```

- 5) Find starable fractions in which stars (representing multiplication) can be interspersed in both numerator and denominator to produce an expression that's equal to the original fraction:

Example:

```

  6*6*6
  ---- = ----
  64676    6*46*76

```

References

- [1] *De certaines propriétés du nombre de la bête.*
Michel J. Dumontier 'Les Nouvelles d'APL' N° 21 pp. 99-102
- [2] *De certaines propriétés du nombre de la bête (suite)*
Michel J. Dumontier 'Les Nouvelles d'APL' N° 28 pp. 69-72
- [3] The page of Mike Keith on the *Number of the Beast* :
<http://users.aol.com/sbsj7gt/mike666.htm>

The Abstraction Mechanisms of Vector Pascal

Paul Cockshott, University of Glasgow, Imaging Faraday Partnership

Abstract

Vector Pascal is a language designed to support elegant and efficient expression of algorithms using the SIMD model of computation. It imports into Pascal abstraction mechanisms derived from functional languages having their origins in APL. In particular it extends all operators to work on vectors of data. The type system is extended to handle pixels and dimensional analysis. Code generation is via the ILCG system that allows retargeting to multiple different SIMD instructionsets based on formal description of the instructionset semantics.

1 Introduction

The introduction of SIMD instruction sets [10][1][21][11] to Personal Computers potentially provides substantial performance increases, but the ability of most programmers to harness this performance is held back by two factors. The first is the limited availability of compilers that make effective use of these instructionsets in a machine independent manner. This remains the case despite the research efforts to develop compilers for multi-media instructionsets [6][19][18][22]. The second is the fact that most popular programming languages were designed on the word at a time model of the classic von Neuman computer rather than the SIMD model.

The functions of a high level programming language (HLPL) are:

1. To lighten the mental load of the programmer.
2. To help obviate program errors.
3. To allow programs to run fast by making efficient use of hardware.

The abstraction mechanisms of HLPLs make the thinking through of a problem, and its solution by algorithmical means, easier. They allow a problem to be thought of as one concrete instance in a more general family, and, as such, reveal that it can be solved by some very general, powerful and well understood algorithmic techniques. The type and abstraction mechanisms of HLPLs condition the thought processes of their users.

If we consider popular imperative languages like Pascal, C, Java and Delphi we see that:

- They are structured around a model of computation in which operations take place on a single value at a time. The abstraction mechanism they use for bulk data operations is serialisation using `for` loops. This reflects the word at a time model of earlier computers.
- Their type system provides inadequate support for pixels and images.

These limitations make it significantly harder than it need be to write image processing programs that will use SIMD parallelism.

Vector Pascal aims to provide an efficient and elegant notation for programmers using Multi-Media enhanced CPUs. In doing so it borrows concepts for expressing data parallelism that have a long history, dating back to Iverson's work on APL in the early '60s[13]. By an elegant algorithm I mean one which is expressed as concisely as possible. Elegance is a goal that one approaches asymptotically, approaching but never attaining[5]. APL and J allow the construction of very elegant programs, but at a cost. An inevitable consequence of elegance is the loss of redundancy. APL programs are as concise as, or even more concise than, conventional mathematical notation[14] and use a special character set. This makes them hard for the uninitiated to understand. J attempts to remedy this by restricting itself to the ASCII character set, but still looks dauntingly unfamiliar to programmers brought up on more conventional languages. Both APL and J are interpretive which makes them ill suited to many of the applications for which SIMD speed is required. The aim of Vector Pascal is to provide the conceptual gains of Iverson's notation within a framework familiar to imperative programmers.

Pascal[17] was chosen as a base language over the alternatives of C and Java. C was rejected because notations like `x+y` for `x` and `y` declared as `int x[4], y[4]`, already have the meaning of adding the addresses of the arrays together. Java was rejected because of the difficulty of efficiently transmitting data parallel operations via its intermediate code to a just in time code generator.

2 Array Mechanisms for Data Parallelism

Vector Pascal extends the array type mechanism of Pascal to provide better support for data parallel programming in general, and SIMD image processing in particular. Data parallel programming can be built up from certain underlying abstractions[8]:

- operations on whole arrays
- array slicing
- reduction operations

We will first consider these in general before moving on to look at how they are supported in Vector Pascal.

2.1 Operations on whole arrays

The basic *conceptual* mechanism is the *map*, which takes an operator and a source array (or pair of arrays) and produces a result array by mapping the source(s) under the operator. Let us denote the type of an array of T as $T[]$. Then if we have a binary operator $\omega: (T \otimes T) \rightarrow T$, we automatically have an operator $\omega: (T[] \otimes T[]) \rightarrow T[]$. Thus if x, y are arrays of integers $k = x + y$ is the array of integers where $k_i = x_i + y_i$:

$$\begin{bmatrix} 3 & 5 & 9 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 5 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 4 \end{bmatrix}$$

Similarly if we have a unary operator $\mu: (T \rightarrow T)$ then we automatically have an operator $\mu: (T[] \rightarrow T[])$. Thus $z = \text{sqr}(x)$ is the array where $z_i = x_i^2$:

$$\begin{bmatrix} 4 & 9 & 25 \end{bmatrix} = \text{sqr} \left(\begin{bmatrix} 2 & 3 & 5 \end{bmatrix} \right)$$

The map replaces the *serialisation* or *for loop* abstraction of classical imperative languages. The map concept is simple, and maps over lists are widely used in functional programming. For array based languages there are complications to do with the semantics of operations between arrays of different lengths and different dimensions, but Iverson[13] provided a consistent treatment of these. Recent languages built round this model are J, an interpretive language[15][3][16], High Performance Fortran[8] and F[20] a modern Fortran subset. In principle though any language with array types can be extended in a similar way. The map approach to data parallelism is machine independent. Depending on the target machine, a compiler can output sequential, SIMD, or MIMD code to handle it.

2.2 Array slicing

It is advantageous to be able to specify sections of arrays as values in expression. The sections may be rows or columns in a matrix, a rectangular sub-range of the elements of an array, as shown in figure 1. In image processing such rectangular sub regions of pixel arrays are called regions of interest. It may also be desirable to provide matrix diagonals[23].

1	1	1	1
1	2	4	8
1	2	4	16
1	2	8	512

1	1	1	1
1	2	4	8
1	2	4	16
1	2	8	512

1	1	1	1
1	2	4	8
1	2	4	16
1	2	8	512

Figure 1: Different ways of slicing the same array

2.3 Reduction operations

In a reduction operation, a dyadic operator injected between the elements of a vector, the rows or columns of a matrix etc., produces a result of lower rank. Examples would be forming the sum of a table or finding the maximum or minimum of a table. So one could use + to reduce

1	2	4	8
---	---	---	---

to $1+2+4+8=15$.

3 Data Parallelism in Vector Pascal

Vector Pascal incorporates Iverson's approach to data parallelism. Its aim is to provide a notation that allows the natural and elegant expression of data parallel algorithms within a base language that is already familiar to a considerable body of programmers and combine this with modern compilation techniques.

3.1 Assignment maps

Standard Pascal allows assignment of whole arrays. Vector Pascal extends this to allow consistent use of mixed rank expressions on the right hand side of an assignment. Given

```
r0:real; r1:array[0..7] of real; r2:array[0..7,0..7] of real
```

then we can write

```
r1:= r2[3]; { supported in standard Pascal }
```

```
r1:= 1/2; { assign 0.5 to each element of r1 }
```

```
r2:= r1*3; { assign 1.5 to every element of r2 }
```

```
r2[3]:= r2[4]+r1 ; { self explanatory }
```

The variable on the left hand side of an assignment defines an array context within which expressions on the right hand side are evaluated. Each array context has a rank given by the number of dimensions of the array on the left hand side. A scalar variable has rank 0. Variables occurring in expressions with an array context of rank r must have r or fewer dimensions. The n bounds of any n

dimensional array variable, with $n \leq r$ occurring within an expression evaluated in an array context of rank r must match with the rightmost n bounds of the array on the left hand side of the assignment statement.

Where a variable is of lower rank than its array context, the variable is replicated to fill the array context. This is shown in examples 2 and 3 above. Because the rank of any assignment is constrained by the variable on the left hand side, no temporary arrays, other than machine registers, need be allocated to store the intermediate array results of expressions.

Maps are implicitly and promiscuously defined on both monadic operators and unary functions. If f is a function or unary operator mapping from type r to type t then if x is an array of r then $a:=f(x)$ assigns an array of t such that $a[i]=f(x[i])$.

Functions can return any data type whose size is known at compile time, including arrays and records. A consistent copying semantics is used.

3.2 Operator reduction

Maps take operators and arrays and deliver array results. The *reduction* abstraction takes a dyadic operator and an array and returns a scalar result. It is denoted by the functional form $\backslash$. Thus if a is an array, $\backslash+a$ denotes the sum over the array. More generally $\backslash\Phi x$ for some dyadic operator Φ means $x_0\Phi(x_1\Phi\ldots(x_n\Phi 1))$ where 1 is the identity element for the operator and the type. Thus we can write $\backslash+$ for Σ , $\backslash*$ for $\prod$ etc. The dot product of two vectors can thus be written as

```
x:=+(y*z);
instead of
x:=0;
for i:=0 to n do x:= x+ y[i]*z[i];
```

A reduction operation takes an argument of rank r and returns an argument of rank $r-1$ except in the case where its argument is of rank 0, in which case it acts as the identity operation. Reduction is always performed along the last array dimension of its argument.

3.3 Operations on implicit indices

Assignment maps and reductions involve implicit indices. It can be useful to have access to these.

3.3.1 *iota*

The syntactic form *iota* i returns the i th current implicit index. Thus given the definitions

`v1:array[1..3]of integer; v2:array[0..4]of integer;`
 the program fragment `v1:=iota 0; v2:=iota 0 *2;`
 would set v1 and v2 as follows:

```
v1= 1 2 3
v2= 0 2 4 6 8
```

whilst given the definitions

`m1:array[1..3,0..4] of integer;m2:array[0..4,1..3]of integer;`
 then the program fragment
`m2:= iota 0 +2*iota 1;`
 would set m2

```
m2=
 2 4 6
 3 5 7
 4 6 8
 5 7 9
 6 8 10
```

The argument to `iota` must be an integer known at compile time within the range of implicit indices in the current context.

3.3.2 trans

The syntactic form `transx` transposes a vector¹ matrix, or tensor. It achieves this by cyclic rotation of the implicit indices. Thus if `trans e` is evaluated in a context with implicit indices

`iota 0.. iota n`
 then the expression `e` is evaluated in a context with implicit indices
`iota'0.. iota'n`
 where
`iota'x = iota ( (x+1)mod n+1)`

It should be noted that transposition is generalised to arrays of rank greater than 2.

Examples Given the definitions used above in section 3.3.1, the program fragment:

```
m1:= (trans v1)*v2;
m2 := trans m1;
will set m1 and m2:
```

¹ Note that `trans` is not strictly speaking an operator, as there exists no Pascal type corresponding to a column vector.

```
m1=
0 2 4 6 8
0 4 8 12 16
0 6 12 18 24
```

```
m2=
0 0 0
2 4 6
4 8 12
6 12 18
8 16 24
```

4 Unary operators

A unary expression is formed by applying a unary operator to another unary or primary expression. The unary operators supported are +, -, *, /, div, not, round, sqrt, sin, cos, tan, abs, ln, ord, chr, succ, pred and @.

Thus the following are valid unary expressions: -1, +b, not true, sqrt abs x, sin theta. In standard Pascal some of these operators are treated as functions. Syntactically this means that their arguments must be enclosed in brackets, as in sin(theta). This usage remains syntactically correct in Vector Pascal.

The dyadic operators +, -, *, /, div are all extended to unary context by the insertion of an identity element under the operation. This is a generalisation of the monadic use of + and - in standard Pascal where +a=0+a and -a = 0-a with 0 being the additive identity, so too 1/2 = 1/2 with 1 the multiplicative identity. For sets the notation -s means the complement of the set s. The implicit value inserted are given below.

type	operators	identity elem
number	+, -	0
set	+	empty set
set	-, *	fullset
number	*, /, div, mod	1
boolean	and	true
boolean	or	false

4.0.3 Dyadic operations

Dyadic operators supported are +, +:, -:, -, *, /, div, mod, **, pow, <, >, >=, <=, =, <>, shr, shl, and, or, in, min, max. All of these are consistently extended to operate over arrays. The operators **, pow denote exponentiation and raising to an integer power as in ISO Extended Pascal. The operators +: and -: exist to support saturated arithmetic on bytes as supported by the MMX instructionset.

5 Extensions to the Pascal Type System

5.1 Dynamic Arrays

A dynamic array is an array whose bounds are determined at run time. Operations on dynamic arrays are essential in general purpose image processing software since the size of an image loaded from a file may not be known at compile time.

Pascal 90[12] introduced the notion of schematic or parameterised types as a means of creating dynamic arrays. Thus where r is some integral type one can write

```
type z(a,b:r)=array[a..b] of t;
If p: ^z, then
new(p,n,m)
```

where $n,m:r$ initialises p to point to an array of bounds $n..m$. The bounds of the array can then be accessed as p^a , p^b . Vector Pascal incorporates this notation from Pascal 90 for dynamic arrays².

5.2 Indexed ranges

Image processing applications often have to deal with regions of interest, rectangular sub-images within a larger image. Vector Pascal extends the array abstraction to define sub-ranges of arrays. A sub-range of an array variable is denoted by the variable followed by a range expression in brackets.

The expressions within the range expression must conform to the index type of the array variable. The type of a range expression $a[i..j]$ where a : array[$p..q$] of t is array[$0..j-i$] of t .

```
Examples
dataset[i..i+2]:=blank;
twoDdata[2..3,5..6]:=twoDdata[4..5,11..12]*0.5;
```

Subranges may be passed in as actual parameters to procedures whose corresponding formal parameters are declared as variables of a schematic type. Hence given the following declarations:

² It should be noted that the use of schematic array types preserves the semantic distinction present in Standard Pascal, between assigning a pointer to an array, and assigning an array itself. This distinction is confused with the "open array" construct allowed by Delphi and Free Pascal.

```

type image(miny,maxy,minx,maxx:integer)=array[miny..maxy,minx..maxx]
of pixel;
procedure invert(var im:image);begin im:= - im; end;
var screen:array[0..319,0..199] of pixel;
then the following statement would be valid:
invert(screen[40..60,20..30]);

```

An array may be indexed by another array. If $x:\text{array}[t0]$ of $t1$ and $y:\text{array}[t1]$ of $t2$, then $y[x]$ denotes the virtual array of type $\text{array}[t0]$ of $t2$ such that $y[x][i]=y[x[i]]$. This construct is useful for performing permutations.

Example Given the declarations
`const perm:array[0..3] of integer=(3,1,2,0);`
`var ma,m0:array[0..3] of integer;`
 then the statements
`m0:= (iota 0)+1;`
`ma:=m0[perm];`
 would set the variables such that

```

m0= 1 2 3 4
perm= 3 1 2 0
ma= 4 2 3 1

```

5.3 Pixels

Standard Pascal is a strongly typed language, with a comparatively rich collection of type abstractions : enumeration, set formation, sub-ranging, array formation, cartesian product³ and unioning⁴. However as an image processing language it suffers from the disadvantage that no support is provided for pixels and images. Given the vintage of the language this is not surprising and, it may be thought, this deficiency can be readily overcome using existing language features. Can pixels not be defined as a subrange 0..255 of the integers, and images modelled as two-dimensional arrays?

They can be, and are so defined in many applications, but such an approach throws onto the programmer the whole burden of handling the complexities of limited precision arithmetic. Among the problems are:

1. When doing image processing it is frequently necessary to subtract one image from another, or to create negatives of an image. Subtraction and negation implies that pixels should be able to take on negative values.

³ The record construct.

⁴ The case construct in records.

2. When adding pixels using limited precision arithmetic, addition is nonmonotonic due to wrap-round. Pixel values of $100 + 200 = 300$, which in 8-bit precision is truncated to 44, a value darker than either of the starting values. A similar problem can arise with subtraction, for instance $100 - 200 = 156$ in 8-bit unsigned arithmetic.
3. When multiplying 8-bit numbers, as one does in executing a convolution kernel, one has to enlarge the representation and shift down by an appropriate amount to stay within range.

These and similar problems make the coding of image filters a skilled task. The difficulty arises because one is using an inappropriate conceptual representation of pixels.

The *conceptual model* of pixels in Vector Pascal is that they are real numbers in the range $-1.0 \dots 1.0$. This representation overcomes the aforementioned difficulties. As a signed representation it lends itself to subtraction. As an unbiased representation, it makes the adjustment of contrast easier, one can reduce contrast 50% simply by multiplying an image by 0.5^5 . Assignment to pixel variables in Vector Pascal is defined to be saturating – real numbers outside the range $-1..1$ are clipped to it. The multiplications involved in convolution operations fall naturally into place.

The *implementation model* of pixels used in Vector Pascal is of 8-bit signed integers treated as fixed point binary fractions. All the conversions necessary to preserve the monotonicity of addition, the range of multiplication etc, are delegated to the code generator which, where possible, will implement the semantics using efficient, saturated multi-media arithmetic instructions.

5.4 Dimensioned types

Dimensional analysis is familiar to scientists and engineers and provides a routine check on the sanity of mathematical expressions. Dimensions cannot be expressed in the otherwise rigorous type system of standard Pascal, but they are a useful protection against the sort of programming confusion between imperial and metric units that caused the demise of a recent Mars probe. They provide a means by which floating point types can be specialised to represent dimensioned numbers as is required in physics calculations. For example:

⁵ When pixels are represented as integers in the range $0..255$, a 50% contrast reduction has to be expressed as $((p-128) + 2) + 128$.

```

kms=(mass,distance,time);
meter=real of distance;
kilo=real of mass;
second=real of time;
newton=real of mass * distance * time POW -2;
meterpersecond = real of distance *time POW -1;

```

The identifier must be a member of a scalar type, and that scalar type is then referred to as the *basis space* of the dimensioned type. The identifiers of the basis space are referred to as the dimensions of the dimensioned type. Associated with each dimension of a dimensioned type there is an integer number referred to as the power of that dimension. This is either introduced explicitly at type declaration time, or determined implicitly for the dimensional type of expressions.

A value of a dimensioned type is a dimensioned value. Let $\log_d t$ of a dimensioned type t be the power to which the dimension d of type t is raised. Thus for $t = \text{newton}$ in the example above, and $d = \text{time}$, $\log_d t = -2$

If x and y are values of dimensioned types t_x and t_y respectively, then the following operators are only permissible if $t_x = t_y$: $+$, $-$, $<$, $>$, $=$, $<=$, $>=$. For $+$ and $-$, the dimensional type of the result is the same as that of the arguments. The operations $*$, $/$ are permitted if the types t_x and t_y share the same basis space, or if the basis space of one of the types is a subrange of the basis space of the other.

The operation POW is permitted between dimensioned types and integers.

Dimension deduction rules

1. If $x = y * z$ for $x : t_1, y : t_2, z : t_3$ with basis space B then $\forall_{d \in B} \log_d t_1 = \log_d t_2 + \log_d t_3$.
2. If $x = y / z$ for $x : t_1, y : t_2, z : t_3$ with basis space B then $\forall_{d \in B} \log_d t_1 = \log_d t_2 - \log_d t_3$.
3. If $x = y \text{ POW } z$ for $x : t_1, y : t_2, z : \text{integer}$ with basis space for t_2 , B then $\forall_{d \in B} \log_d t_1 = \log_d t_2 \times z$.

6 Implementation

The Vector Pascal compiler is implemented in Java. It uses the ILCG[7] (Intermediate Language for Code Generators) portable code generator system. A Vector Pascal program is translated into an abstract semantic tree implemented as a Java datastructure. The tree is passed to a machine-generated Java class corresponding to the code generator for the target machine. Code generator classes currently exist for the Intel 486, Pentium with MMX, and P3 and also the AMD K6. Output is assembler code which is assembled using the NASM

assembler and linked using the gcc loader. Vector Pascal currently runs under Windows98 , Windows2000 and Linux. Separate compilation using Turbo Pascal style units is supported. C calling conventions allow use of existing libraries.

The code generators follow the pattern matching approach described in[2][4]and [9], and are automatically generated from machine specifications written in ILCG . ILCG is a strongly typed language which supports vector data types and the mapping of operators over vectors. It is well suited to describing SIMD instructionsets. The code generator classes export from their interfaces details about the degree of parallelism supported for each data-type. This is used by the front end compiler to iterate over arrays longer than those supported by the underlying machine. Where supported parallelism is unitary, this defaults to iteration over the whole array.

Selection of target machines is by a compile time switch which causes the appropriate code generator class to be dynamically loaded.

7 Conclusions

Vector Pascal provides a new approach to providing a programming environment for multi-media instructionsets. It borrows abstraction mechanisms that have a long history of successful use in interpretive programming languages, combining these with modern compiler techniques to target SIMD instructionsets. It provides a uniform source language that can target multiple different processors without the programmer having to think about the target machine. Use of Java as the implementation language aids portability of the compiler across operating systems. Work is underway to port the BLAS library to Vector Pascal, and to develop an IDE and literate programming system for it.

References

- [1] Advanced Micro Devices, 3DNow! Technology Manual, 1999.
- [2] Aho, A.V., Ganapathi, M, Tjiang S.W.K., Code Generation Using Tree Matching and Dynamic Programming, ACM Trans, Programming Languages and Systems 11, no.4, 1989, pp.491-516.
- [3] Burke, Chris, J User Manual, ISI, 1995.
- [4] Cattell R. G. G., Automatic derivation of code generators from machine descriptions, ACM Transactions on Programming Languages and Systems, 2(2), pp. 173-190, April 1980.
- [5] Chaitin. G., Elegant Lisp Programs, in The Limits of Mathematics, pp. 29-56, Springer, 1997.

- [6] Cheong, G., and Lam, M., An Optimizer for Multimedia Instruction Sets, 2nd SUIF Workshop, Stanford University, August 1997.
- [7] Cockshott, Paul, Direct Compilation of High Level Languages for Multi-media Instruction-sets, Department of Computer Science, University of Glasgow, Nov 2000.
- [8] Ewing, A. K., Richardson, H., Simpson, A. D., Kulkarni, R., Writing Data Parallel Programs with High Performance Fortran, Edinburgh Parallel Computing Centre, Ver 1.3.1.
- [9] Susan L. Graham, Table Driven Code Generation, IEEE Computer, Vol 13, No. 8, August 1980, pp 25..37.
- [10] Intel, Intel Architecture Software Developers Manual Volumes 1 and 2, 1999.
- [11] Intel, Willamette Processor Software Developer's Guide, February, 2000.
- [12] ISO, Extended Pascal ISO 10206:1990, 1991.
- [13] Iverson K. E., A Programming Language, John Wiley & Sons, Inc., New York (1962), p. 16.
- [14] Iverson, K. E. . Notation as a tool of thought. Communications of the ACM, 23(8), 444-465, 1980.
- [15] Iverson K. E, A personal View of APL, IBM Systems Journal, Vol 30, No 4, 1991.
- [16] Iverson, Kenneth E., J Introduction and Dictionary, Iverson Software Inc. (ISI), Toronto, Ontario, 1995.
- [17] Jensen K., and Wirth N., Pascal User Manual and Report, Springer, 1978.
- [18] Krall, A., and Lelait, S., Compilation Techniques for Multimedia Processors, International Journal of Parallel Programming, Vol. 28, No. 4, pp 347-361, 2000.
- [19] Leupers, R., Compiler Optimization for Media Processors, EMMSEC 99/Sweden 1999.
- [20] Metcalf, M., and Reid., J., The F Programming Language, OUP, 1996.
- [21] Peleg, A., Wilke S., Weiser U., Intel MMX for Multimedia PCs, Comm. ACM, vol 40, no. 1 1997.
- [22] Srereman, N., and Govindarajan, G., A Vectorizing Compiler for Multimedia Extensions, International Journal of Parallel Programming, Vol. 28, No. 4, pp 363-400, 2000.
- [23] van der Meulen, S. G., Algol 68 Might Have Beens, SIGPLAN notices Vol. 12, No. 6, 1977.

Three Principles of Coding Clarity

by Stephen Taylor (sjt@lambenttechnology.com)

For most of my professional life I've heard that 80% of a system's lifetime costs are spent on maintenance. More strictly, on modifying and extending what it does. I don't know any references for this; I'm just going to take it as so.

The immediate implication is that the clarity of the software we write is probably its most important property, after accuracy and acceptable performance.

I've read more articles and heard more lore on APL coding practice than I care to remember. I've found many useful rules, but none so useful that I haven't also found an occasion when it seemed right to break it. So in this article I'm looking *behind* the rules-of-thumb for underlying *principles*, principles of coding clarity.

I'm not attempting here to derive practices from first principles. Instead I've been looking for a while at what I do that promotes clarity, and what I avoid that weakens it, and to induce from that some underlying principles. This is a venerable method. The mathematical forms of symbolic logic emerged not from *a priori* reasoning, but were induced and formalised from our intuitions about whether an argument is valid or not.

So with programming clarity. We all recognise clear code, and obscure code, and degrees of clarity in between. Discovering principles of clarity would look like this: we'd recognise that our good practices are informed by them, our bad practices contravene them, and they would also suggest further practices for writing clearly.

To my surprise, I've identified only three principles. I'd expected more. But I find them powerfully productive of ways to improve the clarity of my code. Here's an idea of the results I get from clear coding.

My present practice looks like this. I'll write an entire module of the system I'm working on, for example, a module that produces a report. During this time I'll occasionally test individual expressions, to ensure that some dense piece of array manipulation works the way I think it does. Mostly I just write. At a certain point, I'm satisfied that I've written a complete draft: I've written code for all the system's function. None of it is tested. This is about my half-way point. I now read all the code, looking for ways to clarify to the human reader what's going on. I might make several passes, reorganising, renaming, labelling fragments of code

as 'macro' functions – I'll come back later to these practices. I spend as much time reading and revising the code as I do writing it in the first place.

Now I'm nearly done. I'll run the code. Typically I'll encounter 2-5 typos I've overlooked, fix them and keep going. One or two test runs eliminate these oversights. Now I'm done. The code does what I think it does. Any remaining problems reflect my misunderstanding of the specification. They are not coding problems.

What's distinctive about this is that my practice ensures that the code does what I think it does. How can I know this? This is a point of some subtlety. I have seen work over the years on code-proving mathematics and software to demonstrate that software matches its specification. Nothing useful seems to have resulted. I can run tests to show that the code executes and that the output is as required. But in most cases the combinations of possible inputs are too large to test exhaustively. In the end my best assurance that the code is correct is that I can *see* that it is so. What would it take to be able to *see* that code is correct?

Now there is a level at which any APL programmer can see directly that a piece of code does what he thinks it does. To take an extreme example, I see, as you do, that the expression

$2+2$

adds 2 and 2 and will return 4. As we write code of greater and greater complexity, how can we retain, and how *much* can we retain, of this quality of clear, direct vision?

Before we proceed to look at that, let me make a disclaimer. I don't write perfect code. Even with careful reading and revising I miss typos. Sometimes, aspects of a specification elude me and I have to rethink and rewrite. My point is not that I write perfect code but that I know what my code does. 'Debugging' reveals and removes typos, that's all.

A couple of other disclaimers. Some people, horrified by obscure 'one-liners', have made a virtue of breaking all code up into 'easy pieces'. Not me: I love dense, fast, and viciously economical expressions. This is where we get to use APL's power. My practice is to write code so that where such expressions occur, *what* they do can be clear to the reader independently of *how* they do it. I'll have more to say on this under the heading of semantic domain.

The other disclaimer: it will sound in what follows as if I don't pay much attention to performance. That's true, I don't. PCs are fast enough these days that

most code runs acceptably fast. I optimise code only when performance is a problem. But I notice that clarifying and revising almost always reduces execution time.

The First Principle: Shorten Lines of Communication

Fantasise a moment. Imagine that you're a law-enforcement agent investigating a drug gang. What are its members up to? How do they take decisions, and how do they communicate them? When and where does money change hands? Where are goods moved and exchanged? Who knows what, and when and how do they know it?

In reading someone else's code, you make a similar investigation. Lord help me, in reading my *own* code I'm doing it – I can't remember from one week to another how everything works.

Now suppose you are a member of such a gang, under surveillance, and want to obscure all these matters from an investigator. What would you do? Why, you'd do what everyone knows to do. You'd use code words to disguise what you talk about. When you have to gather with your colleagues in a meeting room, while you talk, you'd also communicate covertly, passing notes beneath the table, hidden from unseen cameras. You'd use cutouts, so that you never exchange money for drugs, but had other people do that in places far from you. You wouldn't have important messages delivered to your home in the post; you would have them left in obscure places for covert collection by someone else, who would bring them to you discreetly.

Why the fantasy? In his book *Visual Explanations*⁶, Edward Tufte describes how, to examine what makes a process *clear* to an observer, he looked first at the practices of those dedicated to making it *obscure*. So in order to distinguish practices to promote clarity, he looked at what conjurors and illusionists do to defeat it.

The imaginary drugs gang has similar lessons for us. The practices distinguished above all have their analogues in coding. We can look at them to learn something about what makes code clear, what obscure.

The first analogue is between a defined function and the meeting room, wired for sound and vision. As the investigator, I'd like all communication in the room to

⁶ Edward R. Tufte, *Visual Explanations: images and quantities, evidence and narrative*, Graphics Press, Cheshire, Connecticut, 1997

be clearly expressed, to be about a single matter, and to be in plain sight – no notes under the table or left in hollow tree in the car park outside!

Central to all this is a notion of 'place'. We 'intuit' that matters handled in the same APL function are being handled in the same 'place'. Implicit in 'place' are concepts of closure and distance. This is encouraging: these are powerful concepts for theory construction⁷.

Distance is our second analogue. Here are some examples to check against your own intuitions. Data passed between functions as arguments and results are like money or drugs passed from hand to hand: the distances – in time and space – are short. Data assigned to a global variable to be read sometime later by some function is like a message left in a 'drop' in a hollow tree, to be picked up sometime later by someone: the separation of parties – in time and space – is great.

Naming a function or value saves it for use later and further away. It's a way to communicate across 'distance' – time and space. The nearer and sooner we use it, the more clearly our investigator sees what's going on. We have an interest here in shortening our communication paths, even if it's still unclear exactly what is shorter than what.

Let's try to formalise our intuitions about distances within the workspace by ranking different instances of distance.

The expression being executed is where the action is. So our first metric is distance within an expression. Consider the following two equivalent function lines.

```
[1] rows+2>pformat ◊ selection+(rowsp0 1)/\rows
```

```
[2] selection+(rowsp0 1)/\rows+2>pformat
```

It's clear that the distance in the second expression between setting and reading the value *rows* is less than in the first expression. Now consider a third equivalent.

⁷ Enclosure: "Distinction is perfect continence," wrote G. Spencer-Brown as his first assertion in *Laws of Form* (Crown, 1972) and went on to derive the fundamentals of mathematics. Distance: we're unable to even *think* about cause-and-effect without using distance as an analogue for time. Julian Jaynes: "history is impossible without the spatialization of time that is characteristic of consciousness." *The Origin Of Consciousness In The Breakdown Of The Bicameral Mind*, Houghton Mifflin, 1982, p. 251.

```
[3] selection+{(wp0 1)/\w}2>format
```

Here an unnamed function is used, carrying the value in its argument w , which is local to its function, and thus clearly extinguished when the function has been evaluated. Because it's clear that no further use will be made of this stored value, [3] is clearer than [2], where *rows* might have had some later use.

Our second intuition of 'distance' is between lines in a function. The more function lines separate the naming and the using of an object, the further apart they are, and the longer is the communication path. Any two name references in the same line of code are closer together than any two references on different lines.

Our third intuition of distance is between levels on the state indicator, or execution stack. References on different levels of the state indicator are further apart than references in the same function or operator.

These intuited distances seem to combine according to discernible rules. Consider: function F calls G , then H . G sets a variable a , and H reads it. (I do not recommend this design.)

```

      r←F y
[1]  G y
[2]  r←H

```

The distance between naming a in G and reading it in H is something like the sum of the distance in F between lines [1] and [2], and the 1-level differences in the state indicator between G and F , and H and F .

Shortening the line of communication increases clarity. Having G return a as a result, and H take it as an argument gives us:

```

      r←F y
[1]  r←H G y

```

And even better:

```
F←H◦G
```

where the distances between symbols F , G and H are even shorter. Here we see that the principle of shortening lines of communication at least recommends to us practices we already know promote clarity.

Shortening lines of communication underlies several well-known rules-of-thumb. We've already noted the good practice of passing values between functions as arguments and results.

We can also see that where named values are to be shared by functions that cannot pass them as arguments and results, we do well to localise those names as near as possible to where they are used. For example, if the functions G and H have to communicate through variable a , then it were better to localise a to F than to define it as a global variable. Thus a would be external to both G and H , but local to F . Our first principle explains why. Localising a to F doesn't itself shorten the line of communication, but it shows the reader that communication through a goes no deeper into the stack, and ends when F has been evaluated. That equates to an increase in clarity.

There is no difference in principle here between assigning names to data and assigning names to functions. We shall see some practices suggested by this insight after we have looked at the second principle.

The Second Principle: Semantic Consistency

Consider our investigator again. Let's suppose that he's a newcomer to the world of drugs, arms-smuggling and international finance. He could easily get confused listening to a conversation that switched between these subjects. Here's a danger: in the middle of a discussion of planned drug movements, someone wants to settle questions regarding the use of bank accounts in the Caribbean. Our poor agent could miss some important detail.

Luckily for him the gang bosses send aides into another conference room to work out the banking details. Our agent has a separate tape recorder in there; he will be able to review their conversation later. Both conversations will be clearer and he won't get confused between them.

The analogy with writing code is obvious, and the practice of 'subordinating detail' is well known. One has an intuitive sense of what is the 'main story' of a function. But here is a formalisation of it that yields new practices.

Consider the concept of *semantic domain*. Words belong to the same semantic domain when they are about the same subject.

A function has complete semantic consistency when all its referents (what its names refer to) belong to the same semantic domain. For example:

```

    document+Report record
[1] document+CoverLetter record
[2] document,+Page1 record
[3] document,+Page2 record
[4] document,+Page3 record

```

The function's referents—all but one—share a semantic domain. The referents are parts of a document that reports facts from *record*, which is the exception. The exception, the argument *record*, belongs to a different semantic domain. It contains facts about someone or something; it is not a part of a document.

The function *Report* describes a relationship between two domains, the facts and the document reporting them. As a rule-of-thumb, functions which relate two semantic domains are more interesting than functions with one. But adding semantic domains pays in clarity for any gain in interest.

There is something slightly misleading about the above. Semantic domains do not exist in nature, nor are they precisely individuated. The concept is useful only as a way to marshall and sharpen your intuitions about the organisation of your code.

Here is a powerful practice generated by the principle of semantic consistency: use names that relate to the semantic domain. That seems obvious, but Ray Cannon proposed a test of it which I have found an extraordinary source of clarity.

The Cannon Test:

Are there numeric constants in the body of the function?

If the answer is Yes, consider assigning the constants to local names that show the meaning of the value in the relevant semantic domain, then using the name instead.

For example (in a function that generates RTF code for tables in Microsoft Word documents) I set up a rank-3 control array of cell properties. Planes of the array correspond to properties like alignment, width, background shading and so on. The third plane specifies background shading, so

```
[10]    format[3;((2>pformat)p0 1)/;2>pformat;]+5
```

specifies a 5% background shading on alternate rows. Applying the Cannon Test led me to declare local names for the planes:

```
align width shading+1 2 3
```

which changed line 10 to

```
[10]    format[shading;((2>pformat)p0 1)/12>pformat;]+5
```

with a rise in clarity. *Shading* shares a semantic domain with *format*. But numeric constants remain. The reshape of 0 and 1 refers to a stripe effect I want on alternate rows of the table. Declaring a variable in the 'set up' of the function makes this clear:

```
alternateRows+((2>pformat)p0 1)/12>pformat
```

enjoys a semantic consistency of its own: *alternateRows* is a function of the size of *format*. We can clarify it by removing repetition:

```
alternateRows+{(wp0 1)/1w}2>pformat
```

Now line 10 gets even clearer:

```
[10]    format[shading;alternateRows;]+5
```

The last numeric constant is 5. Should that go too? Absolutely. The 5 denotes a 5% background shading. But in the world (semantic domain) of report appearances we don't make a hundred distinctions of shading. What we mean here is pale grey. So a declaration in 'set up' of

```
paleGrey+5 A 5% background shading
```

leaves us finally with

```
[10]    format[shading;alternateRows;]+paleGrey
```

Function code and web pages have something in common: they are rarely read, mostly skimmed. The kind of clarification achieved above aids skimming. Skim-reading the assignment to *alternateRows* without analysing it, one sees that its value is a function of the size of *format*, which seems right. On line 10, we can see what is specified for alternate rows (shade background 5%) without being distracted by how alternate rows are selected. (Compare the separation of the gangsters' two conversations.) Should we ever be concerned about how alternate rows are selected, we can examine it as an isolated proposition.

Here's an even more extreme example. In working with RTF, I convert between inches, half points, points and twips (twenty twips to a point; 72 points to an inch). To a programmer immersed in this subject, the numbers 12, 20, 2, 72 and

1440 are clear signs of units being converted. To a casual reader, or the next programmer struggling to master the subject, they're just numbers. So functions that convert units declare some local names:

```
insToTwips+1440*x ◊ twipsToPts+1◊20 ◊ ptsToHalves+2◊x
```

Even the humble *double* function (2◊x) here gets assigned a local name to clarify its semantics! Humble 1s and 0s suffer even more semantic ambiguity, which we can helpfully clarify:

```
do dont+on off+true false+1 0
```

Compare this practice with the alternative of adding comments. Comments are not always maintained with the code they relate to, and can get out of step with it. I systematically prefer to clarify code than to comment on it.

In the examples above, semantic consistency led us to reduce code density slightly: we added symbols and lines. This is not a general argument for reducing code density! If I seem to be belabouring the obvious, it is because the principle of semantic consistency leads me to vary code density in ways that are otherwise not at all obvious.

For example, in another function I combine specifications for the row properties of an RTF table with specifications for the cell properties. We're here manipulating arrays of RTF code strings with the sort of speed that makes us glad to have APL:

```
TABULATE+{
  ML+1
  rProps+PageWidth rowProperties α
  cProps+(cellProperties α){0=pa:'' ◊ α,GROUP w,'\cell '}'w
  ⍪rProps tableRow+ε"⏟[2] cProps
}
```

The original code is a 1-liner. I've relinedated it here for *Vector's* page width. Semantic consistency is complete and suggests no changes to improve clarity. The relinedation has lightened the burden of parsing a long line, but at the cost of slightly lengthening the lines of communication. I doubt that the 5-line version is any clearer than the 1-line version.

In this example, the code is just dense, that's all. It describes some moderately fancy array manipulations. Any difficulty in reading is the difficulty inherent in reading and envisaging the array transformations. The semantics of what is being done is already clear: table row properties are being combined with table cell properties. Style has nothing further to add here to clarity.

Would *TABULATE* be clearer if laid out in loops? It would—to a programmer who thinks in loops. To a programmer who thinks in arrays, a looped description is laborious and less clear. We write in APL because machines are good at manipulating arrays, and we gain mastery of their power by thinking and writing in arrays. A looped version of *TABULATE* would reduce both machine and programmer efficiency.

If this works for you, then the principle of semantic consistency can help you to use dense code without losing clarity.

The Third Principle: Don't Repeat Yourself

People who repeat themselves bore us and lose our attention. We then risk losing important detail in a flood of repetition.

Don't do that to others! This is such a general principle of programming that it hardly bears including. All programmers do this; arguably, it is what programming *is*.

I've included it as our third principle, because, combined with our first two principles it produces coding practices I value but have not seen in common use. One of these is extensive use of localised code fragments or 'macros'.

Macromania

Wherever I see a pattern repeated in code, I consider defining it as a function, named for semantic consistency and localised to minimise the communication path. Direct definition and the *composition* and *each* operators are powerful tools here.

We've already seen above the examples of the unit conversions and the pre-selection of alternate rows of a table. Even for one-time use these were worth giving local names to, if only as a substitute for comments.

The same trick is good for any recurring code fragment. For example, it's a simple piece of code to format a number as a sterling amount to include in text:

```
'up to ',(({v\w* ' ')/w),'P<£>CI12'∘FMT max),' a year'
```

I would declare at the beginning of my function:

```
stg←(({v\w* ' ')/w)∘,∘('P<£>CF12.2'∘FMT)
```

and use the local function *stg*

```
'up to ',(stg max),' a year'
```

So my application functions often begin by assigning names to some local functions and constants. (Note that this practice is as often indicated by semantic consistency as by avoiding repetition.)

Compare this practice with defining, naming and using a 'global' utility function for the whole workspace. The global utility will have more complex arguments to handle many more cases, will thus be less efficient, and will have some name that distinguishes it as belonging to the class of global utilities. In contrast, the macro has just enough code for the job at hand and is named for semantic consistency.

Control structures

Control structures like If-Then-Else and Select-Case lengthen communication paths by adding function lines. I use them rarely and happily to group blocks of code which are not minor variations of each other. But keeping communication paths short leads me to get If-Then-Else handled in a single line where possible.

Here are three handy macros and illustrations of their use:

```
IF+/" ◊ BUT+{×pw:w ◊ α} ◊ WITH+{α}
```

```
→0 IF 0=argument
```

```
→(0 WITH r+'') IF 0=argument
```

```
z+34 45 56 BUT 25 50 75 IF argument=3
```

```
herhim herhis+'her' 'her' BUT 'him' 'his' IF sex='M'  
txt,+'to consult ',herhim,' about ',herhis,' pension'
```

Shy arguments

Supplying a default for an omitted shy left argument seems too small a job to warrant branches or control structures. Try this D-function instead:

```
defaultsTo+{×[]NC α:±α ◊ w}
```

```
left+'left' defaultsTo 0
```

The use of *defaultsTo* has the virtue of making an explicit assignment to *left*, clear to be marked by an automatic code analyser.

Conclusion

Notice that our three principles often tug us in different directions! For example, naming a code fragment in a 'set up' section at the start of a function *lengthens* communication paths by separating definition from use. It is finally perhaps only an æsthetic judgement how to weigh the clarity gained by semantic consistency against that lost by introducing a new 'path of communication', even a short one, within a single function.

This, in my view, is why the many practices I've seen proposed as rules for clear code are fragile guides at best. In æsthetics it is helpful to distinguish principles, and futile to legislate.

These three principles are offered for sharpening your intuitions about the clarity of code. APL shares this with the natural languages: there are useful guides to writing clearly, but no formula for it. *De gustibus non disputandum est*.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner: Packages in Dyalog APL	Ray Cannon	126
Technical Correspondence	Norman Thomson	129
At Play with J: Second-order Josephus	Gene McDonnell	132
Hypergeometric Functions and CDFs in J	Ewart Shaw	139

Erratum

In Vector 18.3's *Hacker's Corner*, page 91, the top code paragraph was incorrect. It should have read:

```

      □SE Next 0
      □SE Next ''
1
      □SE Next ''
2

```

... with no dots in all three cases. Apologies from Production Dept.

Hacker's Corner: An Implementation of Packages in Dyalog APL in 95 chars or less.

by Ray Cannon (*ray_cannon @ compuserve.com*)

Extract taken from "SHARP APL vs. APL2 coding style" by Dan Baronet , Vector 16.2

Packages are unique to SAPL and are very useful. A series of system functions can be used with them, all of which start with `⊖p` as in `⊖pack` , `⊖pdef` , etc. For example, to define variable 'x' with objects (variables, functions) 'a' and <f> (for example to be stored on file) you would do

```
x←⊖pack 'a f'
```

Think of them as static namespaces (found only in Dyalog). You cannot "enter" a package and run something from within it. For example, if you wish to use its contents in a local environment you must first localize the names. For example, this function <execin> will execute an expression using a package's contents:

```
vr←exp execin pack;f
[1]⊖fx>('f',.,';',⊖pnames pack)⊃'⊖pdef pack ◊ r+',exp
v
```

OK, so Sharp's APL had packages and Dyalog does not. How can I emulate them in Dyalog APL/W?

Simple, represent a package as the `⊖OR` of a namespace.

```
Pack←{x←0 ◊ b←⊖EX'x' ◊ b←'x'⊖NS w ◊ ⊖OR'x'}
Pdef←{'##' (⊖NS'') .⊖NS w}
Pnames←{0::' ' ◊ w⊖⊖NL 19'}
```

That's it, according to MS Word all done in 95 characters!

Sample Usage

```

package←Pack +⊖nl 3

Pnames package
Pack
Pdef
Pnames

)erase Pack Pnames

Pdef packages

)fns
Pack Pdef Pnames

```

P

For readers not familiar with "DFNS" I will break then down line by line.

PACK

Pack is in four parts.

Part 3 creates a namespace "x" and copies the right hand argument into it.

```
Pack←{ ⋄ ⋄ b←'x'⊖NS w ⋄ }
```

Then Part 4 returns an Object Representation as the package.

```
Pack←{ ⋄ ⋄ ⋄ ⊖OR'x' }
```

Unfortunately, some "housekeeping" is required.

The name "x" has not been localised, so I can do this by assignment in Part 1.

```
Pack←{x←0 ⋄ ⋄ ⋄ }
```

Part 2 then expunged "x" so it can be re-used as a namespace.

```
Pack←{⋄ b←⊖EX'x' ⋄ ⋄ }
```

(From Dyadic on-line help on Create Namespace : "The namespace x is created if it doesn't already exist. If the name is already in use for an object other than a namespace, APL issues a DOMAIN ERROR.")

PDEF

Pdef copies the contents of the items in the package, into the current namespace.

```
Pdef←{' '⊔NS w}
```

Unfortunately, this fails if the current namespace is either # or ⊔SE, so I run the Create Namespace in a temporary un-named namespace, requesting to copy the items into the “outer” namespace defined by ##.

```
Pdef←{'##' (⊔NS ' ') .⊔NS w}
```

PNames

Pnames lists the contents of the “package”, by executing the Name List system function from within the “package” via dyadic execute.

```
Pnames←{ ⊔ w⊔'⊔NL 19' }
```

In case of error, Pnames returns an empty vector, via the error trap.

```
Pnames←{0::' ' ⊔ }
```

Challenge

Can anyone out there in Vector-land, re-write this in let's say 42 characters?

TECHNICAL CORRESPONDENCE

Rank and Extending the Domains

From: Norman Thomson

Phil Chastney's article on "Extending the Domains" (Vector Vol.18 No.3, pp.115-126) brings home with startling clarity the reasons why some of us have "defected" from APL to J. As Phil demonstrates, extending domains correctly in APL requires not only confidence in understanding a host of matters such as frame extensions, conformability, pervasiveness, splitting, joining, partially bound parameters and so on, but also presumes a knowledge of standards which arbitrarily adjudicate situations with plausible alternative interpretations, for example which should be the default axis. If I had known years ago that APL was this complicated I doubt if I would ever have touched it! By contrast, J domains are extended by correct application of the principles of noun rank and verb rank – that is *all* there is to say on the matter – my 16 words or Phil's 12 pages, take your choice. Never did Occam have his razor so finely honed!

I accept that the concepts of verb and noun rank are initially quite subtle, but what wonderful benefits they bestow! I am sure that in writing interpreters Roger Hui must occasionally have had to tussle with matters of axis conformability, partially bound parameters and so on, but these are really matters of implementation, and removing the burden of them from users is something for which the rest of us are eternally grateful since it lets us get on with things which are *really* interesting, like programming and problem solving (or even play)!

Phil's examples readily illustrate these points. First the J equivalent of `3 4 p 12` is `>:i.3 4` and is presumed to be assigned to `t` for convenience. J is often bracketed with other languages as an array language which I believe to be a misconception. J is first and foremost a *list-processing* language in which lists have the added feature of *rank*, a property which imposes uniformity on list lengths using fill items if necessary. For example, `t` above is not primarily a matrix but

three lists of 4-lists in a rank 2 formation. $< "1 \text{ } t$ is also three lists of lists but with noun rank 1, which means that there is no requirement for the lengths of the lists to be equal.

Now introduce the shift verb, and importantly observe (from the Dictionary) that its left verb rank is 1, which means that in what Phil would call "the foundation model", a scalar left argument is applied to a vector right argument. Put into J parlance, the rank of the right argument must be at least one more than the length of the left, and if the right argument is a scalar, its rank is extended as necessary, so that there is no inconsistency in this limiting case. Thus $1 \text{ } 2 \text{ } |. t$ means shift the lists themselves by 1 and items within each list by 2 so that the value of this expression is

```

7 8 5 6
11 12 9 10
3 4 1 2

```

$1 \text{ } 0 \text{ } 2 \text{ } |. t$ is a length error because it breaks the verb rank rule given above.

If I want to achieve Phil's APL goal of $1 \text{ } 0 \text{ } 2 \phi t$ then the appropriate action is to create a rank 1 list of lists as above and apply $|.$ bound with open to each of them thus:

```
1 0 2 |.<>" 1 t
```

With lists of higher ranks, exactly the same principles operate with no additional rules about conformability, pervasiveness, extension or anything other such paraphernalia. Because verb ranks are made explicit in the Dictionary, semantics are unambiguously defined across objects of all noun ranks, and the task of delivering right answers is shouldered entirely by the combination of language designer and interpreter writer – or perhaps I should say Ken and Roger.

I could address each of Phil's examples in the same way, but will restrict myself to one more example in which, incidentally, I think Phil mis-states the J interpretation.

```
(u=.2 3 $'abcdef'),'XY'
```

is a command to append two 3-lists to a 2-list. The semantics of append require the result to be three lists, which must possess rank 2 because u has rank 2. This means that a fill item is applied to the list 'XY' to give a resulting list :

```
abc
def
XY
```

Phil discusses two other possible ways of "joining XY to u" – these are

```
abcXY  and  abcX
defXY    defy
```

In the first case the list 'XY' is appended to each of the lists on the left, that is appending is applied at verb rank 1, viz. $u, "1 \text{ 'XY'}$. An analogy in either language is $2 \ 3+4$.

The second case is analogous to $2 \ 3+4 \ 5$, that is, there are two items on each side, rank 1 lists on the left and character scalars on the right, and so append is bonded with open in $(< "1 \ u), \> \text{'XY'}$.

Again all that is necessary is to apply the principles of noun and verb rank, no more than that. Perhaps it is worth noting that these ideas concerning rank were first observed in Sharp APL, so it is not too difficult to spot an evolutionary trail!

At Play With J: Second-order Josephus

by Eugene McDonnell

Every once in a blue moon this column is relatively easy to write. This one almost writes itself. Just a short while ago I received an intriguing message. It forms the bulk of this column. I've just had to change a word here and there and adjust typography as needed. Here's how the message opens:

From bantchev@math.bas.bg Thu Jan 17 18:07:53 2002
 Date: Fri, 30 Nov 2001 18:32:13 -0800 (PST)
 From: Boyko Bantchev <bantchev@math.bas.bg>
 To: forum@jssoftware.com
 Subject: Second-order Josephus

In a recent posting Eugene McDonnell defined the verb *s* that gives the survivor number for the Josephus problem (also in Vector 9/2 (1992)):

$S = . 1 \& | . \& . \# :$

Now suppose that, for n persons, $S(n)$ is *not* the survivor number, but the one to be *eliminated*; i.e., every second person in a circle is marked until only one remains -- and that one is eliminated.

This leads to a "second-order survival problem"; having eliminated $S(n)$, start again from the beginning with the remaining $n-1$ people, eliminate the one whose ordinal number *in the new sequence* is $S(n-1)$, then do the same with $S(n-2)$ and so forth until only one is left. What is the number $S_2(n)$ of the second-order survivor?

I must confess that my first reaction was somewhat guarded. I wasn't at all sure that this problem would lead to as much in the way of theory as the original Josephus did. For example, the book *Concrete Mathematics*, by Graham, Knuth, and Patashnik, devotes a full nine pages to Josephus, and gives half a dozen Josephus problems, in section 1.3. However, my mind was open, so I ploughed on. Bantchev's message continues:

The verb

$E = : ((< : @ [\{ .]) , } .) \sim S \#$

eliminates the $S\#y$ -th member from any list y , so, if:

$n = : 9$

and

```

]y=: 1+i.n
1 2 3 4 5 6 7 8 9

E y
1 2 4 5 6 7 8 9    NB. since 3=S 9
E^:2 y
2 4 5 6 7 8 9      NB. since 1=S 8
E^:3 y
2 4 5 6 7 8        NB. since 7=S 7
E^:4 y
2 4 5 6 8           NB. since 5=S 6
E^:5 y
2 4 6 8             NB. since 3=S 5
E^:6 y
4 6 8              NB. since 1=S 4
E^:7 y
4 6                NB. since 3=S 3
E^:8 y
6                  NB. since 1=S 2

```

Therefore, 6 survives ($6=S2(9)$).

In general, we can set

```
S2=: E^:(<:@#)@(>:@i.)
```

but, in fact, it can be shown that, for any $n > 1$, $S2(n)$ is $>: k+2^{<:m}$ when $k < 2^{<:m}$, and 2^m otherwise, where $m =: <.2^{<:n}$ (i.e. $(n>:2^m) * . n < 2^{>:m}$) and $k =: n - 2^m$. So, $S2$ can be defined without resorting to E or S :

```
S2 =: (>:@(1&,@({.+}.)@}.&#:.))"0
```

and we can check the definition:

```

S2 2+i.30
2 2 3 4 4 4 5 6 7 8 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16 16 16
16 16 16

```

To my regret $S2$ is, though clearly inspired by the definition of S , three times longer than S is, and not at all that elegant. But I did get some fun while writing it, hence my posting it to you.

/Boyko

I studied this message for a while and was confused. Mr. Bantchev gives a formula for $S2$ which seemed completely different from the immediately preceding formulas. I couldn't reconcile:

when $k < 2^{\leq m}$, $S2(n)$ is $>: k + 2^{\leq m}$

otherwise, 2^m

$$\begin{aligned} m &=: \leq 2^{\leq n} \\ k &=: n - 2^m \end{aligned} \quad (A)$$

with:

```
S2=: (>: @ (1 & , @ ( {. +. }. ) @ }. &. #: ) ) " 0
```

The leap was too great. I decided to go step by step until I had a better grasp of what was going on. I wrote this function, which follows Boyko's analysis faithfully:

```
Boyko =: monad define
n =. y.
m =. <. 2^ n
k =. n - 2^ m
if.
k < 2^ <: m
do.
>: k + 2^ <: m
else.
2^ m
end.
)
```

The argument to Boyko is an integer > 1 . It yields the 2nd-order Josephus survivor number of that integer:

```
Boyko"0 [ 2+i.30
2 2 3 4 4 4 5 6 7 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16 16
16 16 16 16 (B)
```

This agrees with the result of S2 in his message. The next step was to take his S2 apart, piece by piece. I'll repeat S2 here, so you can follow the steps:

```
S2=: (>: @ (1 & , @ ( {. +. }. ) @ }. &. #: ) ) " 0
```

Taking 19 as argument, convert it to binary:

```
#:19
1 0 0 1 1
```

Behead this:

```
    }.#:19
0 0 1 1
```

"Or" the head with the behead:

```
    ({.+}.).#:19
0 1 1
```

Prefix a 1:

```
    1,({.+}.).#:19
1 0 1 1
```

Find its base-10 value:

```
    #.1,({.+}.).#:19
11
```

Add 1:

```
    >#.1,({.+}.).#:19
12
```

This validates, but doesn't clarify, how S2 was put together. Since we know that Mr. Bantchev was trying to arrive at a solution that used the binary representations of numbers, I suspected that the answer might be found by looking at the binary values of argument and result side by side:

```
q:=2+i,20
(.,q);(#:q);(#:w);(.,w =: Boyko"0 q)
+-----+
| 2|0 0 0 1 0|0 0 1 0| 2|
| 3|0 0 0 1 1|0 0 1 0| 2|
| 4|0 0 1 0 0|0 0 1 1| 3|
| 5|0 0 1 0 1|0 1 0 0| 4|
| 6|0 0 1 1 0|0 1 0 0| 4|
| 7|0 0 1 1 1|0 1 0 0| 4|
| 8|0 1 0 0 0|0 1 0 1| 5|
| 9|0 1 0 0 1|0 1 1 0| 6|
|10|0 1 0 1 0|0 1 1 1| 7|
|11|0 1 0 1 1|1 0 0 0| 8|
|12|0 1 1 0 0|1 0 0 0| 8|
|13|0 1 1 0 1|1 0 0 0| 8|
|14|0 1 1 1 0|1 0 0 0| 8|
|15|0 1 1 1 1|1 0 0 0| 8|
|16|1 0 0 0 0|1 0 0 1| 9|
|17|1 0 0 0 1|1 0 1 0|10|
|18|1 0 0 1 0|1 0 1 1|11|
|19|1 0 0 1 1|1 1 0 0|12|
|20|1 0 1 0 0|1 1 0 1|13|
|21|1 0 1 0 1|1 1 1 0|14|
+-----+
```

It takes a bit of study, but it should be possible eventually to arrive at the S2 solution. Notice that the leading bit plays no role. The significant bit is the second. When this is 1, the result will be a power of 2, because the "or" of the second bit with the trailing bits will produce all 1s, prefixing a 1 keeps them all ones, converting this to integer will produce a result one less than a power of two, and adding one to this will yield a power of two. When the second bit is 0, the trailing bits are unaltered. When 1 is prefixed, the result will be a power of two only when the trailing bits are all 1, as in the case of 11. The binary form of 11 is 1 0 1 1; beheading gives 0 1 1; "or"ing 0 with 1 1 gives 1 1; prefixing 1 gives 1 1 1; converting to integer gives 7; adding one gives 8, a power of two.

I'm guessing that Mr. Bantchev followed a process much like the one I've described just above: finding the values with his algebraic analysis, converting these to binary, and comparing them with the binary form of the arguments. This is now enshrined in Sloane's *On-Line Encyclopedia of Integer Sequences* as sequence A066997.

I thought I had found an interesting property of (B). I noted the indices (in 2-origin) of the first appearance of a power of two were at indices 2, 5, 11, 23. I checked in the *Online Encyclopedia of Integer Sequences* and found that it corresponded to sequence A055010. The entry notes that these numbers, written in binary, are of the form $a(n)$ is 1011111 1. Furthermore, it gave the following formula for $a(n)$:

$$a(n) = (3 \cdot 2^n) - 1$$

I sent a message to Henry Bottomley, the author of this entry, and he in his reply alerted me to the existence of sequence A006165, which is:

1 1 2 2 3 4 4 5 6 7 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16
16 16 ...

This should be familiar, as it is the S2 sequence, with two leading 1s. I kicked myself for not having found this on my own, and began to appreciate that Mr. Bantchev might be on to something not completely trivial.

The entry for series A006165, gives two recursive formulas, one for odd n , and the other for even:

$$a((2 \cdot n) + 1) = a(n + 1) + a(n) \quad (A)$$

$$a(2 \cdot n) = 2 \cdot a(n) \quad (B)$$

By a bit of finagling it's possible to combine these two into one. The ceiling and floor of $(2*n)+1$ are $(n+1)$ and (n) , as in (A) above. The ceiling and floor of $2*n$ are both n , as in (B) above. This makes it possible, for whatever integer, to get the result by the same formula, which takes the sum of the ceiling and floor of half the number. The explicit function A below takes as argument the number of consecutive survivor numbers desired, and yields that many. For an argument of 0 it yields an empty list, and for an argument of 1 it yields a list whose item is 1.

```
A =: monad define
if.
  y. < 2
do.
  y. # 1
else.
  a =. 1 1
  while.
    y. > # a
  do.
    b =. ( < . , > . ) -: <: # a
    a =. a , + / b { a
  end.
end.
)
```

This duplicates A006165 faithfully. It differs from Bantchev's series in using offset 1. I found a much faster way to generate the sequence. It forms two lists, first just the 1-origin integers, then the extra powers of two needed:

```
(>:i./2^i.y.)
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
;(#each+:)2^i.y.
2 4 4 8 8 8 8 16 16 16 16 16 16 16 16
```

Joins these, and sorts them:

```
1 2 2 3 4 4 4 5 6 7 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16 1
6 16 16 16
```

Here is the high-speed Josephus 2 function:

```
hsJ2=: 13 : ' /:~(>:i./2^i.y.),;(#each+:)2^i.y.'
```

Its argument is the number of powers of 2 to use in generating the lists, and the result is a list as long as twice the sum of those powers of 2:

```

      2^i.4
1 2 4 8
  +/2^i.4
15
  +:+/2^i.4
30

      hsJ2 4
1 2 2 3 4 4 4 5 6 7 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16 1
6 16 16 16
  #hsJ2 4
30

```

Hypergeometric Functions and CDFs in J

by Ewart Shaw

Introduction

A valuable but little-mentioned feature of the J language is the conjunction *H.* with which *hypergeometric series* and hence many important mathematical functions can be constructed. A hypergeometric series has the general form

$${}_pF_q(a_1 \dots a_p; b_1 \dots b_q; z) = \sum_{n=0}^{\infty} \frac{(a_1)_n \dots (a_p)_n}{(b_1)_n \dots (b_q)_n} \frac{z^n}{n!},$$

where $(a)_n = a(a+1)\dots(a+n-1)$ for $n > 1$, and $(a)_0 = 1$. The equivalent J representation of ${}_pF_q(a_1 \dots a_p; b_1 \dots b_q; z)$ is $(a_1 \dots a_p \ H. \ b_1 \dots b_q) z$.

For example, ${}_1F_1(1; 1; \bullet)$ reduces to the exponential function, as shown in J by:

```
17j14 ": ^ i.3 NB. exp(0), exp(1), exp(2) to 15 digits
1.000000000000000 2.71828182845905 7.38905609893065
17j14 ": 1 H. 1 i.3 NB. or simply '' H. '' i.3
1.000000000000000 2.71828182845905 7.38905609893065
```

This article describes how the cumulative distribution functions (CDFs) of many common probability distributions can be expressed via hypergeometric series, making it easy to produce statistical tables and (for example) tail areas arising from *t*, chi-squared and *F* tests. Other examples using the hypergeometric conjunction *H.* are given in Iverson (1995), with particular reference to Graham, Knuth & Patashnik (1988) and Abramowitz & Stegun (1970). Another useful source for hypergeometric identities is Spanier & Oldham (1987).

The Error Function and the Normal CDF

The standard Normal distribution $N(0,1)$, with mean 0 and variance 1, has a CDF $\Phi(\bullet)$ easily expressed in terms of the *error function* $\text{erf}(\bullet)$:

```
erf=: (*&(%:4p_1) % ^@:*) * [: 1 H. 1.5 *:
n01cdf=: [: -: 1: + [: erf %&(%:2) NB. CDF of N(0,1)
```

On a PC, the resulting approximation has a maximum absolute error of roughly 10^{-14} , though the relative error might be judged unacceptable more than (say) seven standard deviations from the mean.

```

n01cdf _15 _8 _7 0 1.96 7 NB. accurate in range (-7,7)
8.88178e_16 2.16493e_15 1.28109e_12 0.5 0.975002 1
ncdf=: n01cdf : ([: n01cdf ]) - {.@([) % %:@{:@([)
5 4 ncdf 1 NB. Pr(X<1) where X~N(5,4)
0.0227501

```

For extreme values of z , the tail areas $\Phi(z)$ or $1-\Phi(z)$ are well approximated using continued fractions, see Abramowitz & Stegun. A simple such approximation is:

```

n01pdf=: (%: 0.5p_1)"_ * [: ^ *: % _2:
n01cdf2=: n01pdf * (1: - [: % *: +3:) % ]
n01cdf2 _20 _8 _7 NB. testing
_2.75362e_89 _6.22108e_16 _1.27986e_12

```

Note that other hypergeometric series formulae for the error function exist but suffer serious convergence problems. For example:

```

erfbad=: *&(%:4p_1) * [: 0.5 H. 1.5 -@: *:
14j11 ": erfbad 0.5 + 2*i.4
0.52049987781 0.99959304798 0.999999999883 3.53955177465
14j11 ": erf 0.5 + 2*i.4
0.52049987781 0.99959304798 0.999999999980 1.00000000000

```

The Incomplete Gamma and Beta Functions

The CDFs of gamma (including chi-squared) and Poisson distributions can be obtained from the *incomplete gamma function*, and CDFs of beta, t , F and binomial distributions from the *incomplete beta function*.

```

gamma=: ! & <: NB. gamma function
ig0=: 4 : '(1 H. (1+x.) % x.&(( * ^ ) * ( ^ -)~)) y.'
incgam=: ig0 % gamma@[ NB. incomplete gamma
beta=: *&gamma % gamma@+ NB. beta function
ib0=: 4 : '(((,-.) / x.) H. (1+{.x.) * ( ^ % ]) &({.x.)) y.'
incbet=: ib0 % [: beta/ [ NB. incomplete beta

```

```

      gamma 5 6 7 1.5
24 120 720 0.886227
      6 ig0 0 5 10 20 30
0 46.0847 111.95 119.991 120
      6 incgam 0 5 10 20 30
0 0.384039 0.932914 0.999928 1
      4 beta 2 3 4 1.5
0.05 0.0166667 0.00714286 0.101587
      4 4 2 4 beta 2 3 4 1.5
0.05 0.0166667 0.05 0.101587
      4 2 ib0 0 0.1 0.6 0.7 1
0 2.3e_5 0.016848 0.026411 0.05
      4 2 incbet 0 0.1 0.6 0.7 1
0 0.00046 0.33696 0.52822 1

```

NB. testing gamma...

NB. unnormalised

NB. testing beta...

NB. unnormalised

The interrelationships between these functions and various CDFs, illustrated below, are conveniently summarised in Press *et al.* (1986). As with the Normal distribution, extreme tail areas would require alternative methods.

```

NB. (n,p) bincdf y.      binomial(n,p)
NB. n chisqcdf y.      chi-squared on n d.f.
NB. (n1,n2) fcdf y.    F on (n1,n2) d.f.
NB. mu poissoncdf y.   Poisson, mean mu
NB. n tcdf y.          t on n d.f.

bincdf=: (({.@[ - ]) ,. >:@) incbet"1 _ -.@{:@[
chisqcdf=: incgam&-:
fcdf=: -:@[ incbet ({. % +/ )@(* ,:&1)
poissoncdf=: -.@incgam"0~ >:
tcdf=: [: -:@>: *@] * 1&,@[ fcdf *:@]

binpmf=: (!/ {.)~ * [: */ (, -.)@{:@[ ^ ] ,: {.@[ - ]
+/\ 5 0.2 binpmf i.6      NB. binomial CDF 'by hand'
0.32768 0.73728 0.94208 0.99328 0.99968 1
      5 0.2 bincdf i.6      NB. %_ for y.=n
0.32768 0.73728 0.94208 0.99328 0.99968 _
      5 chisqcdf 0.831211 4.35146 11.0705 20.515
0.025 0.5 0.95 0.999
      3 10 fcdf 0.84508 3.70826 6.55231 12.5527
0.5 0.95 0.99 0.999
      poissonpmf=: ^ * ^@-@[ % !@]
+/\ 2.3 poissonpmf i.5    NB. Poisson CDF 'by hand'
0.100259 0.330854 0.596039 0.799347 0.916249
      2.3 poissoncdf i.5
0.100259 0.330854 0.596039 0.799347 0.916249
      30 tcdf _1 0 1 2.04227 4
0.162654 0.5 0.837346 0.975 0.999809
      NB. table of t CDFs (using table adverb from stdlib)
      1 2 6 10 20 30 tcdf"0 _ table i.6

```

	0	1	2	3	4	5
1	0.5	0.75	0.852416	0.897584	0.922021	0.937167
2	0.5	0.788675	0.908248	0.952267	0.971405	0.981125
6	0.5	0.822041	0.953787	0.987996	0.996441	0.998774
10	0.5	0.829553	0.963306	0.993328	0.998741	0.999731
20	0.5	0.835372	0.970367	0.996462	0.999648	0.999966
30	0.5	0.837346	0.972687	0.997305	0.999809	0.999988

Implementation and Summary

Classes of probability distributions, including their CDFs, pseudorandom variable generation etc., can be created using J's object-oriented capabilities. Implementation details involve splicing together different approximations over different ranges, and iteration to approximate inverse CDFs such as $\Phi^{-1}(\bullet)$, but such details can mercifully be hidden from the user. The hypergeometric conjunction H. makes approximations over the regions of most interest easy in J. Hypergeometric series are also not difficult to implement in traditional APL and other languages, and the above methods deserve to be more widely known and used.

References

- Abramowitz, M. and Stegun, I.A. (1970). *Handbook of Mathematical Functions*, Dover, New York.
- Graham, R.L., Knuth, D.E., and Patashnik, O. (1988). *Concrete Mathematics*, Addison-Wesley, Reading, Massachusetts.
- Iverson, K.E. (1995). *Concrete Math Companion*, Iverson Software Inc., Toronto.
- Press, W.H., Flannery, B.P., Teukolsky, S.A., and Vetterling, W.T. (1986). *Numerical Recipes*, Cambridge University Press, Cambridge.
- Spanier, J. and Oldham, K.B. (1987). *An Atlas of Functions*, Springer-Verlag, Berlin.

Index to Advertisers

Dyadic Systems Ltd	2
Fred Honea	86
MicroAPL Ltd	12
Strand Software	8
Vector Back Numbers	7

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2001/2002 Committee

Chairman	Adrian Smith 01439-788385 apl385@compuserve.com	Brook House Gilling East YORK YO62 4JJ
Secretary	Anthony Camacho 0117-973-0036 acam@tesco.net	11 Auburn Road Redland BRISTOL BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road Waterbeach Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o APL Italiana Corso Vercelli 54 20145 - Milano Italy
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School Swansea University Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Road Blackwater, Camberley Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane YORK YO30 6DF
Education	Vacant post	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road Waterbeach Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Anthony & Sylvia Camacho (0117-973-0036), Ray Cannon (01252-874697), Stephen Taylor (077-1340-0852) Bob Hoekstra (01483-771028), Jon Sandles (01904-612882), Marc Griffiths	

Typeset by APL-385 with MS Word for Windows
Printed in England by Short-Run Press Ltd, Exeter

members in the UK and overseas. The British APL Association is a 501(c)(3) non-profit Computer Society, APL, Fortran, and Programming Language Society. The APL language noted for its speed, portability and fast development time is available on most mainframes, workstations, and on a wide range of computers.

PLANNING MEMBERS

The Committee of the Board and the Association wish to extend warmest wishes to the support of the following planning members in their efforts to advance the APL language and provide manpower to the Association.

Chesway Graphics Systems, Inc.
The Mallards, Castlepoint
MARTON, North York, England
Tel: 0653 659760
Fax: 01653 659779
Email: sales@chasegraphics.com
Web: www.chasegraphics.com

Draft Systems Ltd
Riverside View, Basildon
Old Basildon BASING, Essex
Hares, Redcliff, PA
Tel: 01268 811125
Fax: 01268 811110
Email: sales@draft.co.uk
Web: www.draft.co.uk

Draft Systems AGS
Northern Star, 1180
DK 8350, Newark, Essex
Dunston, PA
Tel: 01268 261121
Fax: 01268 261121
Email: sales@draft.co.uk
Web: www.draft.co.uk

Edgemoor Ltd
The Edgemoor Building
Edgemoor, 1180
DK 8350, Newark, Essex
Edgemoor, 1180
DK 8350, Newark, Essex
Edgemoor, 1180
DK 8350, Newark, Essex

Edgemoor Ltd
The Edgemoor Building
Edgemoor, 1180
DK 8350, Newark, Essex
Edgemoor, 1180
DK 8350, Newark, Essex
Edgemoor, 1180
DK 8350, Newark, Essex

Compass Ltd
Compass House
60 Peshaw Road
GLUDFORD, Surrey
Tel: 01483 514600

Draft APL Systems
Poultice, 1180
34184, Newark
Newark, PA
Tel: 01268 811125

Draft Systems
Northern Star, 1180
DK 8350, Newark, Essex
Dunston, PA
Tel: 01268 261121
Fax: 01268 261121
Email: sales@draft.co.uk
Web: www.draft.co.uk

Edgemoor Ltd
The Edgemoor Building
Edgemoor, 1180
DK 8350, Newark, Essex
Edgemoor, 1180
DK 8350, Newark, Essex
Edgemoor, 1180
DK 8350, Newark, Essex